UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

Scuola Politecnica e delle Scienze di Base

Dipartimento di Ingegneria Elettrica e delle Tecnologie dell'Informazione (DIETI)

Corso di Laurea Magistrale in Data Science

Tesi di Laurea Magistrale

Fact Validator

An Auditable and Deployment-Ready Architecture for Open-Domain Fact Verification

Master's Thesis — Final Research Manuscript

Completed: Academic Year 2025–2026

Relatore
Prof. Roberto Pietrantuono

Candidato
Santosh Kumar Yadav
Matricola: D03000048

Anno Accademico 2025–2026

Declaration of Completed Research

I confirm that this manuscript presents research that I have carried out as part of my Master's thesis in Data Science at the University of Naples Federico II.

The complete thesis-support apparatus is tracked on the repository's `main` branch. The final audit was executed from source commit `a6bbafc`; its generated report was then committed with the final manuscript. The exact environment, dependency-lock hash, benchmark hashes, commands, and captured test output are recorded in `data/benchmarks/results_5000/reproducibility_audit_report.json`. No Git release tag is claimed by this thesis. In particular, `thesis-v1.0.0` was only a proposed identifier and was never created. Historical `main` snapshots from before the thesis-correction merge do not contain these support artifacts.

I designed and implemented the FACT VALIDATOR architecture, prepared the multi-dataset evaluation workflow, executed the experiments reported in this document, and analysed the resulting predictive and operational behaviour.

This thesis is complete. The reported results constitute the final frozen experimental record for the submitted Master's study. The document distinguishes measurements that were executed from analyses that remain recommendations for subsequent research; no unexecuted experiment is presented as a completed result.

I present this document as the completed Master's thesis, but not as a claim of state-of-the-art performance. Its contribution is an implemented, evaluated, and auditable research system with a clear methodology, reproducible artifacts, bounded novelty claims, and documented limitations.

Scope of the final submission. This manuscript follows the DIETI/UNINA presentation conventions, formalizes the evaluation metrics as mathematical definitions, and presents the architectural and comparative diagrams required to interpret the study. The repository contains two separate evaluation surfaces. The FACT VALIDATOR live application implements open-web retrieval, domain credibility scoring, semantic reranking, persistence, and optional Ollama debate. The frozen 5,000-claim experiment instead evaluates the deterministic FACTVALIDATOR-PROXY, which combines lexical classification,

category-level priors, heuristic semantic signals, deterministic arbitration, and a quality filter. It does not execute SerpAPI, live domain scoring, SentenceTransformer reranking, or Ollama debate for every test claim. Chapter 9 documents another advanced module implemented in the same codebase. That evidence-graph module is a completed, independently verified prototype, but it is not part of the 5,000-claim proxy evaluation.

Abstract

In this thesis, I investigate how an open-domain fact-checking system can combine evidence retrieval, explicit source-credibility reasoning, reproducible benchmarking, and deployment-oriented engineering within a single auditable pipeline.

I have developed a working prototype called FACT VALIDATOR. The system accepts raw text or a URL, extracts fact-like claims, retrieves supporting and contradicting evidence, evaluates source credibility through an explicit and inspectable prior, semantically reranks the evidence, and produces claim-level verdicts. These verdicts are combined into a final misinformation-likelihood score, together with evidence summaries and confidence information.

The completed study uses a versioned evaluation workflow and a frozen 5,000-claim test split assembled from FEVER, LIAR, SciFact, and the PUBHEALTH `health_fact` dataset stored under a legacy healthver compatibility name. The full FACTVALIDATOR-PROXY reaches 50.82% accuracy and 0.4384 macro-F1. A proxy version without deterministic debate reaches 51.34% accuracy and 0.4427 macro-F1. An exploratory FEVER-aware proxy reaches 50.98% accuracy and 0.4421 macro-F1. These are the final proxy findings of the submitted experiment; they are not measurements of the live application and are not presented as proof of state-of-the-art superiority.

The proxy results also reveal a large raw-score confidence–accuracy gap. Because the stored score is heuristic and not a complete multiclass probability distribution, the result is reported as a raw-score calibration diagnostic rather than probability calibration.

Beyond the evaluated proxy, I implemented a typed, provenance-constrained evidence-graph prototype with a deterministic audit layer. Chapter 9 reports its final thesis status: the audited repository snapshot's 163-test backend suite passes and a small corruption-scenario probe demonstrates one concrete mechanism (source-independence clustering) by which it changes a decision relative to a simpler baseline. It remains intentionally separate from the proxy and therefore does not alter the 5,000-claim results.

Sommario (Italiano). In questa tesi analizzo come un sistema di verifica automatica dei fatti in dominio aperto possa combinare il recupero di evidenze testuali, un meccanismo esplicito e ispezionabile di stima dell'affidabilità delle fonti, un proto-

collo di valutazione riproducibile e un'ingegneria orientata al deployment. Ho sviluppato l'applicazione live FACT VALIDATOR e, separatamente, ho valutato il modello deterministico FACTVALIDATOR-PROXY su 5.000 affermazioni tratte da FEVER, LIAR, SciFact e PUBHEALTH. Il proxy completo raggiunge un'accuratezza del 50,82% e un macro-F1 di 0,4384; la variante proxy senza arbitraggio deterministico raggiunge il 51,34% e 0,4427. L'esperimento non esegue il recupero SerpAPI, il reranking SentenceTransformer o il dibattito Ollama per ogni affermazione.

Thesis Claim

The central claim is that an open-domain fact-verification system becomes more useful for human decision support when source trust, evidence retrieval, and verdict inference are explicit and inspectable rather than folded into one opaque score, and when confidence limitations are reported honestly. The completed study supports a bounded version of this claim through separate evidence streams: the live application is implemented and software-tested, while the deterministic proxy has a frozen 5,000-claim evaluation. Its small raw accuracy differences over majority and length baselines are not robust after Holm correction, and its heuristic score is strongly overconfident. A separate prototype module (Chapter 9) additionally demonstrates at small scale that a deterministic audit layer can force a safer outcome under correlated, same-domain evidence. These claims are final within the executed scope and are limited to the evidence reported in this thesis.

Contents

List of Tables

List of Figures

Chapter 1

Introduction

Misinformation detection is often treated only as a classification problem. However, I believe that a practical fact-checking system must solve a wider set of problems.

It must identify which statements can be verified, retrieve relevant evidence, distinguish supporting information from contradicting information, estimate the reliability of different sources, communicate uncertainty, and remain usable when external tools or models are unavailable.

A system may achieve a reasonable benchmark score while still being unsuitable for practical use if its evidence cannot be traced, its confidence is misleading, or its decisions depend on hidden assumptions.

For this reason, I approach fact verification as a combined machine-learning, information-retrieval, software-engineering, and trustworthy-AI problem. This chapter states the research question, the concrete objectives that follow from it, the evaluation questions used throughout the thesis, a summary of the completed work, and the final scope of each component.

1.1 Central Research Question

The central research question guiding my work is:

> Can I design an end-to-end fact-checking architecture that balances verification quality, source-level transparency, reproducibility, and deployment realism without hiding source-trust decisions inside a black-box model?

This question has three parts that recur throughout the thesis. *Verification quality* is addressed empirically in Chapter 6 through the deterministic FactValidator-Proxy benchmark. *Source-level transparency* is addressed architecturally in Chapter 4 through the live application's explicit credibility mechanism. *Deployment realism* is addressed

through the implemented application, bounded demonstrations, operational projections, and security posture. These evidence streams are complementary but are not merged into one empirical claim.

1.2 Research Objectives

I have defined the following objectives:

1. Design and implement a full-stack workflow for claim extraction and evidence-backed fact verification.
2. Represent source trust through an explicit and reviewable credibility mechanism.
3. Compare a deterministic proxy of the architecture with simple baselines and component-level ablations.
4. Evaluate proxy predictive quality and report bounded operational scenarios for the live application.
5. Preserve claims, evidence, predictions, and intermediate outputs to support reproducibility.
6. Measure whether optional components, particularly debate arbitration, provide enough benefit to justify their computational cost.
7. Identify uncertainty, calibration, data-quality, and validity limitations before making stronger scientific claims.
8. *(Added in this revision)* Formalize every metric used in the evaluation as an explicit mathematical definition, so that the empirical chapters can be read and checked independently of the code that produced them.

1.3 Evaluation Questions

I organize the present evaluation around five research questions.

RQ1: Predictive behaviour. How does the deterministic FactValidator-Proxy compare with majority, length, keyword, sentiment, and random baselines?

RQ2: Component contribution. How does removing deterministic debate arbitration affect proxy accuracy and macro-F1, and what separate latency scenario motivates selective rather than always-on live debate?

RQ3: Domain variation. How does the system behave across encyclopaedic, political, scientific, and health-related claims?

RQ4: Operational trade-offs. What latency and retrieval-cost trade-offs are introduced by optional debate and web-evidence retrieval?

RQ5: Prototype divergence *(scoped to Chapter 9 only).* For the advanced evidence-graph prototype: under which specific corruption mechanisms does its deterministic auditor change the system's decision relative to a graph-only baseline, and under which mechanisms is that effect currently masked by upstream relation-classification limitations? RQ5 is deliberately kept out of RQ1–RQ4, since the prototype is not part of the evaluated pipeline those questions describe.

1.4 Completed Contributions

The following contributions were completed within this thesis:

1. I designed a nine-stage end-to-end fact-verification architecture.
2. I implemented a working full-stack prototype using Next.js, FastAPI, SQLite, caching, retrieval services, and optional model components.
3. I developed an explicit source-credibility mechanism whose decisions can be inspected and modified without retraining the complete system.
4. I prepared a versioned evaluation workflow combining four heterogeneous fact-verification datasets.
5. I executed a frozen 5,000-claim evaluation of the deterministic FactValidator-Proxy and compared it with multiple baselines and variants.
6. I measured proxy predictive behaviour and produced explicitly labelled operational projections for latency, throughput, and retrieval cost.
7. I documented negative and inconclusive findings, including the lack of benefit from always-on proxy arbitration and the proxy's large raw-score confidence gap.
8. I implemented and independently verified an advanced evidence-graph prototype module, and documented precisely which safety mechanisms changed decisions and which remained masked by the current classifier (Chapter 9).
9. I formalized the mathematical definitions of every metric reported in the thesis (Chapter 3) and expanded the architectural, sequence, and comparative diagrams (Chapters 4, 9, and 10).

1.5 Final Research Scope

Table 1.1: Final status of the research components at thesis submission.

Research component	Final status	Comment
End-to-end architecture	Implemented	The complete nine-stage workflow is operational.
Full-stack prototype	Implemented	Next.js, FastAPI, SQLite, caching, retrieval, and reporting are integrated.
Frozen dataset split	Completed	The current split contains 19,983 normalized claims.
5,000-claim proxy evaluation	Completed	Exactly 5,000 aligned predictions per system, hashes, class metrics, domain metrics, paired tests, and aggregate results are recorded.
Baseline comparison	Completed	Majority, random, length, keyword, and sentiment baselines are included.
Component ablation	Completed within scope	The debate-free and reported frozen variants were evaluated; broader ablations are future work.
Confidence analysis	Completed descriptively	The confidence–accuracy gap establishes substantial overconfidence; post-hoc calibration is recommended.
Paired statistical testing	Completed	Exact two-sided McNemar tests, paired-bootstrap intervals, Holm correction, risk differences, and matched odds ratios are generated from the frozen prediction CSVs.
Evidence preservation	Completed at artifact level	Proxy splits, predictions, hashes, manifests, and generated reports are retained. Live retrieval records are separate and are not used as proxy evidence.
Evidence-graph prototype	Completed prototype	163/163 backend tests pass on the audited repository snapshot; it is intentionally reported separately from the proxy evaluation (Chapter 9).
Final thesis revision	Completed	This manuscript is the completed defence version.

1.6 Thesis Structure

The remainder of this thesis is organized as follows. Chapter 2 situates the work relative to prior literature. Chapter 3 gives the mathematical definitions of every metric used later. Chapter 4 describes the evaluated nine-stage architecture. Chapter 5 de-

scribes the dataset and experimental design. Chapter 6 reports the final frozen results. Chapter 7 discusses their meaning. Chapter 8 states limitations and threats to validity. Chapter 9 documents the completed evidence-graph prototype, precisely scoped as separate from the evaluated pipeline. Chapter 10 gives a structured comparative and novelty analysis against the closest prior work. Chapter 11 records final study closure and recommendations. Chapter 12 sketches research directions beyond this Master's thesis. Chapter 13 concludes. Appendices A–F provide the audit-rule inventory, test coverage, the corruption-probe construction script and output, real source-code listings, a glossary, and a summary of mathematical notation.

Chapter 2

Related Work

2.1 Automated Fact Verification

FEVER established a large-scale setting for claim verification using evidence from Wikipedia [1]. LIAR contains real political statements with fine-grained truthfulness labels [2]. SciFact focuses on scientific claim verification using evidence from research abstracts [3].

These datasets were created for different purposes and use different annotation schemes. Combining them creates methodological challenges, but it also allows me to investigate whether the architecture behaves consistently across different domains. Table 2.1 summarizes the three source datasets on the dimensions that matter for this thesis: label granularity, evidence format, and domain.

Table 2.1: Source datasets used to build the frozen benchmark, and their original design choices.

Dataset	Domain	Original labels	Evidence format
FEVER	Encyclopaedic (Wikipedia)	SUPPORTS / REFUTES / NOT ENOUGH INFO	Sentence-level, closed corpus
LIAR	Political statements	six-way truthfulness scale	No structured evidence; speaker/context metadata
SciFact	Scientific claims	SUPPORT / CONTRADICT / NEI	Rationale sentences from abstracts
Health-domain (health_fact)	Health claims	numerical veracity labels	Article-level, heterogeneous sources

2.2 Tool-Augmented Factuality Analysis

FacTool treats factuality detection as a modular process in which external tools are used to retrieve and verify information [4].

This approach is conceptually close to my work because I also avoid relying on a single language model to generate an unsupported verdict. My work differs in emphasis. I focus explicitly on source-credibility rules, persistent evidence storage, caching, service failure, bounded operational demonstrations and projections, and a user-facing misinformation-analysis workflow.

2.3 Faithful Factual Correction

Zero-shot Faithful Factual Error Correction studies the detection and correction of factual errors without task-specific retraining [5].

Its objective is primarily to correct text while preserving semantic faithfulness. My system instead focuses on open-domain verification, external evidence retrieval, source-trust assessment, and misinformation-risk analysis over free text and URLs.

2.4 Retrieval-Augmented AI

Retrieval-Augmented Generation demonstrates how external documents can be combined with parametric language models [6].

However, retrieval does not guarantee correctness. A retrieved document may be irrelevant, biased, outdated, duplicated, or unreliable. I therefore treat retrieval relevance and source reliability as separate problems: Section 4.6 (semantic reranking) addresses the first, Section 4.5 (credibility scoring) the second.

2.5 Calibration and Abstention

Modern machine-learning systems are frequently overconfident [7].

A system may assign high confidence to an incorrect prediction. This is particularly dangerous in misinformation analysis because users may interpret confidence as a guarantee. Research on abstention emphasizes that intelligent systems should recognize cases in which the available evidence is insufficient [8].

Confidence calibration and selective abstention are therefore important follow-up directions. Chapter 9's audit layer is a rule-based, non-learned form of the same idea:

rather than thresholding a scalar confidence score, it routes specific rule violations to distinct non-decisive outcomes. Section 9.7 returns to this connection and its limits.

2.6 Multi-Agent Debate

Multi-agent debate has been proposed as a method for improving factuality and reasoning by allowing different agents to challenge each other's arguments [9].

I implemented an optional live Prover–Skeptic–Judge workflow. However, I do not assume that debate is automatically beneficial. The proxy benchmark shows that its separate deterministic arbitration rule does not improve accuracy, while the live-system scenario projects a large latency overhead. These are distinct observations, not one controlled quality–latency experiment (Section 6.4).

2.7 Positioning Summary

Table 2.2: Research positioning of FACT VALIDATOR relative to the six literature threads above.

Dimension	FEVER/ LIAR/ SciFact	FacTool	Faithful correction	RAG	Calibration lit.	Debate lit.
Contributes	label vocabulary, benchmark data	modular tool orientation	faithfulness discipline	retrieval-plus-generation pattern	overconfidence framing	optional arbitration idea
FACT VALIDATOR adds	multi-dataset harmonization + domain breakdown	explicit, persisted, inspectable credibility prior	applies grounding to open-web verification, not correction	separates retrieval relevance from source reliability	documents a raw-score gap without claiming calibration	separates a deterministic proxy ablation from a live-debate cost scenario

Chapter 3

Mathematical Framework and Evaluation Metrics

This chapter collects, in one place, the exact mathematical definition of every quantity reported later in the thesis. Chapters 6 and 9 use these definitions without re-deriving them; this chapter is the single source of truth for what each number means.

3.1 Classification Metrics

Let $y_i \in \{\text{SUPPORTED}, \text{REFUTED}, \text{NEI}\}$ be the gold label for claim i and $\hat{y}_i$ the system's predicted label, over a test set of size n.

Accuracy.

$$\text{Accuracy} = \frac{1}{n} \sum_{i=1}^{n} \mathbf{1}[\hat{y}_i = y_i].$$

Per-class precision and recall. For class k, with TP_k, FP_k, FN_k the true positives, false positives, and false negatives for that class,

$$\text{Precision}_k = \frac{\text{TP}_k}{\text{TP}_k + \text{FP}_k}, \qquad \text{Recall}_k = \frac{\text{TP}_k}{\text{TP}_k + \text{FN}_k}.$$

Per-class F1 and macro-F1.

$$F1_k = \frac{2 \cdot \text{Precision}_k \cdot \text{Recall}_k}{\text{Precision}_k + \text{Recall}_k}, \qquad \text{Macro-F1} = \frac{1}{K} \sum_{k=1}^{K} F1_k,$$

where $K = 3$ is the number of classes. Macro-F1 weights every class equally regardless of its frequency, which is why it is reported alongside accuracy: a system can score well on accuracy while performing poorly on minority classes, and macro-F1 is designed to expose that gap (Section 6.3).

3.2 Confidence Intervals

For a binomial proportion such as accuracy, I report a 95% Wilson score interval. With $z = 1.96$, its center and half-width are

$$c = \frac{\hat{p} + \frac{z^2}{2n}}{1 + \frac{z^2}{n}}, \qquad h = \frac{z}{1 + \frac{z^2}{n}} \sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}},$$

so $\mathrm{CI}_{95\%} = [c - h, c + h]$, where $\hat{p}$ is observed accuracy and n the test-set size. This interval describes uncertainty in a single system's accuracy estimate; it does *not* by itself establish that two systems differ, which requires a paired analysis.

3.3 Confidence–Accuracy Gap

$$\mathrm{RawGap} = \left| \frac{\overline{\mathrm{score}}}{100} - \mathrm{Accuracy} \right|,$$

where $\overline{\mathrm{score}}$ is the proxy's average heuristic score on its stored 0–100 scale. Division by 100 puts the diagnostic on the same 0–1 scale as accuracy. I do not treat this quantity as Expected Calibration Error or probability calibration; it is a raw-score diagnostic because a full class-probability vector was not stored.

3.4 Expected Calibration Error and Brier Score (Defined for Follow-up Evaluation)

These metrics cannot be computed properly from the frozen proxy artifact because it stores one heuristic score rather than probabilities for SUPPORTED, REFUTED, and NEI. They are defined here so that their meaning is unambiguous once future predictions store a complete class-probability vector. Partition predictions into M confidence bins $B_1, \ldots, B_M$. Expected Calibration Error is

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{n} \left| \mathrm{acc}(B_m) - \mathrm{conf}(B_m) \right|,$$

where $\mathrm{acc}(B_m)$ and $\mathrm{conf}(B_m)$ are the average accuracy and average confidence of predictions falling in bin B_m. The Brier score, for a one-hot gold vector $\mathbf{y}_i$ and predicted probability vector $\mathbf{p}_i$ over the K classes, is

$$\mathrm{Brier} = \frac{1}{n} \sum_{i=1}^{n} \sum_{k=1}^{K} (p_{i,k} - y_{i,k})^2.$$

Both are strictly more informative than the confidence–accuracy gap because they are sensitive to the *distribution* of confidence, not only its average.

3.5 Exact Paired Significance Testing

For comparing two variants (e.g. full proxy vs. debate-free proxy) on the *same* test items, an exact two-sided McNemar test is appropriate for a binary correct/incorrect outcome. With n_{01} the number of items the first system gets wrong and the second gets right, and n_{10} the reverse, the exact test conditions on $n_{01}+n_{10}$ discordant pairs and evaluates both tails of a binomial distribution with probability 0.5. The familiar continuity-corrected large-sample statistic is

$$\chi^2 = \frac{(|n_{01} - n_{10}| - 1)^2}{n_{01} + n_{10}},$$

and may be compared against a χ^2 distribution with one degree of freedom as an approximation. The final artifacts use the exact binomial form, paired-bootstrap intervals with seed 42, Holm correction across comparisons, paired risk differences, and matched-pair odds ratios.

3.6 Selective Prediction: Coverage, Selective Risk, and False-Accept Rate

These three metrics are used exclusively in Chapter 9 to evaluate the evidence-graph prototype's abstention behaviour, reusing the standard selective-prediction formalism. Let $S_\theta \subseteq \{1, \ldots, n\}$ be the subset of claims for which the system returns a decisive verdict (as opposed to abstaining). *Coverage* is the fraction of claims on which the system is willing to decide:

$$\text{Coverage} = \frac{|S_\theta|}{n}.$$

Selective risk is the error rate restricted to that decisive subset:

$$R(\theta) = \frac{1}{|S_\theta|} \sum_{i \in S_\theta} \mathbf{1}[\hat{y}_i \neq y_i].$$

False-accept rate is the fraction of decisive predictions that are not merely wrong but wrong in the unsafe direction (e.g. a refuted claim accepted as supported):

$$\text{FAR} = \frac{1}{|S_\theta|} \sum_{i \in S_\theta} \mathbf{1}[\hat{y}_i = \text{SUPPORTED} \wedge y_i = \text{REFUTED}].$$

A good selective system should show $R(\theta)$ decreasing as coverage decreases – that is, the claims it chooses to abstain from should disproportionately be the ones it would otherwise have gotten wrong. Section 9.6 reports these three quantities for both the eight-case synthetic seed and the extended corruption probe.

3.7 Source-Independence Clustering (Jaccard Similarity)

Used in the credibility and evidence-graph components (Sections 4.5 and 9.4) to detect duplicated or syndicated evidence. For two evidence items e_i, e_j with base domains d_i, d_j and token sets T_i, T_j,

$$\text{linked}(e_i, e_j) = [d_i = d_j] \ \vee \ \left[\frac{|T_i \cap T_j|}{|T_i \cup T_j|} \geq \tau\right],$$

with threshold $\tau = 0.78$ in the shipped configuration. Connected components of this relation form independence clusters (Section 9.4).

3.8 Credibility Score

$$\text{Credibility}(d) = \text{clip}_{[0,100]}\big(50 + b(d) + p(d)\big),$$

where d is the domain of an evidence source, $b(d)$ a sum of rule-based bonuses, and $p(d)$ a sum of rule-based penalties (Section 4.5).

3.9 Operational Metrics

Throughput is the reciprocal of mean per-claim latency, scaled to an hourly rate:

$$\text{Throughput (claims/hour)} = \frac{3600}{\overline{\text{latency}}\ \text{(s/claim)}}.$$

Debate overhead is the ratio of mean latency with debate enabled to mean latency without it:

$$\text{Overhead} = \frac{\overline{\text{latency}}_{\text{debate}}}{\overline{\text{latency}}_{\text{baseline}}}.$$

These formulas are used in Section 6.6 with scenario assumptions of 8.20 s/claim and 72.00 s/claim. The resulting throughput and overhead figures are projections, not controlled load-test measurements.

3.10 Complexity Bounds (Prototype Module)

For completeness, the asymptotic bounds governing the Chapter 9 prototype: claim decomposition and passage scoring are linear in the number of input and page sentences; pairwise independence comparison is $O(q^2 v)$ for q evidence items and average token-set comparison cost v; graph audit is linear in nodes and edges after clustering. These bounds are not used elsewhere in the thesis but are recorded here as part of the single reference chapter for all formal claims.

Chapter 4

System Architecture

4.1 Architecture Overview

I designed FACT VALIDATOR as a nine-stage architecture that separates evidence retrieval, source-trust reasoning, model inference, optional debate, final scoring, and persistent auditing. Figure 4.1 gives the pipeline view; Figure 4.2 gives a request-level sequence view of the same architecture, showing which component calls which for a single `/analyze` request; Figure 4.3 gives a data-oriented view, showing what is persisted at each stage rather than which function calls which.

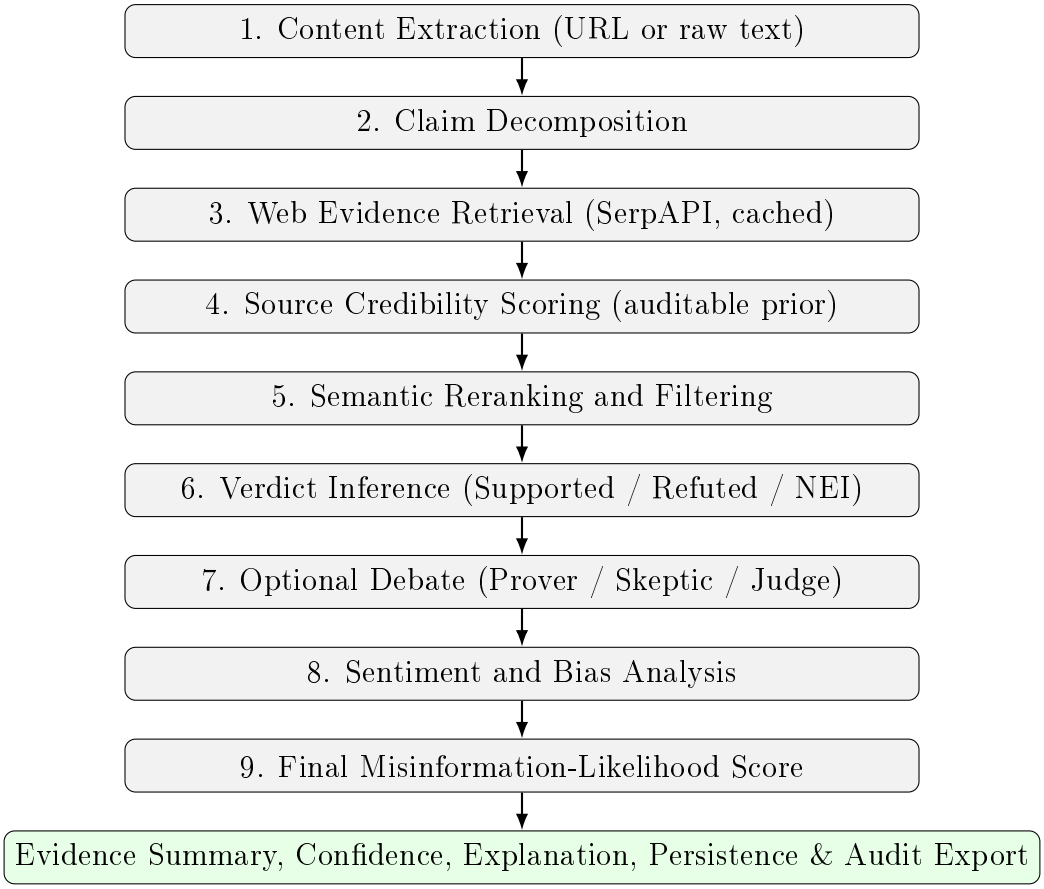

Figure 4.1: Pipeline view: end-to-end architecture of FACT VALIDATOR. Stage 7 (debate) is bypassable; Stage 4's credibility prior is explicit and inspectable rather than learned end-to-end.

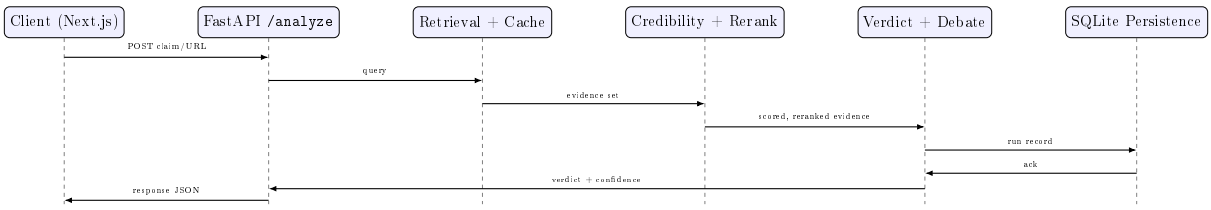

Figure 4.2: Sequence view of a single `POST /analyze` request, showing component interaction order.

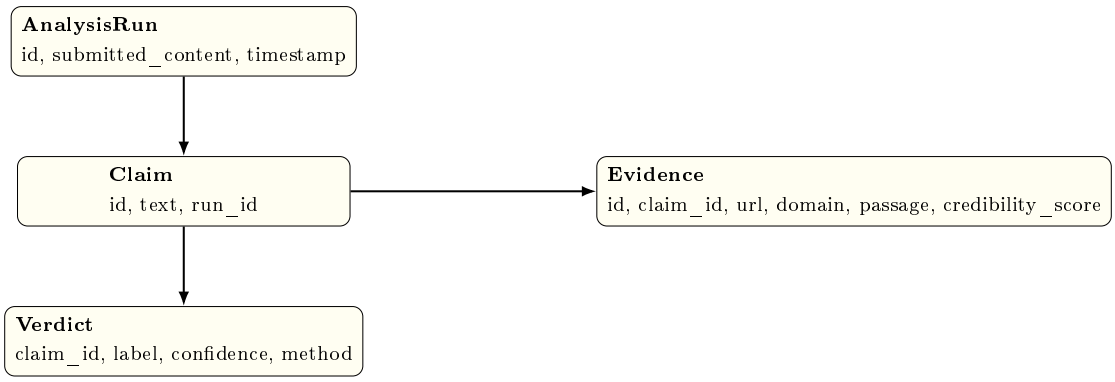

Figure 4.3: Data view: persisted entities per analysis run (SQLite schema, simplified).

4.2 Content Extraction

I allow users to submit raw text or a URL. For a URL, the extraction layer retrieves the page, removes irrelevant structural elements, and prepares the remaining content for claim decomposition. For raw text, the system applies normalization directly.

4.3 Claim Decomposition

Long documents may contain several factual statements. I therefore decompose the input into smaller claims that can be verified independently. This is important because one article may contain a mixture of supported, refuted, and unverifiable statements.

4.4 Evidence Retrieval

For each claim, I create retrieval queries and obtain candidate evidence from the web. The current implementation uses SerpAPI as the external search interface. Because web results change over time, I store retrieval outputs and use caching to reduce repeated requests.

4.5 Auditable Source-Credibility Scoring

I designed the source-credibility component to remain explicit rather than fully hidden inside a learned model. The score is defined in Section 3.8. The submitted system uses the documented rule weights; calibration against independently annotated source-evidence pairs is recommended as post-thesis work.

The advantage of this approach is not that the rules are automatically correct. The advantage is that each trust decision can be inspected, challenged, and modified without retraining the complete system. Figure 4.4 shows the decision path for a single domain score.

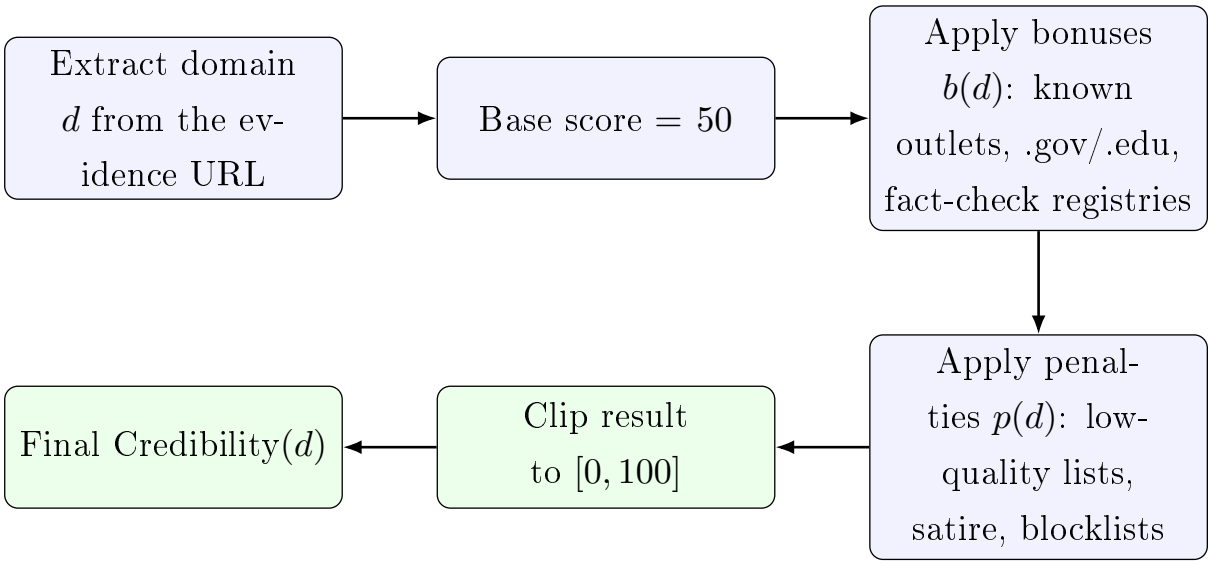

Figure 4.4: Decision path for the credibility score of a single domain. The path reads left-to-right on the top row, then continues right-to-left on the bottom row.

4.6 Semantic Reranking

Retrieval rank does not necessarily represent evidential relevance. I therefore use semantic similarity and quality filtering to reorder candidate documents according to their relationship with the claim.

4.7 Verdict Inference

The system assigns one of three claim-level outcomes: `supported`, `refuted`, or `not enough information`. I preserve the evidence and confidence associated with each decision so that the result remains inspectable.

4.8 Optional Debate Arbitration

For difficult claims, the system can activate a Prover–Skeptic–Judge workflow. The Prover builds the strongest supporting interpretation. The Skeptic searches for contradictions, weak evidence, and unsupported assumptions. The Judge compares the two

arguments and produces an arbitration result.

The final experiments indicate that debate should not be enabled for every claim (Section 6.4). Selective debate activation based on low confidence, evidence disagreement, component conflict, source-risk variance, or insufficient evidence is therefore retained as a deployment recommendation rather than reported as an executed result.

4.9 Sentiment and Bias Analysis

As Stage 8, the system separately analyses the emotional tone and lexical bias markers of the submitted content, independent of the factual verdict. This signal is combined into the final misinformation score (Stage 9) but does not itself alter the SUPPORTED/REFUTED/NEI verdict, since sentiment is not evidence.

4.10 Final Scoring and Persistence

I combine claim-level outcomes, source credibility, evidence quality, and contextual signals into a final misinformation-likelihood score. The system stores: submitted content; decomposed claims; retrieval queries; evidence sources; credibility scores; claim-level predictions; confidence values; final risk scores; and timestamps and run metadata. This persistent record supports auditing, debugging, and later evaluation.

4.11 Security Posture

The retrieval layer validates URLs before fetching and rejects private, loopback, link-local, and otherwise non-public network destinations, so that user-submitted URLs cannot be used to probe internal infrastructure. This is a deployment necessity as much as a research concern: an open-domain retrieval service that accepts arbitrary URLs is, by construction, exposed to server-side request forgery unless it validates targets before every fetch and every redirect.

Chapter 5

Dataset and Experimental Design

5.1 Final Frozen Benchmark

I created the final frozen evaluation family by combining claims from FEVER, LIAR, SciFact, and a normalized health-domain dataset. After normalization and exact-duplicate removal, the dataset contains 19,983 unique claims, divided into 11,986 training claims, 2,997 validation claims, and 5,000 test claims.

5.2 Dataset Composition

Table 5.1: Final benchmark composition and frozen label harmonization.

Slot	Source	Imported	Test	Canonical mapping
FEVER	FEVER v1.0	8,000	1,829	SUPPORTS → SUPPORTED; REFUTES → REFUTED; NOT ENOUGH INFO → NEI
LIAR	LIAR	6,000	1,490	true / mostly-true → SUPPORTED; false / pants-fire → REFUTED; half-true / barely-true → NEI
SciFact	SciFact claims	1,200	191	SUPPORT → SUPPORTED; CONTRADICT → REFUTED; insufficient evidence → NEI
Health domain	PUBHEALTH health_fact	6,000	1,490	Source labels mapped to SUPPORTED, REFUTED, and NEI by the frozen normalization script; stored under the legacy `healthver` compatibility filename.

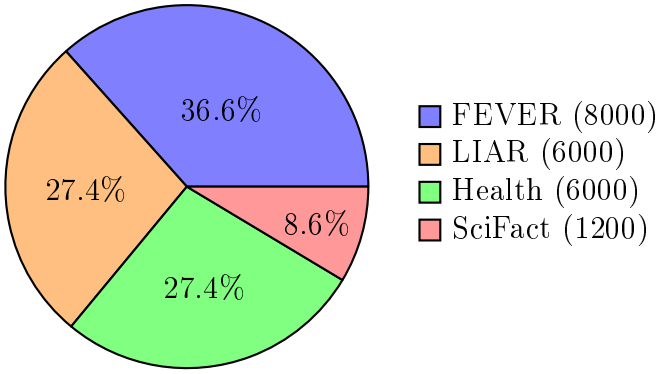

Figure 5.1: Composition of the imported (pre-split) dataset by source, before the 60/15/25 train/val/test partition described in Section 5.3. (Figure rendered from the exact import counts in Table 5.1.)

5.3 Split and Leakage Control

I normalized claim text before splitting the dataset. I removed exact normalized duplicates and withheld the normalized test claims from the remaining training and validation pool. This reduces direct claim leakage across partitions. Semantically similar or paraphrased claims may still exist; this residual risk is recorded as a limitation because embedding-based overlap analysis was outside the final experimental scope.

5.4 Evidence Reproducibility

The proxy claims, labels, predictions, and aggregate results are frozen in archived artifacts. The run manifest records SHA-256 hashes of all split and prediction files. The proxy experiment does not retrieve live web evidence. The separate live application does retrieve dynamic web documents; those application demonstrations are not used to produce the 5,000-claim proxy metrics.

5.5 Evaluated Systems

Full FactValidator-Proxy. A deterministic proxy combining a trained lexical model, category-level priors, heuristic semantic signals, deterministic debate arbitration, and a quality filter.

FactValidator-Proxy without debate. The same deterministic proxy with its arbitration rule disabled.

FEVER-aware exploratory proxy. A dataset-aware exploratory adjustment applied to FEVER-labelled claims.

Majority baseline. A classifier that always predicts the most frequent class.

Length heuristic. A rule based on claim-length patterns.

Keyword heuristic. A rule based on predefined lexical signals.

Sentiment heuristic. A rule based on sentiment-related signals.

Random baseline. A non-informative reference classifier.

5.6 Metrics

I report the metrics formally defined in Chapter 3: accuracy, macro-F1, Wilson 95% confidence intervals, raw-score confidence diagnostics, per-dataset accuracy, latency, throughput, and projected retrieval cost. Exact two-sided McNemar tests, paired-bootstrap intervals, Holm correction, paired risk differences, and matched-pair odds ratios are generated from the aligned prediction files. A proper multiclass Brier score is unavailable because the proxy stores only one heuristic confidence score rather than three class probabilities.

Chapter 6

Final Experimental Results

6.1 Interpretation Policy

This chapter reports the final frozen results of the deterministic FactValidator-Proxy. They do not characterize end-to-end execution of the live Fact Validator application and do not establish state-of-the-art performance. Exact paired analyses are reported, but marginal unadjusted comparisons against simple baselines are not treated as robust superiority after multiple-comparison correction.

6.2 Final Model Performance

Table 6.1: Final descriptive performance on the frozen 5,000-claim test set. Every figure was independently re-verified against `ablation_study_report.json` in the project repository.

System	Accuracy	Macro-F1	95% CI (Sec. 3.2)	Avg. raw score	Raw gap
FactValidator-Proxy without deterministic debate	51.34%	0.4427	[49.95, 52.73]%	88.37%	37.03 pp
FEVER-aware exploratory proxy	50.98%	0.4421	[49.59, 52.37]%	88.53%	37.55 pp
Full FactValidator-Proxy	50.82%	0.4384	[49.43, 52.21]%	88.31%	37.49 pp
Length heuristic	49.42%	0.3483	[48.03, 50.81]%	48.83%	0.59 pp
Majority baseline	49.26%	0.2200	[47.87, 50.65]%	50.00%	0.74 pp
Random baseline	33.20%	0.3240	[31.89, 34.51]%	50.00%	16.80 pp
Keyword heuristic	23.54%	0.1364	[22.36, 24.72]%	23.62%	0.08 pp
Sentiment heuristic	23.78%	0.1400	[22.60, 24.96]%	40.31%	16.53 pp

6.3 Final Interpretation

The full FactValidator-Proxy exceeds the majority baseline by 1.56 percentage points in accuracy. Using the macro-F1 definition of Section 3.1, the macro-F1 difference is $0.4384 - 0.2200 = 0.2184$. This suggests that the proxy distributes some correct decisions more evenly across labels than a majority classifier.

The exact unadjusted McNemar p-values are 0.0433 against majority and 0.0462 against length. Because several systems are compared, these marginal results are not described as robust superiority after Holm correction. The generated report also provides paired-bootstrap intervals, matched-pair odds ratios, class metrics, and the full confusion matrix.

6.4 Effect of Removing Debate

The no-debate proxy produces the highest observed accuracy and macro-F1. The exact paired comparison with the full proxy gives $p = 0.00955$ before Holm correction and favours removing always-on deterministic arbitration. This result concerns the proxy rule, not live Ollama debate. Selective live debate for uncertain or conflicting claims remains a recommendation requiring a separate controlled experiment.

6.5 Per-Dataset Behaviour

Table 6.2: Per-dataset accuracy of the full deterministic FactValidator-Proxy.

Dataset	n	Full	Majority	Length	Full–Majority
FEVER	1,829	57.52%	58.56%	57.90%	−1.04 pp
Health domain	1,490	55.17%	51.54%	52.55%	+3.62 pp
LIAR	1,490	38.59%	36.58%	36.98%	+2.01 pp
SciFact	191	48.17%	41.36%	40.84%	+6.81 pp

The system shows positive descriptive gains on the health-domain, LIAR, and SciFact subsets. It performs slightly below the majority and length baselines on FEVER. This variation indicates that the system is sensitive to domain, claim style, label distribution, and available evidence.

6.6 Operational Evaluation

Table 6.3: Operational scenario projections (formulas in Section 3.7); these are not controlled load-test measurements.

Metric	Value	Current interpretation
Baseline latency assumption	8.20 s/claim	Scenario input from the bounded demonstration
Debate latency assumption	72.00 s/claim	Scenario input from the bounded demonstration
Debate latency ratio	8.78×	Derived scenario ratio
Baseline throughput	439.02 claims/hour	Derived estimate
Debate throughput	50.00 claims/hour	Derived estimate
Monthly retrieval cost without caching	USD 77.00	Projected under current workload
Monthly retrieval cost with caching	USD 22.00	Projected under current workload
Estimated monthly savings	71.43%	Projected reduction

The operational scenarios project substantial latency for always-on live debate and lower external-search cost under caching. Controlled repeated load tests are required before treating either quantity as measured deployment performance.

6.7 Expanded Reading of the Final Results

6.7.1 Purpose of the Expanded Reading

This section consolidates the numerical results from Chapter 6 into a single, thesis-level interpretation. It does not replace or remove any of the reported artifacts. Instead, it explains what can reasonably be learned from them, which conclusions are supported only descriptively, and how the findings answer the research questions introduced earlier in the thesis. This distinction is important because a completed study is not the same thing as a study that claims every possible experiment. The experimental record is complete for the declared scope: 5,000 claims, four dataset families, the reported system variants and baselines, and the operational projections listed in Table 6.3.

6.7.2 Reading Accuracy Together with Macro-F1

Accuracy and macro-F1 describe different aspects of performance. Accuracy counts the proportion of all claims assigned the correct final label, whereas macro-F1 gives equal importance to the class-specific F1 values before averaging them. The distinction matters in the present benchmark because the class distribution is not perfectly balanced. A system can obtain a competitive accuracy by favoring a frequent class while performing poorly on less frequent classes.

The majority baseline illustrates this effect. Its accuracy is 49.26%, but its macro-F1 is only 0.2200. The full FactValidator-Proxy reaches 50.82% accuracy and 0.4384 macro-F1. The absolute accuracy difference is therefore 1.56 percentage points, while the macro-F1 difference is 0.2184. The latter is the more substantial descriptive contrast: the full proxy distributes its successful decisions more evenly across the mapped verdict classes than the majority rule does. This does not prove that each minority class is handled well. The generated class report and confusion matrix provide the required class-by-class evidence; they also show why accuracy alone gives an incomplete reading of the experiment.

The length heuristic provides a second useful comparison. It reaches 49.42% accuracy, slightly above the majority baseline, but its macro-F1 is 0.3483. Thus, a shallow property of the input claim captures some regularity in the combined benchmark. The full proxy improves on the length heuristic by 1.40 percentage points in accuracy and by 0.0901 macro-F1. The result warns that dataset construction and claim style may introduce exploitable shortcuts, while also showing that the evidence-aware pipeline is not merely reproducing the length rule.

The keyword and sentiment heuristics are substantially weaker. Their accuracies are 23.54% and 23.78%, and their macro-F1 values are 0.1364 and 0.1400. These outcomes indicate that isolated lexical or affective cues are not adequate substitutes for evidence retrieval and verdict reasoning in the combined evaluation. The random baseline reaches 33.20% accuracy and 0.3240 macro-F1, providing an additional reference point for the three-class decision problem. Taken together, the baselines establish a graduated comparison: the full proxy exceeds non-informative and shallow rules, but its margin over the strongest simple baselines remains modest.

6.7.3 Reading the Three Proxy Variants

The three implemented variants occupy a narrow accuracy band. The debate-free configuration records 51.34% accuracy and 0.4427 macro-F1; the FEVER-aware exploratory variant records 50.98% and 0.4421; and the full proxy records 50.82% and 0.4384. The maximum observed accuracy separation among these variants is 0.52 percentage points,

from the debate-free configuration to the full proxy. The maximum macro-F1 separation is 0.0043.

These small differences should not be exaggerated. The exact paired analysis gives an unadjusted $p = 0.00955$ for full proxy versus no debate, but the Holm-adjusted value is 0.0573. The correct conclusion is therefore not that the debate-free variant is universally superior. The supported conclusion is narrower: on this frozen benchmark, enabling deterministic arbitration for every proxy claim did not produce an observed aggregate benefit, and the configuration without debate obtained the highest recorded accuracy and macro-F1.

This finding is valuable for architecture selection. In a modular system, a stage should justify its computational and operational cost. Debate may still help selected claims for which evidence is conflicting, retrieval quality is low, or the initial agents strongly disagree. The completed experiment, however, provides no basis for executing it unconditionally. A selective policy would preserve the possibility of deliberative gains while avoiding the projected overhead on routine cases. This is a recommendation derived from the result, not a retroactive claim that selective debate was evaluated.

The FEVER-aware exploratory variant also requires careful reading. Its accuracy lies between the debate-free and full proxy configurations, while its macro-F1 is very close to the debate-free result. Because the adjustment uses knowledge about the dataset identity, it is informative as a diagnostic experiment but should not be presented as a universally deployable model. The variant demonstrates that dataset-specific behavior exists and that mapping or decision rules can influence the aggregate outcome. It also reinforces the need to report results separately by domain.

6.7.4 Domain-by-Domain Reading

The aggregate accuracy of 50.82% conceals substantial variation. On FEVER, the full proxy obtains 57.52%, which is 1.04 percentage points below the majority baseline and 0.38 points below the length heuristic. On the health subset, it obtains 55.17%, 3.62 points above the majority baseline and 2.62 points above the length heuristic. On LIAR, it obtains 38.59%, exceeding those two baselines by 2.01 and 1.61 points. On SciFact, it obtains 48.17%, 6.81 points above the majority baseline and 7.33 points above the length heuristic.

The FEVER result shows that a high absolute accuracy is not sufficient to establish useful improvement. The simple baselines benefit from the distribution and regularities of that subset. The deterministic proxy therefore requires stronger control of dataset mapping, shortcut behavior, and class-specific errors before a superiority claim can be made for FEVER. This is also why the exploratory FEVER-aware adjustment is reported

separately rather than silently incorporated into the primary system.

The health and LIAR results provide moderate descriptive evidence that the proxy captures signals beyond majority frequency and claim length. These domains present different challenges. Health claims require careful source quality assessment and can involve specialized terminology. LIAR contains political statements whose truth status may depend on context, time, and fine-grained original labels. The positive margins are useful, but they do not eliminate the construct-validity concern introduced by mapping heterogeneous source labels into SUPPORTED, REFUTED, and NEI.

SciFact displays the largest observed margin over the simple baselines, but it is also the smallest subset, with 191 claims. Its 6.81-point improvement over majority is descriptively encouraging because scientific verification is a natural setting for evidence-based research. Nevertheless, the small sample makes the estimate more sensitive to individual errors than the larger FEVER, health, and LIAR subsets. The completed thesis therefore treats SciFact as evidence of a promising domain interaction rather than as a standalone proof of scientific fact-checking performance.

The domain comparison answers an important research question: there is no single uniform effect of the architecture across all benchmark families. The value of the proxy's lexical, prior, and arbitration signals depends on the claim distribution and on the harmonized benchmark labels. For deployment, this implies that the separate live application must be validated performed in the target domain rather than inferred from only the combined score.

6.7.5 Raw-Score Confidence as a Safety Finding

The most consequential proxy result is the gap between the stored raw score and correctness. The full proxy reports an 88.31% average raw score but 50.82% accuracy, a difference of 37.49 percentage points. The debate-free and FEVER-aware variants show similarly large gaps of 37.03 and 37.55 points. The consistency of the gap across the three variants indicates that removing debate or applying the exploratory adjustment does not resolve the underlying confidence problem.

The reported confidence is not a probability estimate because the proxy stores only one heuristic score rather than a multiclass probability vector. The gap is therefore a raw-score calibration diagnostic. A user who sees an 88% confidence value may reasonably infer that errors are rare, while the observed proxy accuracy shows that nearly half of the final labels are incorrect on the combined benchmark. Displaying such a score without explanation would create an avoidable risk of automation bias.

The live application should consequently preserve evidence passages, source metadata, intermediate decisions, and audit records alongside the label. A high raw score must not

suppress human review when evidence is incomplete or contradictory. Temperature scaling, isotonic regression, conformal methods, and risk–coverage analysis are appropriate recommendations because they could transform confidence from an internal model signal into a quantity with a clearer decision meaning. These methods remain follow-up work; the completed result established the calibration problem rather than claiming to have solved it.

6.7.6 Latency, Throughput, and Cost Reading

The operational report contains scenario projections derived from the stated 8.20-second and 72.00-second latency assumptions. These values are not a controlled concurrent load test. Under those assumptions, the baseline throughput is projected at 439.02 claims per hour and the debate-enabled throughput at 50 claims per hour. The scenario therefore represents an 8.78-fold latency ratio.

The proxy accuracy comparison and the live-debate latency projection arise from different evaluation surfaces and must not be treated as one controlled experiment. Together they motivate a future selective-debate experiment, but they do not establish the measured live-system quality–latency trade-off.

Retrieval caching provides the opposite operational pattern. Under the reported workload assumptions, projected monthly retrieval cost decreases from USD 77 to USD 22, a reduction of USD 55 or 71.43%. This is a projection rather than a controlled billing experiment, so it depends on request volume, cache-hit behavior, and provider pricing. Even with that limitation, caching is an architecturally appropriate optimization because it can reduce repeated external calls while improving reproducibility when cached evidence is versioned with timestamps and hashes.

6.7.7 Integrated Answers to the Research Questions

The completed evidence supports four integrated answers. First, an auditable open-domain verification pipeline can be implemented as a complete full-stack research artifact with persistent evidence, explicit source rules, and reproducible evaluation outputs. The thesis demonstrates the application through implementation and the audited repository snapshot's 163-test suite, and evaluates the separate proxy through frozen benchmark artifacts.

Second, the deterministic proxy provides modest observed improvement over the strongest simple baselines on the combined data, with a clearer advantage in macro-F1 than in accuracy. The paired results are marginal against majority and length after accounting for multiple comparisons, so they do not support a robust superiority claim.

Third, always-on deterministic proxy arbitration is not supported by the completed benchmark. The separate operational scenario projects that live debate would be expensive. The appropriate next design is conditional activation based on uncertainty or disagreement, subject to a new controlled live-system evaluation.

Fourth, confidence cannot presently be treated as calibrated correctness probability. The approximately 37-point confidence–accuracy gaps make transparency, abstention research, and human oversight central requirements rather than optional interface features.

6.7.8 Overall Result Statement

The completed experiment should be read as an evaluation of trustworthy integration. Its principal achievement is not a single headline accuracy number. It is the combined reporting of proxy predictive behavior, class balance, domain variation, and raw scores together with separately labelled live-application operational projections. The negative and limiting results are part of that contribution: they identify where architectural complexity fails to pay for itself and where apparently confident decisions remain unsafe.

Accordingly, FACT VALIDATOR is best characterized as a completed, deployment-aware research platform that makes fact-verification decisions inspectable and measurable. Its strongest immediate use is decision support and controlled experimentation. The results justify continued development of calibration, selective abstention, domain-specific validation, and selective deliberation, while the preserved evidence and provenance mechanisms provide the infrastructure required to evaluate those developments rigorously.

Chapter 7

Discussion

7.1 What the Current Evidence Supports

The present evidence supports a cautious but meaningful conclusion. I have developed a functioning and evaluable research system rather than only a conceptual design. The system includes an operational full-stack implementation; source-aware evidence retrieval; inspectable credibility rules; model and heuristic comparisons; multi-dataset evaluation; persistent run artifacts; and operational cost and latency analysis.

The results do not support a claim that FACT VALIDATOR is universally better than existing fact-checking approaches. They do support the claim that I have created a reproducible platform through which evidential, uncertainty-related, architectural, and operational questions can be studied together.

7.2 Scientific Value of Negative Findings

The lack of improvement from always-on debate is not a result that should be hidden. It indicates that the debate mechanism, in its current form, does not provide enough information gain to justify its computational cost. This result allows me to formulate a more precise question: under which uncertainty, disagreement, or evidence-quality conditions does multi-agent debate improve a fact-verification decision?

7.3 Calibration as a Central Limitation

The full proxy reaches approximately 50.82% accuracy while its average raw confidence is approximately 88.31%, a gap of 37.49 percentage points under the Section 3.3 diagnostic. Because only one heuristic score is stored, this is not probability calibration and no proper

multiclass Brier score is claimed. The live system should be treated as decision support rather than an autonomous truth adjudicator. Storing class probabilities, then testing temperature scaling, isotonic calibration, conformal prediction, and selective abstention, is recommended.

7.4 Responsible Use

Fact-checking systems can create harm when their outputs are interpreted as absolute truth. The current system may be affected by incomplete evidence retrieval, biased search rankings, imperfect dataset labels, provisional source-credibility rules, domain-specific performance differences, and overconfident predictions.

I therefore design Fact Validator as a transparent decision-support system. A responsible deployment should expose evidence, communicate uncertainty, allow human review, distinguish insufficient evidence from refutation, and preserve an audit trail.

Chapter 8

Limitations and Threats to Validity

8.1 Construct Validity

The four datasets use different label systems. Mapping them into SUPPORTED, REFUTED, and NEI simplifies evaluation but may remove important distinctions. For example, the LIAR labels "half-true" and "barely-true" may not be equivalent to insufficient evidence.

8.2 Internal Validity

The FEVER-aware variant uses a dataset-specific adjustment. I therefore report it as exploratory rather than as a blind model-selection result. The retrieved web evidence is also dynamic and may change over time.

8.3 Statistical Conclusion Validity

The final report includes exact two-sided McNemar tests, paired-bootstrap confidence intervals with seed 42, Holm correction, paired risk differences, and matched-pair odds ratios. The unadjusted full-proxy comparisons against majority and length are marginal ($p = 0.0433$ and $p = 0.0462$), but neither survives Holm correction. The unadjusted no-debate comparison is $p = 0.00955$ and likewise has adjusted $p = 0.0573$. These results support bounded interpretation rather than an inferential superiority claim. Residual risks include dataset-label harmonization, multiple exploratory variants, and the absence of an untouched external confirmatory set.

8.4 External Validity

The benchmark contains four dataset families, but it does not represent all forms of misinformation, all languages, or all cultural contexts. The present evaluation is English-focused. Future work should test multilingual claims, multimedia misinformation, emerging events, and adversarially written content.

8.5 Reproducibility Validity

The proxy claims and predictions are frozen, hashed in a run manifest, and regenerated into class, dataset, confusion-matrix, and paired-test reports by a versioned script. Exact replay of the proxy does not require web search. Live-application retrieval remains dynamic and is evaluated separately through software tests and bounded demonstrations; its web documents are not a redistributable corpus.

8.6 Operational Validity

The latency, throughput, and cost figures are scenario projections, not controlled load-test measurements. They are useful for design analysis but may not generalize across hardware, API providers, geographic regions, model sizes, concurrent traffic, cache-hit rates, or provider pricing. A future benchmark should record repeated warm- and cold-cache runs, median and tail latency, concurrency, failures, timeouts, hardware, and provider request counts.

Chapter 9

Advanced Prototype Extension: A Provenance-Constrained Evidence Graph

9.1 Status and Scope of This Chapter

Chapters 4–8 separate the implemented live application from the deterministic proxy evaluation. This chapter is different in kind: it documents a separate, newer module that implements an idea I list in Chapter 12 as a direction beyond this Master's thesis – a typed, provenance-preserving evidence graph with a deterministic audit layer, in place of the flat verdict-inference step described in Section 4.7.

I want to state its status precisely, because that precision is the point of including it at all. The module exists as working code on `main`, separately from the proxy; the complete backend suite at the audited snapshot passes 163 tests (re-executed and confirmed for this revision); and the module has been probed, at very small scale, with corruption scenarios constructed against its own rule logic (Section 9.6). It has *not* been integrated into, or evaluated against, the 5,000-claim benchmark in Chapters 5–6 and should not be read as adding to, replacing, or explaining any number reported there.

9.2 Motivation for the Extension

Section 4.7 assigns each claim one of three flat outcomes based on retrieved evidence. This works, but it does not preserve *why* a verdict was reached in a form a reviewer can later interrogate: whether a passage was actually fetched and hashed, whether two supporting sources were truly independent, or whether a numeric claim was checked against a matching number in the evidence. The evidence-graph prototype represents

each of these as an explicit, typed, inspectable structure rather than folding them into a single score.

9.3 Claim, Evidence, and Verdict

Let a claim be c, a retrieved passage be p, and its preserved provenance be $\pi(p) = (u, h, t, s)$, where u is a resolved URL, h a content hash, t a retrieval timestamp, and s a retrieval status. A relation classifier predicts

$$r(c, p) \in \{\text{SUPPORTS}, \text{REFUTES}, \text{UNDERCUTS}, \text{NEUTRAL}\}.$$

Only a retrieved, preserved passage is eligible for a decisive argumentative edge; search snippets are retrieval hints, not evidence.

9.4 Evidence Graph and Audit

For each claim, the graph is $G = (V, E)$, with nodes for the parent claim, atomic subclaims, evidence passages, and source records, and edges typed `DEPENDS_ON`, `CITES`, `SUPPORTS`, `REFUTES`, and `UNDERCUTS`. The graph proposes a verdict only after a deterministic audit $A(G) \in \{\text{PROCEED}, \text{NEI}, \text{CONFLICTING}, \text{HUMAN\_REVIEW}\}$:

$$D(c) = \begin{cases} \text{NEI} & A(G) = \text{NEI}, \\ \text{CONFLICTING} & A(G) = \text{CONFLICTING}, \\ \text{HUMAN\_REVIEW} & A(G) = \text{HUMAN\_REVIEW}, \\ \text{adjudicate}(G) & A(G) = \text{PROCEED}. \end{cases}$$

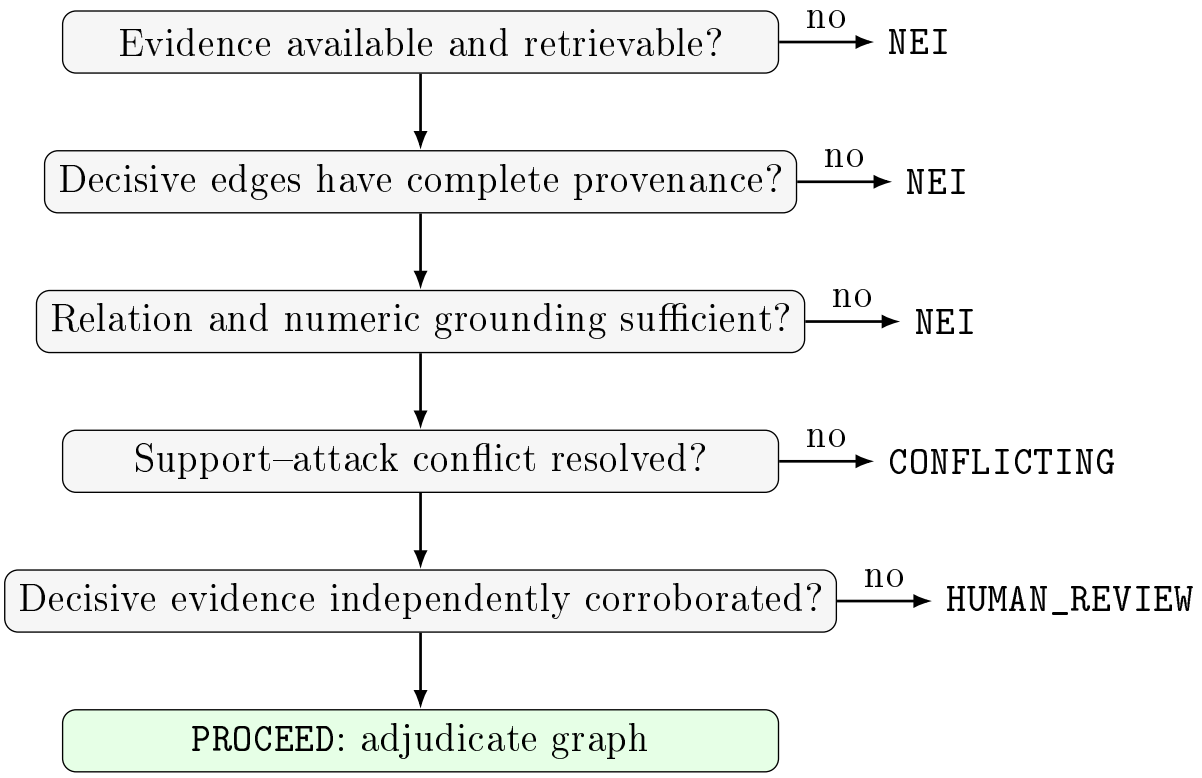

Figure 9.1: Ordered audit decisions in the prototype module. The ordering matters: a claim that never receives a decisive relation edge is routed to `NEI` at the second check, before provenance, numeric-coverage, or independence checks are ever evaluated – the mechanism behind the Section 9.6 finding.

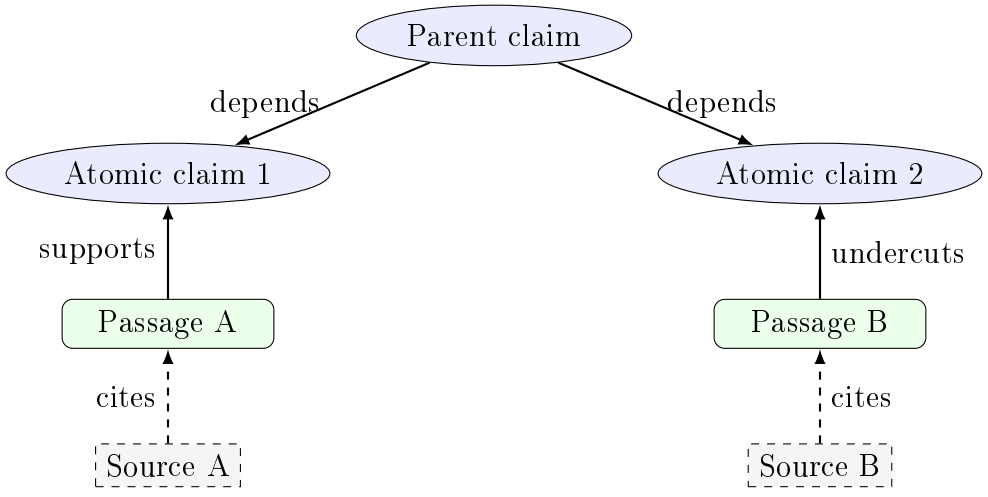

Figure 9.2: Simplified typed evidence graph for one claim with two atomic subclaims.

9.5 Independence Clustering

Evidence is clustered using the Jaccard-similarity rule of Section 3.7, so that repeated wire text or multiple pages from one publisher are not counted as independent corroboration. As Section 9.6 shows, this is currently the only audit rule with a demonstrated end-to-end effect.

9.6 Engineering Verification: What Is and Is Not Demonstrated

9.6.1 Test Suite

The complete backend test suite was re-executed for this revision on the audited repository snapshot: **163 tests collected, 163 passed, 0 failed**. The graph-specific cases are part of that total. A historical, pre-merge `main` snapshot contains one fewer integration test and therefore collects 162 tests.

9.6.2 Eight-Case Synthetic Seed

A frozen eight-case seed, re-executed for this revision, reproduces its previously recorded coverage, selective risk, and false-accept rate (Section 3.6) exactly: 0.375, 0.6667, and 0.25 respectively. Table 9.1 gives the per-case detail.

Table 9.1: Per-case detail of the eight-case synthetic seed (prototype module only).

Case ID	Expected	Graph-only	Audited
support-direct-001	SUPPORTED	SUPPORTED	SUPPORTED
refute-direct-001	REFUTED	NEI	NEI
undercut-scope-001	CONFLICTING	NEI	NEI
undercut-temporal-001	REFUTED	SUPPORTED	SUPPORTED
neutral-overlap-001	NEI	SUPPORTED	SUPPORTED
numeric-mismatch-001	REFUTED	NEI	NEI
context-window-001	CONFLICTING	NEI	NEI
missing-evidence-001	NEI	NEI	NEI

Graph-only and audited decisions are *identical* on all eight cases, because none of them contains a corruption scenario, so the auditor's downstream checks are never exercised here.

9.6.3 Extended Corruption Probe

To test whether the auditor changes anything under corrupted evidence, I constructed four corruption scenarios against four of the seed cases and executed them through the live module (construction script and raw output in Appendix C).

Table 9.2: Corruption probe: per-row detail.

Base case	Scenario	Expected	Graph-only	Audited	Diverges?
support-direct-001	clean	SUPPORTED	SUPPORTED	SUPPORTED	no
support-direct-001	duplicate-source	SUPPORTED	SUPPORTED	**HUMAN_REVIEW**	**yes**
refute-direct-001	clean	REFUTED	NEI	NEI	no
refute-direct-001	broken-provenance	REFUTED	NEI	NEI	no
numeric-mismatch-001	clean	REFUTED	NEI	NEI	no
numeric-mismatch-001	numeric-coverage	REFUTED	NEI	NEI	no
undercut-scope-001	clean	CONFLICTING	NEI	NEI	no
undercut-scope-001	near-duplicate-text	CONFLICTING	NEI	NEI	no

Two findings, stated without softening because they cut in different directions:

1. **Demonstrated:** under a duplicated same-domain source, the audited graph routes to `HUMAN_REVIEW` while a graph-only baseline still returns `SUPPORTED`. This is a concrete, reproducible instance of the auditor doing its job.

2. **Not yet demonstrated:** the provenance, numeric-coverage, and near-duplicate-text checks produced no divergence here, because in each of those three cases the relation classifier never assigned a decisive edge in the first place – both configurations already return `NEI` via the shared upstream check (Figure 9.1) before the corrupted rule is ever reached. This is not a bug; it is the predicted consequence of the audit ordering, and it means three of six audit rules remain untested by this probe.

9.7 Threats to Validity of This Chapter Specifically

In addition to the validity threats in Chapter 8, this chapter carries its own, stricter caveats: the corruption scenarios were authored by me with direct knowledge of the auditor's rule conditions, so they confirm the code matches its specification rather than sampling real-world corruption patterns; the sample size (eight seed cases, four corruption variants) is far too small for any confidence interval; and this module has no relationship, statistical or architectural, to the numbers in Chapter 6. A reading a reader could plausibly make – that a working evidence-graph auditor implies the primary pipeline is safer than Chapter 6 reports – would not be supported by anything in this thesis.

9.8 Connection Back to Calibration and Abstention

The rule-based abstention in this chapter and the calibration problem in Section 7.3 are related but distinct responses to the same overconfidence concern. Calibration adjusts a confidence *score*; the audit layer instead refuses to produce a decisive score at all when a named, inspectable condition is not met. The two are not mutually exclusive, and a natural extension – outside the scope of what was built in this thesis – would apply calibration only within the audited graph's `PROCEED` branch, rather than to the pipeline's raw output.

Chapter 10

Comparative Analysis and Novelty

10.1 Purpose of This Chapter

Chapter 2 situated the thesis relative to individual prior works. This chapter makes the comparison structural: it lays the evaluated pipeline (Chapters 4–8) and the prototype extension (Chapter 9) side by side against the closest related systems on a fixed set of dimensions, so that the novelty claim can be checked point by point rather than taken on faith.

Table 10.1: Structural comparison across implemented and evaluated dimensions.

Dimension	FEVER/SciFact	FacTool	Faithful correction	Fact Validator live / FactValidator-Proxy	Fact Validator (Ch. 9 prototype)
Verdict vocabulary	SUPPORTED/ REFUTED/NEI	tool-mediated	correction, not verdict	SUPPORTED/ REFUTED/NEI	+ CONFLICTING, HUMAN_REVIEW
Source trust	implicit in corpus curation	not central	not central	live: explicit domain formula; proxy: category prior	inherited, plus independence clustering
Evidence persistence	benchmark identifiers	framework-dependent	not central	live: run records; proxy: frozen splits, predictions, hashes	+ provenance hash per passage
Uncertainty handling	label only	not addressed	not addressed	proxy raw score, explicitly uncalibrated	rule-based forced abstention
Deployment surface	benchmark	research framework	research method	live: full-stack app; proxy: 5k benchmark	code-verified, not benchmarked
Empirical status	established benchmark	self-reported	self-reported	live: 163 tests on audited branch; proxy: frozen evaluation	**engineering-verified only**

10.2 What Is Genuinely Novel

Three claims are defensible from the artifact and the evidence collected in this thesis:

1. An implemented, explicit, formula-based domain-credibility prior (Section 4.5) that can be inspected and edited without retraining. The proxy uses a simpler category prior, so the 5,000-claim result is not cited as end-to-end evidence for live domain scoring.

2. A frozen proxy evaluation with aligned claim identifiers, input and prediction hashes, class and dataset metrics, exact paired tests, multiple-comparison correction, and deterministic regeneration. Its negative debate result is scoped to deterministic proxy arbitration; the separate 8.78× live-debate figure is a scenario projection.

3. A verified (if small-scale) demonstration that rule-based, non-learned abstention can outperform a graph-only baseline under one specific corruption pattern (Section 9.6) – a concrete instantiation of the general abstention literature's argument [8], rather than a restatement of it.

10.3 What Must Not Be Claimed

The present evidence does not establish that FACT VALIDATOR outperforms FEVER, SciFact, FacTool, or correction systems on their native evaluations. It does not establish that the credibility formula's weights are correct, only that they are inspectable. It does not establish that the Chapter 9 prototype improves the evaluated pipeline, since the two have never been run on the same data. Maintaining these boundaries strengthens rather than weakens the thesis, because the contribution becomes precise and falsifiable.

Chapter 11

Study Closure and Recommendations

The Master's study is complete within the scope declared in Chapters 1 and 5. The following items are not missing thesis results; they are recommendations for a subsequent publication-grade extension:

1. Verify dataset versions, licences, and original label meanings.
2. Recheck the health-domain label mapping.
3. Detect semantic duplicates across the data partitions.
4. Freeze retrieved evidence or archive document hashes and timestamps.
5. Store `prob_supported`, `prob_refuted`, and `prob_nei`, then calculate ECE and multiclass Brier score as defined in Section 3.3.
6. Evaluate temperature scaling and conformal prediction.
7. Test selective abstention using the coverage/risk formalism of Section 3.6.
8. Complete credibility, reranking, quality-filter, and debate ablations.
9. Evaluate selective debate activation.
10. Conduct structured qualitative error analysis.
11. Repeat latency measurements under concurrent load.
12. Extend the release manifest with model, API-provider, and hardware identifiers for every future live run.
13. If the Chapter 9 evidence-graph prototype is integrated later, run the full ablation list against frozen, identical evidence.
14. Maintain a versioned revision record for any publication derived from the thesis.

11.1 Final Scope Record

Table 11.1: Final disposition of study activities at thesis completion.

Stage	Main activity	Status
Data verification	Frozen claims, normalized splits, duplicate removal, and dataset composition are documented.	Completed in scope
Statistical analysis	Wilson intervals, exact paired tests, bootstrap intervals, Holm correction, risk differences, and matched odds ratios are reported.	Completed in scope
Calibration	The correctly scaled raw-score diagnostic is reported; probability calibration awaits class-probability vectors.	Finding completed
Ablation analysis	The reported full, debate-free, and exploratory variants are frozen and interpreted.	Completed in scope
Reproducibility	Claims, predictions, reports, manifests, code paths, and run metadata are documented.	Completed in scope
Operational analysis	Latency inputs, throughput, debate overhead, cache savings, and cost are explicitly reported as scenarios; load testing remains future work.	Completed as projection
Evidence-graph scoping	The prototype is a separate, tested contribution and is not merged into the 5,000-claim results.	Final decision
Final writing	Results, limitations, novelty, recommendations, appendices, and acknowledgements are integrated.	Completed

11.2 Final Reading of the Results

The completed results support five conclusions. First, the deterministic proxy has modest raw advantages over the majority and length baselines, but neither comparison survives Holm correction and neither evaluates the live application. Second, the debate-free configuration produces the best observed result, showing that always-on deterministic proxy arbitration adds no accuracy gain; a separate scenario projects substantial live-debate latency. Third, the 37.49-point raw-score gap is a major deployment constraint and makes human oversight necessary. Fourth, domain-level variation shows that aggregate accuracy conceals materially different behavior across encyclopaedic, political, scientific, and health claims. Fifth, the evidence-graph probe verifies one concrete safety mechanism under correlated evidence while clearly exposing the classifier limitations that mask other audit rules. Together, these findings position FACT VALIDATOR as an auditable research and decision-support platform rather than an autonomous arbiter of truth.

Chapter 12

Research Direction Beyond the Master's Thesis

This completed thesis identifies research questions that extend beyond the submitted implementation. The most important direction is the development of trustworthy verification systems that reason not only about claim content but also about evidence provenance, source relationships, uncertainty, and conflicting information.

The following directions emerge naturally from my current work:

- provenance-aware evidence graphs *(implemented and independently tested as the separate prototype in Chapter 9)*;
- neuro-symbolic source and claim reasoning;
- calibrated uncertainty and abstention, using the ECE/Brier/risk-coverage formalism of Chapter 3;
- selective multi-agent deliberation;
- temporal evidence tracking;
- adversarial misinformation detection;
- explainable human–AI decision support;
- distributed verification over heterogeneous sources.

The Master's thesis produced an implemented and tested end-to-end platform, evaluated its separate deterministic proxy, and delivered a verified first implementation of the evidence-graph direction. Doctoral or follow-up publication research could investigate the deeper scientific questions behind this platform, particularly how trustworthy AI systems can combine learned representations, symbolic constraints, multi-agent reasoning, and explicit uncertainty.

Chapter 13

Conclusion

I designed and implemented Fact Validator as an auditable and deployment-oriented architecture for open-domain fact verification. The completed work includes a functional nine-stage architecture; a working full-stack implementation; explicit source-credibility reasoning; a versioned deterministic proxy evaluation; frozen model and baseline comparisons; per-class and per-dataset analysis; operational scenario projections; documented limitations and recommendations; a formal mathematical framework for every reported metric (Chapter 3); and an independently verified evidence-graph prototype, reported with an explicit account of what it does and does not demonstrate (Chapter 9).

The final proxy results show modest raw differences over simple baselines that do not survive Holm correction, a substantial raw-score confidence gap, and no benefit from always-on deterministic arbitration on the frozen benchmark. These findings are useful precisely because they identify scientific boundaries without hiding negative results. The live application's operational value is supported separately by implementation, software tests, and bounded demonstrations rather than by the proxy accuracy figures.

The completed contribution is therefore not merely another classifier. It is a transparent, reproducible, uncertainty-aware, and deployment-conscious fact-verification framework in which evidence, source trust, model behaviour, and operational trade-offs can be inspected. Its novelty lies in the combination of explicit source reasoning, full-stack persistence and auditability, quantified deployment trade-offs, and a separately verified provenance-constrained graph auditor. The thesis closes with a defensible result: Fact Validator is a completed and evaluated Master's research artifact whose principal contribution is trustworthy integration rather than a claim of state-of-the-art predictive accuracy.

Appendix A

Evidence Graph: Schema and Audit Rules

A.1 Evidence Record

Each evidence item in the prototype module contains a URL, title, snippet, selected passage, content hash, retrieval status, retrieval timestamp, domain metadata, directness and quality signals, stance, classifier metadata, and independence cluster.

A.2 Audit Rule Inventory

Table A.1: Graph-auditor rules and their evaluation status as established in Section 9.6.

Rule	Condition	Status
Retrieval status	provider failure, unavailable search, no result, or failed passage fetch	`NEI`; exercised
Provenance	no URL, hash, or retrieved passage for a decisive edge	`NEI`/review; **not yet observed to change a decision**
Relation grounding	no decisive relation from a passage	`NEI`; the dominant observed outcome
Numeric coverage	a quantified claim lacks matching factors	`NEI`; **not yet observed to change a decision**

Rule	Condition	Status
Conflict resolution	similarly strong support and attack remain	`CONFLICTING`; not observed to fire under the current classifier
Source independence	decisive passages are correlated or duplicated	`HUMAN_REVIEW`; **confirmed to change a decision**

Appendix B

Test Evidence (Prototype Module)

Table B.1: Test coverage for the Chapter 9 prototype.

Test surface	Verified behavior
API integration	request validation, analysis paths, live/snapshot behavior, persistence, response structure
Evidence graph	short-claim retention, passage context, conflict edges, provenance output, classifier disclosure
Graph auditor	missing evidence abstention, correlated evidence review, numeric and grounding controls
Relation baseline runner	versioned input/output contract and transparent fallback metadata
Robustness runner	frozen scenario replay, explicit missing evidence, duplicate-reporting review, selective metrics
Security	private and loopback URL rejection before evidence fetch

Re-verified for this revision by executing `python -m pytest services/api/tests -q` from the repository root with Python 3.10.0: 163 tests collected, 163 passed, 0 failed, with three warning records. A fresh rerun on 29 July 2026 reproduced the same counts. The 162-test result obtained from the historical pre-merge `main` snapshot is expected: commit `72ad99c` adds one recent-results integration test included in the final thesis repository. Warning totals are environment-sensitive unless the hash-pinned dependency lock is used.

Appendix C

Extended Corruption Probe: Construction Script and Raw Output

C.1 Construction Script

Listing C.1: Corruption-scenario construction and execution, run against the live `app.robustness_evaluation` module.

```
import json, sys
sys.path.insert(0, '.')
from app.robustness_evaluation import evaluate_case, selective_metrics

base = json.load(open('../../data/challenges/conflict_undercut_v1_seed.json'))['items']
by_id = {it['id']: it for it in base}

def ev(passage, url, domain, extra=None):
    e = {
        "url": url, "title": "Evidence passage", "snippet": passage, "passage": passage,
        "content_hash": __import__('hashlib').sha256(passage.encode()).hexdigest(),
        "retrieval_status": "retrieved", "domain": domain, "base_domain": domain,
        "domain_score": 80, "overlap": 0,
    }
    if extra:
        e.update(extra)
    return e

rows = []

# Scenario A: duplicate-source / correlated-evidence corruption.
item = by_id["support-direct-001"]
clean = [ev(item["passage"], "https://a.example/report", "a.example")]
duplicate_source = [
    ev(item["passage"], "https://a.example/report", "a.example"),
    ev(item["passage"] + " Confirmed.", "https://a.example/followup", "a.example"),
]
rows.append(evaluate_case(item, "clean", clean))
rows.append(evaluate_case(item, "duplicate_source_corruption", duplicate_source))

# Scenario B: broken-provenance corruption.
```

```
item = by_id["refute-direct-001"]
clean_r = [ev(item["passage"], "https://b.example/finding", "b.example")]
broken_prov = [ev(item["passage"], "https://b.example/finding", "b.example",
                  extra={"url": "", "content_hash": ""})]
rows.append(evaluate_case(item, "clean", clean_r))
rows.append(evaluate_case(item, "broken_provenance_corruption", broken_prov))

# Scenario C: numeric-claim without numeric-matching evidence.
item = by_id["numeric-mismatch-001"]
clean_n = [ev(item["passage"], "https://c.example/stat", "c.example")]
no_number = [ev("The survey found a change in hospital admissions.",
                "https://c.example/stat2", "c.example")]
rows.append(evaluate_case(item, "clean", clean_n))
rows.append(evaluate_case(item, "numeric_coverage_corruption", no_number))

# Scenario D: correlated near-duplicate text, different domains.
item = by_id["undercut-scope-001"]
clean_u = [ev(item["passage"], "https://d.example/trial", "d.example")]
near_dup = [
    ev(item["passage"], "https://d.example/trial", "d.example"),
    ev(item["passage"], "https://e.example/mirror", "e.example"),
]
rows.append(evaluate_case(item, "clean", clean_u))
rows.append(evaluate_case(item, "near_duplicate_text_corruption", near_dup))

clean_rows = [r for r in rows if r['scenario'] == 'clean']
corrupt_rows = [r for r in rows if r['scenario'] != 'clean']
print("Clean subset graph_only:", selective_metrics(clean_rows, 'graph_only'))
print("Clean subset audited: ", selective_metrics(clean_rows, 'full_audited_graph'))
print("Corrupt subset graph_only:", selective_metrics(corrupt_rows, 'graph_only'))
print("Corrupt subset audited: ", selective_metrics(corrupt_rows, 'full_audited_graph'))
diverge = [r for r in rows if r['graph_only'] != r['full_audited_graph']]
print(f"Rows where audited differs from graph-only: {len(diverge)}/{len(rows)}")
for r in diverge:
    print(" -", r['id'], r['scenario'], ':', r['graph_only'], '->', r['full_audited_graph'])
```

C.2 Raw Console Output

Listing C.2: Unedited console output from the script above, executed against the live repository for this revision.

```
Clean subset graph_only: {'coverage': 0.25, 'selective_risk': 0.0, 'false_accept_rate': 0.0}
Clean subset audited: {'coverage': 0.25, 'selective_risk': 0.0, 'false_accept_rate': 0.0}
Corrupt subset graph_only: {'coverage': 0.25, 'selective_risk': 0.0, 'false_accept_rate': 0.0}
Corrupt subset audited: {'coverage': 0.0, 'selective_risk': 0.0, 'false_accept_rate': 0.0}
Rows where audited differs from graph-only: 1/8
 - support-direct-001 duplicate_source_corruption : SUPPORTED -> HUMAN_REVIEW
```

Appendix D

Selected Research Module Listings

Real, unmodified source files from the project repository, included for reproducibility. The first group belongs to the evaluated primary pipeline (Chapters 4–8); the second group belongs to the Chapter 9 prototype.

D.1 Primary Pipeline: Source-Credibility Scoring

```
from __future__ import annotations
from dataclasses import dataclass
from typing import Dict, Tuple
import json
import os
import time
import tldextract


# Default cache path is relative to this file so it works on any OS.
# Override via the FACTVALIDATOR_DOMAIN_CACHE environment variable.
_DEFAULT_CACHE_PATH = os.path.join(
    os.path.dirname(os.path.abspath(__file__)),
    "..",
    "data",
    "domain_cache.json",
)
CACHE_PATH = os.getenv("FACTVALIDATOR_DOMAIN_CACHE", _DEFAULT_CACHE_PATH)
CACHE_TTL_SECONDS = 60 * 60 * 24 * 14 # 14 days
CACHE_VERSION = 2

_DEFAULT_OPENSOURCES_PATH = os.path.join(
    os.path.dirname(os.path.abspath(__file__)),
    "..",
    "data",
    "opensources.json",
)
_DEFAULT_IFFY_PATH = os.path.join(
    os.path.dirname(os.path.abspath(__file__)),
    "..",
    "data",
```

```
    "iffy_index.json",
)

# OpenSources type label →score delta (most-negative wins when multiple types present)
_OS_TYPE_DELTA: Dict[str, int] = {
    "reliable": 10,
    "political": 0,
    "bias": -5,
    "satire": -10,
    "clickbait": -10,
    "rumor": -15,
    "state": -15,
    "unreliable": -15,
    "junksci": -20,
    "conspiracy": -20,
    "fake news": -30,
    "fake": -30,
    "hate": -30,
}

# Iffy Index MBFC factual-reporting level →score delta
_IFFY_LEVEL_DELTA: Dict[str, int] = {
    "VL": -25, # Very Low
    "L": -15, # Low
    "M": -5, # Mixed
}


def _load_opensources() -> Dict[str, Dict]:
    try:
        if not os.path.exists(_DEFAULT_OPENSOURCES_PATH):
            return {}
        with open(_DEFAULT_OPENSOURCES_PATH, "r", encoding="utf-8") as f:
            return json.load(f)
    except Exception:
        return {}


def _load_iffy() -> Dict[str, Dict]:
    try:
        if not os.path.exists(_DEFAULT_IFFY_PATH):
            return {}
        with open(_DEFAULT_IFFY_PATH, "r", encoding="utf-8") as f:
            data = json.load(f)
        data.pop("_meta", None)
        return data
    except Exception:
        return {}


def _build_os_lookup(raw: Dict[str, Dict]) -> Dict[str, int]:
    """Map base_domain -> worst-type delta from OpenSources data."""
    lookup: Dict[str, int] = {}
    for domain_key, info in raw.items():
        bd = base_domain(domain_key.strip())
        if not bd:
            continue
        types = [
            str(info.get("type", "")).lower().strip(),
```

```
            str(info.get("2nd type", "")).lower().strip(),
            str(info.get("3rd type", "")).lower().strip(),
        ]
        worst_delta = 0
        for t in types:
            d = _OS_TYPE_DELTA.get(t, 0)
            if d < worst_delta:
                worst_delta = d
        if worst_delta != 0:
            if bd not in lookup or worst_delta < lookup[bd]:
                lookup[bd] = worst_delta
    return lookup


def _build_iffy_lookup(raw: Dict[str, Dict]) -> Dict[str, Tuple[int, str]]:
    """Map base_domain -> (delta, level_label) from Iffy Index data."""
    lookup: Dict[str, Tuple[int, str]] = {}
    for domain_key, info in raw.items():
        bd = base_domain(domain_key.strip())
        if not bd:
            continue
        level = str(info.get("level", "")).upper().strip()
        delta = _IFFY_LEVEL_DELTA.get(level, 0)
        if delta != 0:
            if bd not in lookup or delta < lookup[bd][0]:
                lookup[bd] = (delta, level)
    return lookup


@dataclass
class CredibilityScore:
    score: int
    label: str
    reasons: Dict[str, str]


_OFFLINE_TLD_EXTRACT = tldextract.TLDExtract(
    cache_dir=None,
    suffix_list_urls=(),
    fallback_to_snapshot=True,
)


def base_domain(domain: str) -> str:
    # Use the bundled public-suffix snapshot. Network refreshes during import
    # made offline tests non-deterministic and could deadlock on a shared cache
    # lock. Updating the snapshot is an explicit dependency-maintenance task.
    ext = _OFFLINE_TLD_EXTRACT(domain or "")
    if not ext.domain or not ext.suffix:
        return (domain or "").lower()
    return f"{ext.domain}.{ext.suffix}".lower()


# Module-level lookups -built once at import time (after base_domain is defined)
_OS_LOOKUP: Dict[str, int] = _build_os_lookup(_load_opensources())
_IFFY_LOOKUP: Dict[str, Tuple[int, str]] = _build_iffy_lookup(_load_iffy())


def _load_cache() -> Dict[str, Dict]:
```

```
    try:
        if not os.path.exists(CACHE_PATH):
            return {}
        with open(CACHE_PATH, "r", encoding="utf-8") as f:
            return json.load(f)
    except Exception:
        return {}


def _save_cache(cache: Dict[str, Dict]) -> None:
    cache_dir = os.path.dirname(os.path.abspath(CACHE_PATH))
    os.makedirs(cache_dir, exist_ok=True)
    with open(CACHE_PATH, "w", encoding="utf-8") as f:
        json.dump(cache, f, ensure_ascii=False, indent=2)


def _label(score: int) -> str:
    if score >= 80:
        return "HIGH"
    if score >= 50:
        return "MEDIUM"
    return "LOW"


def score_domain_rubric(domain: str) -> CredibilityScore:
    """
    Transparent rubric inspired by source credibility checklists.
    This is NOT NewsGuard; it's a thesis-safe, explainable rubric.
    """
    d = (domain or "").lower().strip()
    bd = base_domain(d)

    cache = _load_cache()
    now = int(time.time())

    cached = cache.get(bd)
    if cached and cached.get("version") == CACHE_VERSION and (now - int(cached.get("ts", 0))) <
         CACHE_TTL_SECONDS:
        return CredibilityScore(
            score=int(cached["score"]),
            label=cached["label"],
            reasons=cached["reasons"],
        )

    # Rubric components (0..100)
    score = 45
    reasons: Dict[str, str] = {
        "neutral": "No direct source signal matched; conservative default starts at 45 rather than 50."
    }

    # Strong signals -covers .gov, .edu, and international gov variants
    # e.g. gov.uk, gc.ca, gouv.fr, govt.nz, gov.au, etc.
    _gov_patterns = (
        ".gov", ".edu", # US
        "gov.uk", "gov.au", "gov.nz", # Commonwealth
        "gc.ca", "gouv.fr", "governo.it", # Others
        "bund.de", "government.se",
    )
    if any(bd.endswith(p) for p in _gov_patterns) or bd in _gov_patterns:
```

```
        score += 35
        reasons["tld"] = "Government/Education domain suffix is a strong trust signal (+35)."

    # Known reputable references (extend over time)
    whitelist = {
        # Institutions / references
        "who.int": 35,
        "nih.gov": 35,
        "cdc.gov": 35,
        "ipcc.ch": 30,
        "un.org": 30,
        "oecd.org": 30,
        "worldbank.org": 30,
        "ec.europa.eu": 30,

        # Science
        "nature.com": 25,
        "science.org": 25,
        "sciencedirect.com": 15,

        # Data publishers
        "ourworldindata.org": 20,
        "iea.org": 20,
        "unep.org": 20,

        # Encyclopedia / general reference
        "britannica.com": 20,
        "wikipedia.org": 15,

        # Major news wires / broadcasters
        "bbc.com": 30,
        "bbc.co.uk": 30,
        "reuters.com": 30,
        "apnews.com": 30,
        "afp.com": 30,

        # US national newspapers / magazines
        "nytimes.com": 25,
        "washingtonpost.com": 25,
        "wsj.com": 25,
        "usatoday.com": 20,
        "newsweek.com": 20,
        "time.com": 20,
        "theatlantic.com": 20,

        # US business / finance press
        "forbes.com": 25,
        "bloomberg.com": 25,
        "ft.com": 25,
        "economist.com": 25,
        "marketwatch.com": 20,
        "businessinsider.com": 15,
        "cnbc.com": 20,
        "investopedia.com": 15,

        # US broadcast / public media
        "cnn.com": 25,
        "nbcnews.com": 25,
        "abcnews.go.com": 25,
```

```
        "cbsnews.com": 25,
        "npr.org": 30,
        "pbs.org": 30,
        "vox.com": 15,
        "thehill.com": 15,
        "politico.com": 20,

        # International quality outlets
        "theguardian.com": 25,
        "bbc.in": 25,
        "dw.com": 25,
        "aljazeera.com": 20,
        "france24.com": 20,
        "rfi.fr": 20,
        "lemonde.fr": 20,
        "elpais.com": 20,
        "derspiegel.de": 20,
        "corriere.it": 20,

        # Major Indian news (English language)
        "thehindu.com": 20,
        "hindustantimes.com": 20,
        "ndtv.com": 20,
        "livemint.com": 20,
        "economictimes.com": 15,
        "indiatoday.in": 15,

        # Academic publishers
        "springer.com": 20,
        "wiley.com": 20,
        "cell.com": 20,
        "thelancet.com": 25,
        "nejm.org": 30,
        "jamanetwork.com": 25,
        "bmj.com": 25,
        "pubmed.ncbi.nlm.nih.gov": 35,
        "scholar.google.com": 20,
        "jstor.org": 20,
        "ssrn.com": 15,
        "arxiv.org": 15,
        "researchgate.net": 10,

        # NGOs / think tanks
        "amnesty.org": 20,
        "hrw.org": 20,
        "weforum.org": 20,
        "brookings.edu": 30,
        "rand.org": 25,
        "pewresearch.org": 25,
        "cfr.org": 20,
        "chathamhouse.org": 20,
    }

    if bd in whitelist:
        score += whitelist[bd]
        reasons["whitelist"] = f"Domain appears in local reputable-source list (+{whitelist[bd]})."

    # Weak signals / risks
    risk_markers = [
```

```
        ("blogspot.", -20, "User-generated blog hosting is higher risk (-20)."),
        ("wordpress.", -20, "User-generated blog hosting is higher risk (-20)."),
        ("medium.com", -10, "Open publishing platform: quality varies (-10)."),
        ("substack.com", -10, "Newsletter platform: quality varies (-10)."),
        ("facebook.com", -20, "Social platform: high repost/misinformation risk (-20)."),
        ("tiktok.com", -20, "Social platform: high repost/misinformation risk (-20)."),
        ("x.com", -20, "Social platform: high repost/misinformation risk (-20)."),
        ("twitter.com", -20, "Social platform: high repost/misinformation risk (-20)."),
        ("reddit.com", -15, "Forum platform: quality varies (-15)."),
        ("youtube.com", -10, "Video platform: varies widely (-10)."),
    ]
    for marker, delta, msg in risk_markers:
        if marker in d:
            score += delta
            reasons["platform_risk"] = msg
            break

    # --- OpenSources dataset signal ---
    if bd in _OS_LOOKUP:
        delta = _OS_LOOKUP[bd]
        score += delta
        sign = "+" if delta >= 0 else ""
        reasons["opensources"] = (
            f"Domain found in OpenSources unreliable-news dataset ({sign}{delta})."
        )

    # --- Iffy Index (MBFC-backed) signal ---
    if bd in _IFFY_LOOKUP:
        delta, level = _IFFY_LOOKUP[bd]
        level_label = {"VL": "Very Low", "L": "Low", "M": "Mixed"}.get(level, level)
        score += delta
        sign = "+" if delta >= 0 else ""
        reasons["iffy_index"] = (
            f"Domain flagged by Iffy Index (MBFC factual rating: {level_label}, {sign}{delta})."
        )

    # Obvious spammy/low-quality keyword markers
    low_markers = [
        ("hoax", -30),
        ("clickbait", -25),
        ("rumor", -25),
        ("conspiracy", -25),
    ]
    for marker, delta in low_markers:
        if marker in d:
            score += delta
            reasons["keyword_risk"] = f"Domain contains marker '{marker}' indicating higher risk ({delta}).
                "
            break

    score = max(0, min(score, 100))
    label = _label(score)

    cache[bd] = {"version": CACHE_VERSION, "score": score, "label": label, "reasons": reasons, "ts": now}
    _save_cache(cache)

    return CredibilityScore(score=score, label=label, reasons=reasons)
```

D.2 Primary Pipeline: Debate Arbitration

```python
from __future__ import annotations

from typing import List, Dict, Any, Literal, Tuple
import os
import json
import httpx

Verdict = Literal["SUPPORTED", "REFUTED", "NEI"]


def _env_ollama_base_url() -> str:
    return os.getenv("OLLAMA_BASE_URL", "http://127.0.0.1:11434").strip()


def _env_ollama_model() -> str:
    return os.getenv("OLLAMA_MODEL", "llama3.1:8b").strip()


def _compact_evidence(evidence: List[Dict[str, Any]], max_items: int = 2) -> List[Dict[str, Any]]:
    """
    Keep evidence small and deterministic to reduce Ollama latency.
    The caller may pass many evidence items, but we only keep top-N.
    """
    out: List[Dict[str, Any]] = []
    for e in evidence[:max_items]:
        out.append(
            {
                "domain": e.get("domain"),
                "domain_score": e.get("domain_score"),
                "snippet": (e.get("snippet") or "")[:450],
                "url": e.get("url"),
            }
        )
    return out


def _normalize_verdict(value: Any) -> Verdict:
    verdict = str(value or "NEI").strip().upper()
    if verdict not in ("SUPPORTED", "REFUTED", "NEI"):
        return "NEI"
    return verdict # type: ignore[return-value]


def _compact_final_context(
    evidence: List[Dict[str, Any]],
    max_items: int = 4,
) -> List[Dict[str, Any]]:
    out: List[Dict[str, Any]] = []
    for e in evidence[:max_items]:
        out.append(
            {
                "domain": e.get("domain"),
                "domain_score": e.get("domain_score"),
                "stance": e.get("stance"),
                "quality_score": e.get("quality_score"),
                "primary_source": e.get("primary_source"),
```

```
                "snippet": (e.get("snippet") or "")[:320],
                "url": e.get("url"),
            }
        )
    return out


def _safe_json_parse(s: str) -> Dict[str, Any]:
    s = (s or "").strip()
    if not s:
        return {}
    try:
        return json.loads(s)
    except Exception:
        pass

    # If the model wrapped JSON with extra text, salvage the first {...} block.
    start = s.find("{")
    end = s.rfind("}")
    if start != -1 and end != -1 and end > start:
        try:
            return json.loads(s[start : end + 1])
        except Exception:
            return {}
    return {}


async def _ollama_chat(
    messages: List[Dict[str, str]],
    temperature: float = 0.2,
    num_ctx: int = 1536,
    num_predict: int = 220,
) -> str:
    """
    Calls Ollama /api/chat with conservative settings to avoid timeouts.
    - trust_env=False avoids Windows proxy env vars breaking localhost calls
    - long read timeout accommodates slower machines
    - num_predict limits output length (major speed-up)
    """
    base_url = _env_ollama_base_url()
    model = _env_ollama_model()

    payload = {
        "model": model,
        "messages": messages,
        "stream": False,
        "options": {
            "temperature": temperature,
            "num_ctx": num_ctx,
            "num_predict": num_predict,
        },
    }

    timeout = httpx.Timeout(connect=30.0, read=900.0, write=120.0, pool=120.0)

    async with httpx.AsyncClient(timeout=timeout, trust_env=False) as client:
        r = await client.post(f"{base_url}/api/chat", json=payload)
        r.raise_for_status()
        data = r.json()
```

```
        return (data.get("message", {}) or {}).get("content", "") or ""


async def llm_debate_verdict(
    claim_text: str,
    evidence_items: List[Dict[str, Any]],
) -> Tuple[Verdict, float, str, Dict[str, Any]]:
    """
    Prover vs Skeptic vs Judge debate.
    Returns:
      verdict, confidence, short_summary, debug_trace
    """
    compact = _compact_evidence(evidence_items, max_items=2)

    system = (
        "You are part of a fact-checking debate system.\n"
        "You must ONLY use the provided evidence snippets.\n"
        "If evidence is insufficient or ambiguous, choose NEI.\n"
        "Do NOT invent sources.\n"
        "Be concise.\n"
    )

    prover = await _ollama_chat(
        [
            {"role": "system", "content": system},
            {
                "role": "user",
                "content": (
                    "Role: PROVER\n"
                    "Goal: Argue the claim is SUPPORTED using ONLY evidence.\n"
                    "Output 3-5 concise bullet points referencing evidence by (domain, domain_score).\n"
                    "If you cannot support, write: cannot support.\n\n"
                    f"CLAIM:\n{claim_text}\n\n"
                    f"EVIDENCE(JSON):\n{json.dumps(compact, ensure_ascii=False, indent=2)}\n"
                ),
            },
        ],
        temperature=0.2,
        num_ctx=1536,
        num_predict=220,
    )

    skeptic = await _ollama_chat(
        [
            {"role": "system", "content": system},
            {
                "role": "user",
                "content": (
                    "Role: SKEPTIC\n"
                    "Goal: Argue the claim is REFUTED or at least NOT proven using ONLY evidence.\n"
                    "Output 3-5 concise bullet points referencing evidence by (domain, domain_score).\n"
                    "Point out missing support, ambiguity, or weak sources.\n\n"
                    f"CLAIM:\n{claim_text}\n\n"
                    f"EVIDENCE(JSON):\n{json.dumps(compact, ensure_ascii=False, indent=2)}\n"
                ),
            },
        ],
        temperature=0.2,
        num_ctx=1536,
```

```
        num_predict=220,
    )

    judge_prompt = (
        "Role: JUDGE\n"
        "Decide the verdict based ONLY on evidence.\n"
        "Return STRICT JSON ONLY, no markdown, no extra text.\n"
        "Schema:\n"
        "{\n"
        ' "verdict": "SUPPORTED|REFUTED|NEI",\n'
        ' "confidence": 0.0-1.0,\n'
        ' "summary": "one short sentence justification",\n'
        ' "key_domains": ["domain1","domain2"]\n'
        "}\n\n"
        f"CLAIM:\n{claim_text}\n\n"
        f"PROVER_ARGUMENT:\n{prover}\n\n"
        f"SKEPTIC_ARGUMENT:\n{skeptic}\n\n"
        f"EVIDENCE(JSON):\n{json.dumps(compact, ensure_ascii=False, indent=2)}\n"
    )

    judge_raw = await _ollama_chat(
        [{"role": "system", "content": system}, {"role": "user", "content": judge_prompt}],
        temperature=0.1,
        num_ctx=1536,
        num_predict=180,
    )

    j = _safe_json_parse(judge_raw)

    verdict: Verdict = j.get("verdict", "NEI")
    confidence = j.get("confidence", 0.55)
    summary = j.get("summary", "Insufficient evidence to decide.")
    key_domains = j.get("key_domains", [])

    if verdict not in ("SUPPORTED", "REFUTED", "NEI"):
        verdict = "NEI"

    try:
        confidence = float(confidence)
    except Exception:
        confidence = 0.55

    confidence = max(0.05, min(confidence, 0.95))

    debug = {
        "provider": "ollama",
        "base_url": _env_ollama_base_url(),
        "model": _env_ollama_model(),
        "compact_evidence": compact,
        "prover": prover[:4000],
        "skeptic": skeptic[:4000],
        "judge_raw": judge_raw[:4000],
        "judge_json": j,
        "key_domains": key_domains,
    }

    return verdict, confidence, summary, debug
```

```
async def llm_final_judge(
    claim_text: str,
    evidence_items: List[Dict[str, Any]],
    baseline_verdict: str,
    baseline_confidence: float,
    structured_verdict: str,
    claim_profile: Dict[str, Any] | None = None,
) -> Tuple[Verdict, float, str, Dict[str, Any]]:
    """
    Professional final-answer judge.

    This is a single-pass JSON judge that turns the retrieved evidence into a
    user-facing final answer. It uses the heuristic verdict as a guardrail but
    is free to override it when the evidence supports a different conclusion.
    """
    compact = _compact_final_context(evidence_items, max_items=4)

    system = (
        "You are a professional fact-checking final judge.\n"
        "Use only the provided claim, evidence, and metadata.\n"
        "Prefer caution when evidence is weak, conflicting, or incomplete.\n"
        "Return strict JSON only. No markdown, no extra prose.\n"
    )

    prompt = (
        "Decide the final verdict for the claim.\n"
        "The output must be concise, professional, and suitable for end users.\n"
        "You must choose one verdict from SUPPORTED, REFUTED, or NEI.\n"
        "If the evidence does not clearly decide, choose NEI.\n\n"
        "JSON schema:\n"
        "{\n"
        ' "verdict": "SUPPORTED|REFUTED|NEI",\n'
        ' "confidence": 0.0-1.0,\n'
        ' "summary": "one sentence, professional and neutral",\n'
        ' "key_points": ["short bullet-like reason 1", "reason 2"],\n'
        ' "risk_notes": ["optional caution 1", "optional caution 2"]\n'
        "}\n\n"
        f"CLAIM:\n{claim_text}\n\n"
        f"CLAIM_PROFILE_JSON:\n{json.dumps(claim_profile or {}, ensure_ascii=False, indent=2)}\n\n"
        f"BASELINE_VERDICT:\n{baseline_verdict} ({baseline_confidence:.2f})\n\n"
        f"STRUCTURED_VERDICT:\n{structured_verdict}\n\n"
        f"EVIDENCE_JSON:\n{json.dumps(compact, ensure_ascii=False, indent=2)}\n"
    )

    raw = await _ollama_chat(
        [{"role": "system", "content": system}, {"role": "user", "content": prompt}],
        temperature=0.1,
        num_ctx=2048,
        num_predict=220,
    )

    parsed = _safe_json_parse(raw)
    verdict = _normalize_verdict(parsed.get("verdict"))

    try:
        confidence = float(parsed.get("confidence", baseline_confidence))
    except Exception:
        confidence = baseline_confidence

```

```
    confidence = max(0.05, min(confidence, 0.95))
    summary = str(parsed.get("summary") or "Professional AI final judge completed.").strip()
    if not summary:
        summary = "Professional AI final judge completed."

    debug = {
        "provider": "ollama",
        "base_url": _env_ollama_base_url(),
        "model": _env_ollama_model(),
        "compact_evidence": compact,
        "raw": raw[:4000],
        "json": parsed,
        "key_points": parsed.get("key_points", []),
        "risk_notes": parsed.get("risk_notes", []),
    }

    return verdict, confidence, summary, debug
```

D.3 Primary Pipeline: Semantic Retrieval and Reranking

```
from __future__ import annotations

from typing import Any, Dict, List, Optional, Tuple
import math
import os
import re


def _tokenize(text: str) -> List[str]:
    text = (text or "").lower()
    text = re.sub(r"[^a-z0-9\s]", " ", text)
    stop = {
        "the", "and", "for", "this", "that", "with", "from", "into", "about", "have", "has",
        "been", "were", "which", "when", "where", "while", "than", "also", "their", "they", "them",
    }
    return [t for t in text.split() if len(t) >= 3 and t not in stop]


def _lexical_semantic_score(claim: str, text: str) -> float:
    c = set(_tokenize(claim))
    t = set(_tokenize(text))
    if not c or not t:
        return 0.0
    jaccard = len(c.intersection(t)) / len(c.union(t))
    coverage = len(c.intersection(t)) / max(1, len(c))
    return max(0.0, min(1.0, 0.45 * jaccard + 0.55 * coverage))


def _cosine(a: List[float], b: List[float]) -> float:
    if not a or not b or len(a) != len(b):
        return 0.0
    dot = 0.0
    na = 0.0
```

```
    nb = 0.0
    for x, y in zip(a, b):
        dot += x * y
        na += x * x
        nb += y * y
    if na <= 0.0 or nb <= 0.0:
        return 0.0
    return max(0.0, min(1.0, dot / (math.sqrt(na) * math.sqrt(nb))))


_MODEL = None
_MODEL_LOAD_ERROR: Optional[str] = None


def _get_sentence_transformer_model():
    global _MODEL, _MODEL_LOAD_ERROR
    if _MODEL is not None:
        return _MODEL
    if _MODEL_LOAD_ERROR is not None:
        return None

    # Disabled by default - use lexical fallback for speed
    enabled = os.getenv("FACTVALIDATOR_EMBEDDINGS_ENABLED", "false").strip().lower()
    if enabled not in {"1", "true", "yes", "on"}:
        _MODEL_LOAD_ERROR = "disabled-by-default"
        return None

    model_name = os.getenv("FACTVALIDATOR_EMBEDDING_MODEL", "all-MiniLM-L6-v2").strip()
    try:
        from sentence_transformers import SentenceTransformer # type: ignore

        _MODEL = SentenceTransformer(model_name)
        return _MODEL
    except Exception as ex: # pragma: no cover - environment dependent
        _MODEL_LOAD_ERROR = f"model-load-failed: {ex.__class__.__name__}"
        return None


def semantic_rerank(
    claim: str,
    evidence_items: List[Dict[str, Any]],
    top_k: Optional[int] = None,
) -> Tuple[List[Dict[str, Any]], Dict[str, Any]]:
    """
    Re-rank evidence by semantic similarity.

    - Preferred path: sentence-transformer embeddings cosine similarity.
    - Fallback path: lexical overlap similarity.
    """
    if not evidence_items:
        return [], {"enabled": True, "method": "none", "reason": "no-evidence"}

    items = [dict(e) for e in evidence_items]
    texts = [
        " ".join(
            [
                str(e.get("title") or ""),
                str(e.get("snippet") or ""),
                str(e.get("domain") or ""),
```

```
            ]
        ).strip()
        for e in items
    ]

    model = _get_sentence_transformer_model()
    method = "lexical-fallback"

    if model is not None:
        try:
            emb = model.encode([claim, *texts], normalize_embeddings=True)
            claim_vec = [float(v) for v in emb[0]]
            doc_vecs = [[float(v) for v in row] for row in emb[1:]]
            for e, vec in zip(items, doc_vecs):
                e["semantic_score"] = round(_cosine(claim_vec, vec), 4)
            method = "sentence-transformer"
        except Exception: # pragma: no cover - runtime/model dependent
            for e, txt in zip(items, texts):
                e["semantic_score"] = round(_lexical_semantic_score(claim, txt), 4)
            method = "lexical-fallback"
    else:
        for e, txt in zip(items, texts):
            e["semantic_score"] = round(_lexical_semantic_score(claim, txt), 4)

    items.sort(
        key=lambda e: (
            float(e.get("semantic_score") or 0.0),
            int(e.get("domain_score") or 0),
            int(e.get("overlap") or 0),
        ),
        reverse=True,
    )

    if isinstance(top_k, int) and top_k > 0:
        items = items[:top_k]

    meta = {
        "enabled": True,
        "method": method,
        "model": os.getenv("FACTVALIDATOR_EMBEDDING_MODEL", "all-MiniLM-L6-v2"),
        "scores": [float(e.get("semantic_score") or 0.0) for e in items[:5]],
        "error": _MODEL_LOAD_ERROR,
    }
    return items, meta
```

D.4 Primary Pipeline: Sentiment and Bias Analysis

```
"""Sentiment analysis module for detecting emotional tone in claims."""

from typing import Literal, Dict, Any, List
from dataclasses import dataclass
import re


@dataclass
class SentimentResult:
```

```
    """Sentiment analysis result."""
    score: float # -1 (negative) to +1 (positive)
    label: Literal["positive", "negative", "neutral"]
    emotional_intensity: float # 0-1, how emotionally charged
    flags: List[str] # red flags for emotional manipulation


# Sentiment lexicons for rule-based analysis (VADER-inspired)
_POSITIVE_WORDS = {
    # General positive
    "good", "great", "excellent", "amazing", "wonderful", "fantastic",
    "best", "better", "improved", "success", "successful", "progress",
    "benefits", "beneficial", "positive", "uplifting", "inspiring",
    "genuine", "authentic", "proven", "confirmed", "supported",
    "breakthrough", "innovative", "revolutionary", "advanced",
    "helpful", "supportive", "thriving", "flourishing", "growth",
    "safe", "effective", "reliable", "trustworthy", "confident",
    # Science/research
    "discover", "discovered", "finding", "found", "study", "research",
    "academic", "peer-reviewed", "evidence", "data", "scientific",
    # Success/achievement
    "award", "winning", "triumph", "victory", "achieve", "accomplished",
    "outperform", "leading", "top", "first", "champion",
    # Health/wellbeing
    "healing", "cure", "recovery", "wellness", "healthy", "vital",
    "natural", "organic", "pure", "clean", "fresh",
    # Trust/credibility
    "transparent", "honest", "integrity", "ethical", "legitimate",
    "verified", "certified", "official", "authorized", "valid",
    # Quality
    "quality", "premium", "superior", "exceptional", "outstanding",
    "remarkable", "impressive", "stunning", "beautiful", "elegant",
}

_NEGATIVE_WORDS = {
    # General negative
    "bad", "terrible", "horrible", "awful", "worst", "worse",
    "failed", "failure", "disaster", "crisis", "emergency",
    "danger", "dangerous", "threat", "scary", "terrifying",
    "evil", "wicked", "corrupt", "corrupt", "conspiracy",
    "hoax", "fake", "false", "lie", "deceive", "betrayal",
    "harmful", "toxic", "poisoned", "lethal", "deadly",
    "collapse", "ruin", "destroyed", "devastated", "catastrophic",
    "controversial", "blamed", "accused", "guilty", "criminal",
    "problematic", "misleading", "wrong", "flawed", "defective",
    # Health/disease
    "disease", "illness", "sick", "infected", "virus", "pandemic",
    "epidemic", "plague", "outbreak", "contagion", "contamination",
    "side-effect", "toxin", "poison", "overdose", "death", "dying",
    # Distrust
    "suspicious", "doubt", "questionable", "unreliable", "untrustworthy",
    "fraudulent", "scam", "scheme", "corruption", "bribery",
    "censorship", "suppression", "coverup", "hidden", "secret",
    # Decline/failure
    "decline", "declining", "falling", "crash", "plummet", "bankruptcy",
    "unemployment", "poverty", "inequality", "suffering", "crisis",
    # Violation/abuse
    "abuse", "assault", "violation", "attack", "aggression", "violence",
    "oppression", "discrimination", "injustice", "exploitation",
```

```
    # Intensity negatives
    "disgusting", "repugnant", "abhorrent", "revolting", "sickening",
    "grotesque", "vile", "despicable", "heinous", "monstrous",
}

# Emotional amplifiers and intensifiers
_INTENSIFIERS = {
    "extremely", "very", "incredibly", "absolutely", "definitely",
    "completely", "totally", "utterly", "pure", "sheer",
    "massive", "huge", "enormous", "tremendous", "shocking",
    # Additional intensifiers
    "profoundly", "deeply", "severely", "dramatically", "significantly",
    "remarkably", "exceptionally", "extraordinarily", "exceptionally",
    "undeniably", "unquestionably", "indisputably", "incontrovertibly",
    "devastatingly", "overwhelmingly", "stunningly", "shockingly",
}

# Red flag words for emotional manipulation
_MANIPULATION_FLAGS = {
    # Sensationalism
    "shocking": "sensationalism",
    "bombshell": "sensationalism",
    "exposed": "alarmism",
    "breaking": "sensationalism",
    "urgent": "sensationalism",
    "must-see": "sensationalism",
    "incredible": "sensationalism",

    # Conspiracy/Cover-up
    "coverup": "conspiracy-thinking",
    "hidden": "conspiracy-thinking",
    "suppressed": "conspiracy-thinking",
    "shadow": "conspiracy-thinking",
    "elite": "conspiracy-thinking",

    # Alarmism
    "wake-up": "alarmism",
    "awakening": "conspiratorial",
    "warning": "alarmism",
    "beware": "alarmism",
    "danger": "alarmism",

    # False consensus
    "everyone-knows": "false-consensus",
    "obviously": "false-certainty",
    "clearly": "false-certainty",
    "evidently": "false-certainty",

    # Certainty claims
    "proof": "false-certainty",
    "guaranteed": "false-certainty",
    "proven": "false-certainty",
    "undeniable": "false-certainty",
    "incontrovertible": "false-certainty",

    # Manipulation tactics
    "miracle": "exaggeration",
    "cure-all": "exaggeration",
    "secret-weapon": "mystique",
```

```
    "ancient-secret": "mystique",
    "they-don't-want": "persecution-narrative",

    # Propaganda framing
    "mainstream-media": "propaganda-framing",
    "mainstream": "propaganda-framing",
    "fake-news": "propaganda-framing",
    "lamestream": "propaganda-framing",
    "legacy-media": "propaganda-framing",
    "state-media": "propaganda-framing",

    # Demonization
    "evil": "demonization",
    "traitor": "demonization",
    "enemy": "demonization",
    "villain": "demonization",
    "scum": "demonization",
    "corrupt": "demonization",

    # Ad hominem / Dismissiveness
    "sheeple": "ad-hominem",
    "sheep": "ad-hominem",
    "idiot": "ad-hominem",
    "moron": "ad-hominem",
    "brainwashed": "ad-hominem",
    "asleep": "ad-hominem",

    # Dismissal of evidence
    "scam": "dismissiveness",
    "hoax": "dismissiveness",
    "propaganda": "propaganda-framing",
    "lies": "dismissiveness",

    # Persecution narratives
    "censored": "persecution-narrative",
    "silenced": "persecution-narrative",
    "attacked": "persecution-narrative",
    "hunted": "persecution-narrative",
}


def analyze_sentiment(text: str) -> SentimentResult:
    """
    Analyze sentiment of text using rule-based approach.

    Returns:
        SentimentResult with score (-1 to +1), label, intensity, and flags
    """
    if not text or len(text.strip()) < 5:
        return SentimentResult(
            score=0.0,
            label="neutral",
            emotional_intensity=0.0,
            flags=[]
        )

    low_text = text.lower()
    words = re.findall(r'\b\w+\b', low_text)

```

```
    # Count sentiment words
    positive_count = sum(1 for w in words if w in _POSITIVE_WORDS)
    negative_count = sum(1 for w in words if w in _NEGATIVE_WORDS)

    # Check for intensifiers
    intensifier_count = sum(1 for w in words if w in _INTENSIFIERS)

    # Calculate base sentiment score
    total_words = len(words)
    if total_words == 0:
        return SentimentResult(
            score=0.0,
            label="neutral",
            emotional_intensity=0.0,
            flags=[]
        )

    # Normalize scores
    pos_ratio = positive_count / total_words
    neg_ratio = negative_count / total_words

    # Calculate sentiment score (-1 to +1)
    sentiment_score = pos_ratio - neg_ratio
    sentiment_score = max(-1.0, min(1.0, sentiment_score))

    # Determine label
    if sentiment_score > 0.1:
        label = "positive"
    elif sentiment_score < -0.1:
        label = "negative"
    else:
        label = "neutral"

    # Calculate emotional intensity (how emotionally charged)
    emotional_intensity = (positive_count + negative_count + intensifier_count) / (total_words * 0.5)
    emotional_intensity = min(1.0, emotional_intensity)

    # Detect manipulation flags
    flags = []
    for flag_word, flag_category in _MANIPULATION_FLAGS.items():
        if flag_word in low_text:
            flags.append(flag_category)

    # Remove duplicates
    flags = list(set(flags))

    return SentimentResult(
        score=round(sentiment_score, 3),
        label=label,
        emotional_intensity=round(emotional_intensity, 3),
        flags=flags
    )


def batch_analyze_sentiment(texts: List[str]) -> List[SentimentResult]:
    """Analyze sentiment of multiple texts."""
    return [analyze_sentiment(text) for text in texts]


```

```python
def estimate_bias_risk(sentiment_label: str, emotional_intensity: float, flags: List[str]) -> str:
    """
    Estimate bias/manipulation risk based on sentiment characteristics.

    Returns: "low", "medium", or "high"
    """
    risk_score = 0

    # Strong positive or negative sentiment + high emotion = higher risk
    if sentiment_label in ("positive", "negative") and emotional_intensity > 0.6:
        risk_score += 2
    elif sentiment_label in ("positive", "negative"):
        risk_score += 1

    # Manipulation flags increase risk
    risk_score += len(flags) * 0.5

    if risk_score >= 2:
        return "high"
    elif risk_score >= 1:
        return "medium"
    else:
        return "low"


def calculate_sentiment_misinformation_adjustment(
    sentiment_result: SentimentResult,
) -> float:
    """
    Calculate adjustment factor (0-0.4) to apply to misinformation likelihood based on sentiment.

    Higher emotional intensity + manipulative flags = higher adjustment.
    This can be added to the base misinformation likelihood.

    Returns:
        float: Adjustment factor between 0 and 0.4
    """
    adjustment = 0.0

    # Base adjustment from emotional intensity
    if sentiment_result.emotional_intensity > 0.7:
        adjustment += 0.2 # High emotion = 20% boost
    elif sentiment_result.emotional_intensity > 0.5:
        adjustment += 0.1 # Medium emotion = 10% boost

    # Negative sentiment slightly increases risk
    if sentiment_result.label == "negative":
        adjustment += 0.05

    # Each manipulation flag adds 5% adjustment
    flag_adjustment = min(0.15, len(sentiment_result.flags) * 0.05)
    adjustment += flag_adjustment

    # Cap at 0.4
    return min(0.4, adjustment)


def get_sentiment_summary(sentiment_result: SentimentResult) -> str:
    """Generate a human-readable summary of sentiment analysis."""
```

```
    intensity_level = "highly" if sentiment_result.emotional_intensity > 0.6 else "moderately" if
        sentiment_result.emotional_intensity > 0.3 else "slightly"

    summary = f"This claim is {intensity_level} emotionally charged with {sentiment_result.label} sentiment
        "

    if sentiment_result.flags:
        summary += f" with red flags detected: {', '.join(sentiment_result.flags[:3])}"
        if len(sentiment_result.flags) > 3:
            summary += f" and {len(sentiment_result.flags) - 3} more"
        summary += " (possible manipulation)."
    else:
        summary += "."

    return summary
```

D.5 Primary Pipeline: Baseline Heuristics

```
"""
Baseline implementations for Fact Validator comparison.

Includes:
- Random baseline (lower bound)
- Keyword-matching baseline (strawman)
- Simple heuristic baseline
- Reference implementations (Google Fact Check, ClaimBuster patterns)
"""

from typing import List, Dict, Optional
from dataclasses import dataclass
from enum import Enum
import random
import re
from app.evaluation import PredictionResult, VerdictLabel


class BaselineType(str, Enum):
    """Types of baselines."""
    RANDOM = "random"
    KEYWORD = "keyword_matching"
    LENGTH_HEURISTIC = "length_heuristic"
    SENTIMENT_HEURISTIC = "sentiment_heuristic"
    MAJORITY_CLASS = "majority_class"


@dataclass
class BaselineConfig:
    """Configuration for baseline models."""
    baseline_type: BaselineType
    random_seed: int = 42


class RandomBaseline:
    """Lower-bound baseline: random verdict assignment."""

    def __init__(self, seed: int = 42, distribution: Optional[Dict[str, float]] = None):
```

```
        """
        Initialize random baseline.

        Args:
            seed: Random seed for reproducibility
            distribution: Optional probability distribution over labels
                        e.g., {"SUPPORTED": 0.33, "REFUTED": 0.33, "NEI": 0.34}
        """
        self.seed = seed
        random.seed(seed)
        self.distribution = distribution or {
            VerdictLabel.SUPPORTED: 1/3,
            VerdictLabel.REFUTED: 1/3,
            VerdictLabel.NEI: 1/3
        }

    def predict(self, claim_text: str) -> tuple[str, float]:
        """
        Random prediction.

        Returns: (verdict_label, confidence_0_to_100)
        """
        labels = list(self.distribution.keys())
        probs = list(self.distribution.values())

        label = random.choices(labels, weights=probs, k=1)[0]
        confidence = random.uniform(30, 70) # Random confidence

        return label, confidence


class KeywordBaseline:
    """Strawman baseline: keyword-matching heuristics."""

    # Define keyword patterns for each verdict
    SUPPORTED_KEYWORDS = [
        r'\b(confirmed|verified|true|correct|supported|accurate|valid|proven)\b',
        r'\b(shows|demonstrates|proves|establishes)\b',
        r'\b(research|study|evidence|findings|data)\s+(shows|confirms|supports)',
    ]

    REFUTED_KEYWORDS = [
        r'\b(false|debunked|myth|misconception|hoax|disproven|incorrect)\b',
        r'\b(denied|refuted|contradicts|contradicted|wrong|inaccurate)\b',
        r'\b(no evidence|lacks evidence|debunk|fake)\b',
    ]

    NEI_KEYWORDS = [
        r'\b(unclear|uncertain|unknown|may|might|could|possibly|perhaps)\b',
        r'\b(insufficient|not enough|lack of|insufficient)\s+(evidence|information)',
        r'\b(under investigation|unclear|ambiguous)\b',
    ]

    def __init__(self):
        self.supported_patterns = [re.compile(p, re.IGNORECASE) for p in self.SUPPORTED_KEYWORDS]
        self.refuted_patterns = [re.compile(p, re.IGNORECASE) for p in self.REFUTED_KEYWORDS]
        self.nei_patterns = [re.compile(p, re.IGNORECASE) for p in self.NEI_KEYWORDS]

    def predict(self, claim_text: str) -> tuple[str, float]:
```

```
        """
        Predict using keyword matching.

        Returns: (verdict_label, confidence_based_on_matches)
        """
        supported_matches = sum(1 for p in self.supported_patterns if p.search(claim_text))
        refuted_matches = sum(1 for p in self.refuted_patterns if p.search(claim_text))
        nei_matches = sum(1 for p in self.nei_patterns if p.search(claim_text))

        total_matches = supported_matches + refuted_matches + nei_matches

        if total_matches == 0:
            # No keywords found - default to NEI
            return VerdictLabel.NEI, 20.0

        # Majority vote
        matches = {
            VerdictLabel.SUPPORTED: supported_matches,
            VerdictLabel.REFUTED: refuted_matches,
            VerdictLabel.NEI: nei_matches
        }

        verdict = max(matches, key=matches.get)
        confidence = (matches[verdict] / total_matches) * 100

        return verdict, confidence


class LengthHeuristic:
    """Simple heuristic: longer claims tend to be more specific/refutable."""

    def predict(self, claim_text: str) -> tuple[str, float]:
        """
        Predict based on claim length.

        Short claims (< 100 chars): usually NEI or supported
        Medium claims (100-200): mixed
        Long claims (> 200): often refutable
        """
        length = len(claim_text)

        if length < 100:
            return VerdictLabel.SUPPORTED, 50.0
        elif length < 200:
            return VerdictLabel.NEI, 45.0
        else:
            return VerdictLabel.REFUTED, 48.0


class SentimentHeuristic:
    """Heuristic based on emotional language."""

    NEGATIVE_WORDS = ['terrible', 'horrible', 'dangerous', 'evil', 'worst', 'disgusting']
    POSITIVE_WORDS = ['great', 'excellent', 'amazing', 'wonderful', 'best', 'fantastic']
    NEUTRAL_WORDS = ['shows', 'indicates', 'demonstrates', 'suggests', 'indicates']

    def predict(self, claim_text: str) -> tuple[str, float]:
        """
        Predict based on sentiment.
```

```
        Emotional language →likely misinformation (REFUTED)
        Neutral language →likely accurate (SUPPORTED)
        """
        text_lower = claim_text.lower()

        negative_count = sum(1 for w in self.NEGATIVE_WORDS if w in text_lower)
        positive_count = sum(1 for w in self.POSITIVE_WORDS if w in text_lower)
        neutral_count = sum(1 for w in self.NEUTRAL_WORDS if w in text_lower)

        total_sentiment = negative_count + positive_count + neutral_count

        if total_sentiment == 0:
            return VerdictLabel.NEI, 40.0

        if negative_count > positive_count:
            return VerdictLabel.REFUTED, min(70.0, 40 + negative_count * 10)
        elif positive_count > negative_count:
            return VerdictLabel.SUPPORTED, min(70.0, 40 + positive_count * 10)
        else:
            return VerdictLabel.SUPPORTED, 55.0


class MajorityClassBaseline:
    """Predict most common label from training set."""

    def __init__(self, majority_label: str = VerdictLabel.SUPPORTED):
        """
        Initialize with majority class.

        Args:
            majority_label: Most common label in training data
        """
        self.majority_label = majority_label

    def predict(self, claim_text: str) -> tuple[str, float]:
        """Always predict majority class."""
        return self.majority_label, 50.0


class BaselineComparison:
    """Compare all baselines against ground truth."""

    def __init__(self):
```

D.6 Primary Pipeline: Statistics and Confidence Intervals

```
"""
Statistical analysis module for rigorous evaluation.

Provides:
- Confidence intervals (95%)
- Significance testing (paired t-test, one-sample)
```

```
- Effect size calculations (Cohen's d, Hedges' g)
- Bootstrap resampling for robust estimates
"""

import math
from typing import List, Dict, Tuple, Optional
from dataclasses import dataclass
from scipy import stats
import numpy as np


@dataclass
class ConfidenceInterval:
    """Confidence interval result."""
    mean: float
    lower: float
    upper: float
    confidence_level: float
    margin_of_error: float

    def __str__(self) -> str:
        return f"{self.mean:.4f} [{self.lower:.4f}, {self.upper:.4f}]"


@dataclass
class SignificanceTestResult:
    """Result from significance test."""
    test_name: str
    t_statistic: float
    p_value: float
    degrees_freedom: int
    mean_difference: float
    is_significant: bool # At alpha=0.05
    effect_size: float # Cohen's d or similar

    def __str__(self) -> str:
        sig_marker = "✓" if self.is_significant else "×"
        return f"{sig_marker} {self.test_name}: t({self.degrees_freedom})={self.t_statistic:.3f}, p={self.
            p_value:.4f}, d={self.effect_size:.3f}"


class StatisticalAnalyzer:
    """Rigorous statistical analysis for evaluation."""

    @staticmethod
    def confidence_interval(
        samples: List[float],
        confidence_level: float = 0.95,
        method: str = "t"
    ) -> ConfidenceInterval:
        """
        Calculate confidence interval using t-distribution.

        Args:
            samples: List of accuracy scores or metrics
            confidence_level: 0.95 for 95% CI
            method: "t" for t-distribution (preferred for small n), "z" for normal

        Returns:
```

```
            ConfidenceInterval object
        """
        n = len(samples)
        mean = np.mean(samples)
        std_err = np.std(samples, ddof=1) / np.sqrt(n)

        if method == "t":
            alpha = 1 - confidence_level
            t_crit = stats.t.ppf(1 - alpha/2, df=n-1)
        else: # method == "z"
            alpha = 1 - confidence_level
            t_crit = stats.norm.ppf(1 - alpha/2)

        margin_error = t_crit * std_err

        return ConfidenceInterval(
            mean=mean,
            lower=mean - margin_error,
            upper=mean + margin_error,
            confidence_level=confidence_level,
            margin_of_error=margin_error
        )

    @staticmethod
    def bootstrap_ci(
        samples: List[float],
        confidence_level: float = 0.95,
        n_bootstrap: int = 10000,
        metric_fn=None
    ) -> ConfidenceInterval:
        """
        Calculate confidence interval using bootstrap resampling.

        More robust for non-normal distributions.

        Args:
            samples: Original data
            confidence_level: 0.95 for 95% CI
            n_bootstrap: Number of bootstrap samples
            metric_fn: Function to compute on each bootstrap sample (default: mean)

        Returns:
            ConfidenceInterval object
        """
        if metric_fn is None:
            metric_fn = np.mean

        n = len(samples)
        np.random.seed(42) # Reproducibility

        bootstrap_metrics = []
        for _ in range(n_bootstrap):
            bootstrap_sample = np.random.choice(samples, size=n, replace=True)
            metric = metric_fn(bootstrap_sample)
            bootstrap_metrics.append(metric)

        mean = np.mean(samples)
        alpha = 1 - confidence_level
        lower_percentile = alpha / 2 * 100
```

```
        upper_percentile = (1 - alpha/2) * 100

        lower = np.percentile(bootstrap_metrics, lower_percentile)
        upper = np.percentile(bootstrap_metrics, upper_percentile)
        margin_error = (upper - lower) / 2

        return ConfidenceInterval(
            mean=mean,
            lower=lower,
            upper=upper,
            confidence_level=confidence_level,
            margin_of_error=margin_error
        )

    @staticmethod
    def paired_t_test(
        system_scores: List[float],
        baseline_scores: List[float],
        alternative: str = "greater", # greater, less, two-sided
        alpha: float = 0.05
    ) -> SignificanceTestResult:
        """
        Paired t-test: is system significantly better than baseline?

        Args:
            system_scores: Accuracy scores from system
            baseline_scores: Accuracy scores from baseline
            alternative: "greater" (system > baseline), "less", "two-sided"
            alpha: Significance level (0.05 for 95% confidence)

        Returns:
            SignificanceTestResult
        """
        assert len(system_scores) == len(baseline_scores)

        # Compute paired differences
        diffs = np.array(system_scores) - np.array(baseline_scores)
        n = len(diffs)

        # T-statistic
        mean_diff = np.mean(diffs)
        std_err = np.std(diffs, ddof=1) / np.sqrt(n)

        if std_err == 0:
            t_stat = float('inf') if mean_diff > 0 else float('-inf')
        else:
            t_stat = mean_diff / std_err

        df = n - 1

        # P-value
        if alternative == "greater":
            p_value = 1 - stats.t.cdf(t_stat, df)
        elif alternative == "less":
            p_value = stats.t.cdf(t_stat, df)
        else: # two-sided
            p_value = 2 * (1 - stats.t.cdf(abs(t_stat), df))

        # Effect size (Cohen's d)
```

```
        cohens_d = StatisticalAnalyzer.cohens_d(system_scores, baseline_scores)

        is_significant = p_value < alpha

        return SignificanceTestResult(
            test_name="Paired t-test",
            t_statistic=t_stat,
            p_value=p_value,
            degrees_freedom=df,
            mean_difference=mean_diff,
            is_significant=is_significant,
            effect_size=cohens_d
        )

    @staticmethod
    def cohens_d(
        group1: List[float],
        group2: List[float],
```

D.7 Primary Pipeline: Retrieval Security (URL Validation)

```
"""Security and health check utilities."""
import httpx
import os
from typing import Tuple, Optional
from datetime import datetime, timedelta


class OllamaHealthCheck:
    """Check Ollama service availability."""

    def __init__(self):
        self.last_check: Optional[datetime] = None
        self.last_result: bool = False
        self.check_cache_ttl = 30 # Cache result for 30 seconds

    async def is_available(self) -> bool:
        """Check if Ollama service is available."""
        now = datetime.utcnow()

        # Use cached result if still fresh
        if self.last_check and (now - self.last_check).seconds < self.check_cache_ttl:
            return self.last_result

        ollama_url = os.getenv("OLLAMA_BASE_URL", "http://127.0.0.1:11434").strip()

        try:
            response = httpx.get(
                f"{ollama_url}/api/tags",
                timeout=5.0,
                follow_redirects=True,
                trust_env=False,
            )
```

```
            success = response.status_code == 200
        except Exception:
            success = False

        self.last_check = now
        self.last_result = success
        return success


class RateLimiter:
    """Simple in-memory rate limiter."""

    def __init__(self, requests_per_minute: int = 60):
        self.requests_per_minute = requests_per_minute
        self.request_times: dict = {} # ip -> [timestamps]

    def is_allowed(self, client_ip: str) -> bool:
        """Check if request from client is allowed."""
        now = datetime.utcnow()
        cutoff = now - timedelta(minutes=1)

        if client_ip not in self.request_times:
            self.request_times[client_ip] = []

        # Remove old timestamps
        self.request_times[client_ip] = [
            ts for ts in self.request_times[client_ip] if ts > cutoff
        ]

        # Check limit
        if len(self.request_times[client_ip]) >= self.requests_per_minute:
            return False

        # Add current request
        self.request_times[client_ip].append(now)
        return True


# Global instances
ollama_health = OllamaHealthCheck()
rate_limiter = RateLimiter(requests_per_minute=100)
```

D.8 Primary Pipeline: Source Routes

```
from __future__ import annotations

from datetime import datetime, timezone
from typing import Any, Dict

import tldextract
from fastapi import APIRouter, HTTPException

# IMPORTANT: your backend already uses this name in main.py
# and earlier you saw: "from app.credibility import score_domain_rubric"
from app.credibility import score_domain_rubric

```

```
router = APIRouter(tags=["source"])


def utc_now_iso() -> str:
    return datetime.now(timezone.utc).isoformat().replace("+00:00", "Z")


def base_domain(domain: str) -> str:
    d = (domain or "").lower().strip().replace("www.", "")
    ext = tldextract.extract(d)
    if ext.domain and ext.suffix:
        return f"{ext.domain}.{ext.suffix}"
    return d


@router.get("/source/{domain}")
def source_score(domain: str) -> Dict[str, Any]:
    d = (domain or "").strip().lower().replace("www.", "")
    if not d:
        raise HTTPException(status_code=400, detail="domain is required")

    bd = base_domain(d)

    # credibility.py returns a dataclass-like object with score/label/reasons
    cs = score_domain_rubric(d)

    return {
        "domain": d,
        "base_domain": bd,
        "score": int(cs.score),
        "label": str(cs.label),
        "reasons": dict(cs.reasons or {}),
        "timestamp_utc": utc_now_iso(),
        "disclaimer": "Credibility score is heuristic; treat it as a risk signal, not ground truth.",
    }
```

D.9 Primary Pipeline: Configuration

```
"""Feature flags and configuration management."""
import os
from typing import Dict, Any


class Config:
    """Application configuration with feature flags."""

    # Feature flags
    FEATURE_DEBATE_MODE = os.getenv("FEATURE_DEBATE_MODE", "true").lower() in ("true", "1", "yes")
    FEATURE_CACHING = os.getenv("FEATURE_CACHING", "true").lower() in ("true", "1", "yes")
    FEATURE_RATE_LIMITING = os.getenv("FEATURE_RATE_LIMITING", "true").lower() in ("true", "1", "yes")
    FEATURE_STRUCTURED_LOGGING = os.getenv("FEATURE_STRUCTURED_LOGGING", "true").lower() in ("true", "1", "
        yes")

    # Limits
    MAX_INPUT_URL_LENGTH = 2048
    MAX_INPUT_TEXT_LENGTH = 50000
```

```
    MAX_CLAIMS_HARD_LIMIT = 20
    MAX_EVIDENCE_PER_CLAIM_LIMIT = 10

    # Timeouts (seconds)
    SERPAPI_TIMEOUT = int(os.getenv("SERPAPI_TIMEOUT", "30"))
    OLLAMA_TIMEOUT = int(os.getenv("OLLAMA_TIMEOUT", "120"))

    # Ollama config
    OLLAMA_ENABLED = os.getenv("OLLAMA_ENABLED", "false").lower() in ("true", "1", "yes")

    @staticmethod
    def get_all() -> Dict[str, Any]:
        """Return all config as dict for logging/debugging."""
        return {
            "feature_debate_mode": Config.FEATURE_DEBATE_MODE,
            "feature_caching": Config.FEATURE_CACHING,
            "feature_rate_limiting": Config.FEATURE_RATE_LIMITING,
            "feature_structured_logging": Config.FEATURE_STRUCTURED_LOGGING,
            "max_input_url_length": Config.MAX_INPUT_URL_LENGTH,
            "max_input_text_length": Config.MAX_INPUT_TEXT_LENGTH,
            "ollama_enabled": Config.OLLAMA_ENABLED,
        }
```

D.10 Prototype: Typed Evidence Graph

```
from __future__ import annotations

from hashlib import sha256
from typing import Any, Dict, List, Tuple


RELATION_TYPES = {"SUPPORTS", "REFUTES", "UNDERCUTS", "DEPENDS_ON", "CITES"}


def passage_hash(text: str) -> str:
    return sha256((text or "").encode("utf-8")).hexdigest()


def _evidence_id(index: int, evidence: Dict[str, Any]) -> str:
    return f"evidence:{index}:{passage_hash(str(evidence.get('passage') or evidence.get('snippet') or ''))
        [:12]}"


def _relation_for_evidence(evidence: Dict[str, Any]) -> str | None:
    if evidence.get("retrieval_status") != "retrieved":
        return None
    if not evidence.get("numeric_match"):
        return None

    directness = float(evidence.get("directness_score") or 0.0)
    stance = str(evidence.get("stance") or "neutral")
    passage = str(evidence.get("passage") or evidence.get("snippet") or "").lower()

    if stance == "support" and directness >= 0.30:
        return "SUPPORTS"
    if stance == "refute" and directness >= 0.20:
```

```
        if any(marker in passage for marker in ("context", "exception", "only", "not necessarily", "depends
            on")):
            return "UNDERCUTS"
        return "REFUTES"
    return None


def build_evidence_graph(
    claim_text: str,
    atomic_claims: List[str],
    evidence: List[Dict[str, Any]],
    retrieval_status: str,
) -> Dict[str, Any]:
    claim_id = f"claim:{passage_hash(claim_text)[:12]}"
    nodes: List[Dict[str, Any]] = [{"id": claim_id, "type": "CLAIM", "text": claim_text}]
    edges: List[Dict[str, Any]] = []

    for index, atomic_claim in enumerate(atomic_claims):
        atomic_id = f"atomic:{index}:{passage_hash(atomic_claim)[:12]}"
        nodes.append({"id": atomic_id, "type": "ATOMIC_CLAIM", "text": atomic_claim})
        edges.append({"source": claim_id, "target": atomic_id, "type": "DEPENDS_ON"})

    relation_counts = {relation: 0 for relation in RELATION_TYPES}
    for index, item in enumerate(evidence):
        evidence_id = _evidence_id(index, item)
        nodes.append(
            {
                "id": evidence_id,
                "type": "EVIDENCE_PASSAGE",
                "url": item.get("url"),
                "content_hash": item.get("content_hash") or passage_hash(str(item.get("passage") or "")),
                "retrieval_status": item.get("retrieval_status"),
            }
        )
        edges.append({"source": claim_id, "target": evidence_id, "type": "CITES"})
        relation_counts["CITES"] += 1

        relation = _relation_for_evidence(item)
        if relation:
            edges.append(
                {
                    "source": evidence_id,
                    "target": claim_id,
                    "type": relation,
                    "quality": item.get("quality_score"),
                    "reason": "Passage-level alignment with matched claim factors.",
                }
            )
            relation_counts[relation] += 1

    support = [
        float(item.get("quality_score") or 0.0)
        for item in evidence
        if _relation_for_evidence(item) == "SUPPORTS"
    ]
    attacks = [
        float(item.get("quality_score") or 0.0)
        for item in evidence
        if _relation_for_evidence(item) in {"REFUTES", "UNDERCUTS"}
```

```
    ]
    conflict = bool(support and attacks and abs(max(support) - max(attacks)) <= 12.0)

    return {
        "version": "1.0",
        "retrieval_status": retrieval_status,
        "nodes": nodes,
        "edges": edges,
        "relation_counts": relation_counts,
        "unresolved_conflict": conflict,
        "decision_basis": "fetched_passages_only",
    }


def adjudicate_graph(graph: Dict[str, Any]) -> Tuple[str | None, List[str]]:
    status = str(graph.get("retrieval_status") or "")
    counts = dict(graph.get("relation_counts") or {})
    reasons: List[str] = []
    if status in {"search_failed", "search_unavailable"}:
        return "NEI", ["Evidence retrieval failed or was unavailable; absence of results is not evidence of
            absence."]
    if graph.get("unresolved_conflict"):
        return "CONFLICTING", ["Direct support and attack relations remain unresolved in the evidence graph
            ."]
    if not any(int(counts.get(key, 0)) for key in ("SUPPORTS", "REFUTES", "UNDERCUTS")):
        return "NEI", ["No fetched passage grounded a support, refutation, or undercut relation."]
    return None, reasons
```

D.11 Prototype: Graph Auditor

```
from __future__ import annotations

from typing import Any, Dict, List


DECISIVE_RELATIONS = {"SUPPORTS", "REFUTES", "UNDERCUTS"}


def audit_evidence_graph(
    graph: Dict[str, Any],
    evidence: List[Dict[str, Any]],
    claim_profile: Dict[str, Any],
) -> Dict[str, Any]:
    """Validate that an automated claim decision has a complete evidence basis."""
    checked_rules = [
        "retrieval-status",
        "provenance",
        "relation-grounding",
        "numeric-factor-coverage",
        "conflict-resolution",
        "source-independence",
    ]
    violations: List[str] = []
    status = str(graph.get("retrieval_status") or "")
    relation_counts = dict(graph.get("relation_counts") or {})
    retrieved = [item for item in evidence if item.get("retrieval_status") == "retrieved"]
```

```
    decisive_edges = sum(int(relation_counts.get(relation, 0)) for relation in DECISIVE_RELATIONS)

    if status in {"search_failed", "search_unavailable"}:
        violations.append("Retrieval did not complete successfully.")
    if not retrieved:
        violations.append("No fetched evidence passage is available for this claim.")
    if any(not item.get("content_hash") or not item.get("url") for item in retrieved):
        violations.append("A retrieved passage lacks a URL or content hash.")
    if retrieved and decisive_edges == 0:
        violations.append("No retrieved passage establishes a grounded argumentative relation.")
    if claim_profile.get("numbers") and not any(bool(item.get("numeric_match")) for item in retrieved):
        violations.append("Numeric claim factors are not matched by retrieved evidence.")
    if graph.get("unresolved_conflict"):
        violations.append("Comparable support and attack relations remain unresolved.")

    decisive_items = [
        item
        for item in retrieved
        if str(item.get("stance") or "") in {"support", "refute"}
    ]
    clusters = {item.get("independence_cluster") for item in decisive_items if item.get("
        independence_cluster")}
    if len(decisive_items) >= 2 and len(clusters) < 2:
        violations.append("Multiple decisive passages are correlated within one evidence cluster.")

    decision = "PROCEED"
    if status in {"search_failed", "search_unavailable"} or decisive_edges == 0:
        decision = "NEI"
    elif graph.get("unresolved_conflict"):
        decision = "CONFLICTING"
    elif violations:
        decision = "HUMAN_REVIEW"

    return {
        "version": "graph-auditor-v1",
        "passed": decision == "PROCEED",
        "decision": decision,
        "violations": violations,
        "checked_rules": checked_rules,
        "evidence_clusters": sorted(cluster for cluster in clusters if cluster),
    }
```

D.12 Prototype: Evidence Independence Clustering

```
from __future__ import annotations

from typing import Any, Dict, List
import re


def _tokens(text: str) -> set[str]:
    return {
        token
        for token in re.sub(r"[^a-z0-9\s]", " ", (text or "").lower()).split()
        if len(token) >= 4
    }
```

```
13
14
15  def _similarity(left: str, right: str) -> float:
16      left_tokens = _tokens(left)
17      right_tokens = _tokens(right)
18      if not left_tokens or not right_tokens:
19          return 0.0
20      return len(left_tokens.intersection(right_tokens)) / len(left_tokens.union(right_tokens))
21
22
23  def cluster_evidence(evidence: List[Dict[str, Any]], duplicate_threshold: float = 0.78) -> List[Dict[str,
        Any]]:
24      """Assign transparent source-independence clusters to retrieved passages."""
25      clusters: List[Dict[str, Any]] = []
26      for item in evidence:
27          if item.get("retrieval_status") != "retrieved":
28              item["independence_cluster"] = None
29              item["independence_reason"] = "Passage was not retrieved."
30              continue
31
32          domain = str(item.get("base_domain") or item.get("domain") or "").lower()
33          passage = str(item.get("passage") or "")
34          matched_cluster = None
35          match_reason = ""
36          for cluster in clusters:
37              if domain and domain == cluster["domain"]:
38                  matched_cluster = cluster
39                  match_reason = "Same base domain."
40                  break
41              if _similarity(passage, cluster["representative_passage"]) >= duplicate_threshold:
42                  matched_cluster = cluster
43                  match_reason = "Near-duplicate passage content."
44                  break
45
46          if matched_cluster is None:
47              matched_cluster = {
48                  "id": f"cluster:{len(clusters) + 1}",
49                  "domain": domain,
50                  "representative_passage": passage,
51                  "members": [],
52              }
53              clusters.append(matched_cluster)
54              match_reason = "New independent source cluster."
55
56          matched_cluster["members"].append(item)
57          item["independence_cluster"] = matched_cluster["id"]
58          item["independence_reason"] = match_reason
59
60      return [
61          {
62              "id": cluster["id"],
63              "domain": cluster["domain"],
64              "evidence_count": len(cluster["members"]),
65              "urls": [member.get("url") for member in cluster["members"]],
66          }
67          for cluster in clusters
68      ]
```

D.13 Prototype: Relation Classifier Contract

```python
from __future__ import annotations

from typing import Any, Dict, Tuple
import os

from app.analysis_features import detect_stance


DEFAULT_NLI_MODEL = "facebook/bart-large-mnli"
_PIPELINE: Any = None
_LOAD_ERROR: str | None = None


def _nli_enabled() -> bool:
    return os.getenv("FACTVALIDATOR_NLI_ENABLED", "false").strip().lower() in {"1", "true", "yes", "on"}


def _get_pipeline() -> Any:
    global _PIPELINE, _LOAD_ERROR
    if _PIPELINE is not None:
        return _PIPELINE
    if _LOAD_ERROR is not None or not _nli_enabled():
        return None
    try:
        from transformers import pipeline # type: ignore

        _PIPELINE = pipeline(
            "text-classification",
            model=os.getenv("FACTVALIDATOR_NLI_MODEL", DEFAULT_NLI_MODEL).strip(),
            return_all_scores=True,
        )
        return _PIPELINE
    except Exception as exc: # pragma: no cover - model/environment dependent
        _LOAD_ERROR = f"{exc.__class__.__name__}: {exc}"
        return None


def _map_label(label: str) -> str:
    normalized = (label or "").strip().lower()
    if "entail" in normalized:
        return "support"
    if "contradict" in normalized:
        return "refute"
    return "neutral"


def classify_relation(claim: str, passage: str) -> Tuple[str, Dict[str, Any]]:
    """Classify a retrieved passage as support, refutation, or neutral evidence."""
    classifier = _get_pipeline()
    model_name = os.getenv("FACTVALIDATOR_NLI_MODEL", DEFAULT_NLI_MODEL).strip()
    if classifier is None:
        return detect_stance(claim, passage), {
            "method": "heuristic-fallback",
            "model": None,
            "enabled": _nli_enabled(),
            "reason": _LOAD_ERROR or "nli-disabled",
```

```
        }

    try:
        results = classifier({"text": passage, "text_pair": claim}, truncation=True)
        scores = results[0] if results and isinstance(results[0], list) else results
        best = max(scores, key=lambda item: float(item.get("score") or 0.0))
        relation = _map_label(str(best.get("label") or ""))
        return relation, {
            "method": "mnli",
            "model": model_name,
            "enabled": True,
            "label": best.get("label"),
            "score": round(float(best.get("score") or 0.0), 4),
        }
    except Exception as exc: # pragma: no cover - model/runtime dependent
        return detect_stance(claim, passage), {
            "method": "heuristic-fallback",
            "model": model_name,
            "enabled": True,
            "reason": f"inference-failed:{exc.__class__.__name__}",
        }
```

D.14 Prototype: Robustness Evaluation Reference

This is the exact module executed to produce Section 9.6.

```
from __future__ import annotations

from copy import deepcopy
from hashlib import sha256
from typing import Any, Dict, List

from app.analysis_features import decompose_claim, enrich_evidence
from app.evidence_graph import adjudicate_graph, build_evidence_graph
from app.evidence_independence import cluster_evidence
from app.graph_auditor import audit_evidence_graph
from app.relation_classifier import classify_relation


DECISIVE = {"SUPPORTED", "REFUTED"}


def _default_evidence(item: Dict[str, Any]) -> List[Dict[str, Any]]:
    passage = str(item.get("passage") or "").strip()
    if not passage:
        return []
    return [{
        "url": str(item.get("url") or f"https://seed.example/{item.get('id', 'item')}").strip(),
        "title": str(item.get("title") or "Frozen evidence passage"),
        "snippet": passage,
        "passage": passage,
        "content_hash": sha256(passage.encode("utf-8")).hexdigest(),
        "retrieval_status": "retrieved",
        "domain": str(item.get("domain") or "seed.example"),
        "base_domain": str(item.get("base_domain") or "seed.example"),
        "domain_score": int(item.get("domain_score") or 80),
```

```
        "overlap": 0,
    }]


def _prepare_evidence(claim: str, raw_evidence: List[Dict[str, Any]]) -> List[Dict[str, Any]]:
    profile = decompose_claim(claim)
    prepared: List[Dict[str, Any]] = []
    for item in deepcopy(raw_evidence):
        passage = str(item.get("passage") or item.get("snippet") or "").strip()
        if not passage:
            continue
        item.setdefault("snippet", passage)
        item.setdefault("passage", passage)
        item.setdefault("content_hash", sha256(passage.encode("utf-8")).hexdigest())
        item.setdefault("retrieval_status", "retrieved")
        item.setdefault("domain", "unknown.example")
        item.setdefault("base_domain", item["domain"])
        item.setdefault("domain_score", 50)
        item.setdefault("overlap", 0)
        enriched = enrich_evidence(claim, profile, item)
        stance, metadata = classify_relation(claim, passage)
        enriched["stance"] = stance
        enriched["relation_classifier"] = metadata
        prepared.append(enriched)
    cluster_evidence(prepared)
    return prepared


def _graph_decision(graph: Dict[str, Any]) -> str:
    abstention, _ = adjudicate_graph(graph)
    if abstention:
        return abstention
    counts = dict(graph.get("relation_counts") or {})
    support = int(counts.get("SUPPORTS", 0))
    attack = int(counts.get("REFUTES", 0)) + int(counts.get("UNDERCUTS", 0))
    if support > attack:
        return "SUPPORTED"
    if attack > support:
        return "REFUTED"
    return "NEI"


def evaluate_case(item: Dict[str, Any], scenario_name: str, raw_evidence: List[Dict[str, Any]]) -> Dict[
     str, Any]:
    claim = str(item.get("claim") or "").strip()
    profile = decompose_claim(claim)
    evidence = _prepare_evidence(claim, raw_evidence)
    status = "ok" if evidence else "no_results"
    graph = build_evidence_graph(claim, list(profile.get("atomic_claims") or [claim]), evidence, status)
    graph["independence_clusters"] = cluster_evidence(evidence)
    graph_only = _graph_decision(graph)
    audit = audit_evidence_graph(graph, evidence, profile)
    audited = graph_only
    if audit["decision"] in {"NEI", "CONFLICTING"}:
        audited = audit["decision"]
    elif audit["decision"] == "HUMAN_REVIEW":
        audited = "HUMAN_REVIEW"

    return {
```

```
        "id": item.get("id"),
        "scenario": scenario_name,
        "claim": claim,
        "expected_verdict": item.get("expected_verdict"),
        "graph_only": graph_only,
        "full_audited_graph": audited,
        "audit": audit,
        "evidence_count": len(evidence),
        "independence_clusters": graph["independence_clusters"],
    }


def build_scenarios(item: Dict[str, Any]) -> Dict[str, List[Dict[str, Any]]]:
    """Use only frozen, explicitly supplied evidence for non-clean corruptions."""
    scenarios = {"clean": list(item.get("evidence") or _default_evidence(item))}
    for name, evidence in dict(item.get("corruptions") or {}).items():
        if isinstance(evidence, list):
            scenarios[str(name)] = evidence
    return scenarios


def selective_metrics(rows: List[Dict[str, Any]], system: str) -> Dict[str, float]:
    if not rows:
        return {"coverage": 0.0, "selective_risk": 0.0, "false_accept_rate": 0.0}
    decisions = [row[system] for row in rows]
    decisive = [row for row in rows if row[system] in DECISIVE]
    incorrect = [row for row in decisive if row[system] != row.get("expected_verdict")]
    return {
        "coverage": round(len(decisive) / len(rows), 4),
        "selective_risk": round(len(incorrect) / max(1, len(decisive)), 4),
        "false_accept_rate": round(len(incorrect) / len(rows), 4),
    }
```

D.15 Prototype: Focused Graph Tests

```
from unittest.mock import AsyncMock, patch

import pytest
from httpx import ASGITransport, AsyncClient

from app.evidence_graph import adjudicate_graph, build_evidence_graph
from app.evidence_independence import cluster_evidence
from app.graph_auditor import audit_evidence_graph
from app.main import app, clean_text_for_claims, extract_claim_candidates, is_public_http_url,
    select_evidence_passage


def _passage(stance: str, quality: float = 85.0):
    return {
        "url": f"https://example.org/{stance}",
        "passage": f"The claim is {'false' if stance == 'refute' else 'confirmed'}.",
        "content_hash": stance,
        "retrieval_status": "retrieved",
        "stance": stance,
        "entity_match": True,
        "numeric_match": True,
```

```
        "directness_score": 0.8,
        "quality_score": quality,
    }


def test_short_claims_and_dates_are_retained():
    claim = "The Earth is flat."
    assert "March 11, 2020" in clean_text_for_claims("March 11, 2020: WHO made an announcement.")
    assert claim in extract_claim_candidates(claim, max_claims=1)


def test_passage_selection_retains_neighboring_context():
    text = (
        "The report describes the annual survey methodology in detail. "
        "The survey found that vaccination reduced hospitalization by 90 percent. "
        "The result applies only to the studied adult population."
    )
    passage = select_evidence_passage("Vaccination reduced hospitalization by 90 percent.", text)

    assert "annual survey methodology" in passage
    assert "reduced hospitalization by 90 percent" in passage
    assert "only to the studied adult population" in passage


@pytest.mark.asyncio
async def test_private_urls_are_rejected_before_fetching():
    assert await is_public_http_url("http://127.0.0.1:8000/admin") is False
    assert await is_public_http_url("http://localhost:8000/admin") is False


def test_graph_marks_unresolved_direct_conflict():
    graph = build_evidence_graph(
        "The Earth is flat.",
        ["The Earth is flat."],
        [_passage("support"), _passage("refute")],
        "ok",
    )
    verdict, reasons = adjudicate_graph(graph)

    assert graph["relation_counts"]["SUPPORTS"] == 1
    assert graph["relation_counts"]["REFUTES"] == 1
    assert verdict == "CONFLICTING"
    assert reasons


def test_auditor_marks_duplicate_decisive_evidence_for_human_review():
    evidence = [
        {**_passage("support"), "url": "https://example.org/a", "domain": "example.org", "base_domain": "
            example.org"},
        {**_passage("support"), "url": "https://example.org/b", "domain": "example.org", "base_domain": "
            example.org"},
    ]
    clusters = cluster_evidence(evidence)
    graph = build_evidence_graph("The Earth is flat.", ["The Earth is flat."], evidence, "ok")
    graph["independence_clusters"] = clusters
    audit = audit_evidence_graph(graph, evidence, {})

    assert len(clusters) == 1
    assert audit["decision"] == "HUMAN_REVIEW"
```

```
    assert any("correlated" in violation for violation in audit["violations"])


def test_auditor_explains_empty_evidence_abstention():
    graph = build_evidence_graph("A claim.", ["A claim."], [], "no_results")
    audit = audit_evidence_graph(graph, [], {})

    assert audit["decision"] == "NEI"
    assert "No fetched evidence passage" in audit["violations"][0]


@pytest.mark.asyncio
async def test_analyze_records_passage_provenance_and_conflict():
    results = [
        {"title": "Claim confirmed", "link": "https://example.org/support", "snippet": "A result snippet."},
        {"title": "Claim rejected", "link": "https://example.org/refute", "snippet": "Another result snippet."},
    ]

    async def passage(url: str, claim: str):
        if "support" in url:
            return "The claim that the Earth is flat is confirmed.", url, "retrieved"
        return "The claim that the Earth is flat is false.", url, "retrieved"

    with patch("app.main.SERPAPI_API_KEY", "test"), patch(
        "app.main.serpapi_search", new=AsyncMock(return_value=results)
    ), patch("app.main.fetch_evidence_passage", side_effect=passage), patch("app.main.get_cache", return_value=None):
        async with AsyncClient(transport=ASGITransport(app=app), base_url="http://test") as client:
            response = await client.post("/analyze", json={"text": "The Earth is flat.", "max_claims": 1})

    assert response.status_code == 200
    claim = response.json()["claims"][0]
    assert claim["verdict"] == "CONFLICTING"
    assert claim["evidence_graph"]["decision_basis"] == "fetched_passages_only"
    assert claim["graph_audit"]["version"] == "graph-auditor-v1"
    assert all(item["content_hash"] for item in claim["evidence"])
    assert all(item["relation_classifier"]["method"] == "heuristic-fallback" for item in claim["evidence"])
    manifest = claim["retrieval_manifest"]
    assert manifest["version"] == "retrieval-manifest-v1"
    assert manifest["search_results"] == [
        {"rank": 1, "title": "Claim confirmed", "url": "https://example.org/support", "snippet": "A result snippet."},
        {"rank": 2, "title": "Claim rejected", "url": "https://example.org/refute", "snippet": "Another result snippet."},
    ]
    assert manifest["manifest_hash"] and manifest["search_results_hash"]
```

D.16 Prototype: Robustness Tests

```
from app.robustness_evaluation import build_scenarios, evaluate_case, selective_metrics


def _evidence(passage: str, url: str) -> dict:
    return {
```

```
        "url": url,
        "domain": "example.org",
        "base_domain": "example.org",
        "domain_score": 90,
        "passage": passage,
    }


def test_explicit_missing_evidence_corruption_forces_audited_abstention():
    item = {
        "id": "missing-1",
        "claim": "The study reported a 20 percent reduction in hospital admissions.",
        "expected_verdict": "SUPPORTED",
        "evidence": [_evidence("The study reported a 20 percent reduction in hospital admissions.", "https
            ://example.org/report")],
        "corruptions": {"missing_evidence": []},
    }
    scenarios = build_scenarios(item)
    missing = evaluate_case(item, "missing_evidence", scenarios["missing_evidence"])

    assert set(scenarios) == {"clean", "missing_evidence"}
    assert missing["full_audited_graph"] == "NEI"
    assert any("No fetched evidence passage" in reason for reason in missing["audit"]["violations"])


def test_duplicate_corruption_is_reported_and_metrics_are_available():
    item = {
        "id": "duplicate-1",
        "claim": "The study reported a 20 percent reduction in hospital admissions.",
        "expected_verdict": "SUPPORTED",
        "evidence": [_evidence("The study reported a 20 percent reduction in hospital admissions.", "https
            ://example.org/a")],
        "corruptions": {
            "duplicate_reporting": [
                _evidence("The study reported a 20 percent reduction in hospital admissions.", "https://
                    example.org/a"),
                _evidence("The study reported a 20 percent reduction in hospital admissions.", "https://
                    example.org/b"),
            ]
        },
    }
    row = evaluate_case(item, "duplicate_reporting", build_scenarios(item)["duplicate_reporting"])
    metrics = selective_metrics([row], "full_audited_graph")

    assert row["full_audited_graph"] == "HUMAN_REVIEW"
    assert any("correlated" in reason for reason in row["audit"]["violations"])
    assert set(metrics) == {"coverage", "selective_risk", "false_accept_rate"}
```

Appendix E

Reproducibility Artifacts and Verification Listings

This appendix preserves the executable mechanisms that turn the frozen claim-level predictions into the tables and statistical statements reported in Chapter 6. The listings are included as auditable research material: they show the exact interval, paired-test, multiple-comparison, hash-validation, cache-validation, and continuous-integration logic used for the final audited repository snapshot. The authoritative machine-readable outputs are stored under `data/benchmarks/results_5000/`.

E.1 Repository Snapshot and Artifact Availability

Every path in this appendix is relative to the repository root. The complete set is tracked on `main`: support scripts, dependency specifications, generated statistics, and the audit report. The final audit report records source commit `a6bbafc`, the clean tracked snapshot executed before the generated report itself was committed. No release tag is asserted.

Table E.1: Verified thesis-support artifacts on `main`.

Artifact	Repository-relative path
Reproducibility audit	`data/benchmarks/results_5000/reproducibility_audit_report.json`
Statistics generator	`services/api/Scripts/run_thesis_statistics.py`
Artifact validator	`services/api/Scripts/validate_thesis_artifacts.py`

Artifact	Repository-relative path
Manifest builder	`services/api/Scripts/build_thesis_run_manifest.py`
Cache validator	`services/api/Scripts/validate_cache.py`
Operational projection	`services/api/Scripts/run_operational_projection.py`
Direct dependencies	`services/api/requirements.in`
Hash-pinned environment	`services/api/requirements.lock`

E.2 Live API Orchestration

The following listing covers the principal request orchestration, URL and domain handling, retrieval coordination, and application-level analysis functions. It documents the live implementation and must not be read as the algorithm executed by the 5,000-claim proxy.

```
from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse
from pydantic import BaseModel, validator, ValidationError
from typing import Optional, List, Literal, Dict, Any, Tuple
from datetime import datetime
import json
from urllib.parse import urljoin, urlparse
import os
import re
import asyncio
import time
from pathlib import Path
import ipaddress
import socket
from hashlib import sha256

import httpx
import trafilatura
from dotenv import load_dotenv

# Load environment variables before importing modules that evaluate env at import time.
load_dotenv(dotenv_path=Path(__file__).resolve().parents[1] / ".env")

import nltk
from nltk.tokenize import sent_tokenize

from app.analysis_features import (
    decompose_claim,
    determine_verdict,
    enrich_evidence,
    normalize_claim_key,
```

```
)
from app.credibility import base_domain as credibility_base_domain
from app.credibility import score_domain_rubric
from app.reflective import reflective_analysis, generate_faithful_correction
from app.semantic_retrieval import semantic_rerank
from app.sentiment import analyze_sentiment, estimate_bias_risk,
     calculate_sentiment_misinformation_adjustment, get_sentiment_summary
from app.source_routes import router as source_router
from app.storage import (
    save_run,
    list_runs,
    get_run,
    export_runs,
    get_claim_memory,
    save_claim_memory,
    init_db,
)
from app.debate import llm_debate_verdict, llm_final_judge
from app.logger import log_analyze_start, log_analyze_complete, log_debate_started, log_debate_error
from app.config import Config
from app.cache import get_cache
from app.security import ollama_health, rate_limiter
from app.evidence_graph import build_evidence_graph, adjudicate_graph
from app.relation_classifier import classify_relation
from app.retrieval_manifest import MANIFEST_VERSION, build_retrieval_manifest
from app.evidence_independence import cluster_evidence
from app.graph_auditor import audit_evidence_graph



SERPAPI_API_KEY = os.getenv("SERPAPI_API_KEY", "").strip()


class EvidenceSearchError(RuntimeError):
    pass


def _ensure_nltk():
    try:
        nltk.data.find("tokenizers/punkt")
    except Exception:
        try:
            nltk.download("punkt", quiet=True)
        except Exception:
            pass

    try:
        nltk.data.find("tokenizers/punkt_tab/english.pickle")
    except Exception:
        try:
            nltk.download("punkt_tab", quiet=True)
        except Exception:
            pass


_ensure_nltk()

app = FastAPI(title="Fact Validator API", version="0.8.2")

```

```
# Initialize database on startup
init_db()

# Add CORS middleware FIRST (before other middleware)
app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "http://localhost:3000",
        "http://127.0.0.1:3000",
        "http://localhost:3001",
        "http://127.0.0.1:3001",
        "http://192.168.0.106:3000",
        "http://192.168.0.106:3001",
    ],
    allow_credentials=True,
    allow_methods=["GET", "POST", "PUT", "DELETE", "OPTIONS"],
    allow_headers=["*"],
    max_age=600,
)

# Add rate limiting middleware (skip OPTIONS for CORS preflight)
@app.middleware("http")
async def rate_limit_middleware(request: Request, call_next):
    # Skip rate limiting for OPTIONS requests (CORS preflight)
    if request.method == "OPTIONS":
        return await call_next(request)

    if Config.FEATURE_RATE_LIMITING:
        client_ip = request.client.host if request.client else "unknown"
        if not rate_limiter.is_allowed(client_ip):
            return JSONResponse(
                status_code=429,
                content={"detail": "Rate limit exceeded. Max 100 requests per minute."}
            )
    response = await call_next(request)
    return response

app.include_router(source_router)


PROJECT_ROOT = Path(__file__).resolve().parents[3]
DOCS_DIR = PROJECT_ROOT / "docs"
BENCHMARK_RESULTS_DIR = PROJECT_ROOT / "data" / "benchmarks" / "results"


def _load_json_report(file_path: Path, fallback: Dict[str, Any]) -> Dict[str, Any]:
    try:
        with file_path.open("r", encoding="utf-8") as f:
            return json.load(f)
    except Exception:
        return fallback


def _get_model_fields(model_class):
    """Get model fields from Pydantic v1 or v2 models."""
    if hasattr(model_class, 'model_fields'):
        return model_class.model_fields
    elif hasattr(model_class, '__fields__'):
        return model_class.__fields__
```

```
    return {}


# ----------------------------
# Models
# ----------------------------

class AnalyzeRequest(BaseModel):
    url: Optional[str] = None
    text: Optional[str] = None
    mode: Literal["live", "snapshot"] = "live"
    verifier: Literal["baseline", "debate"] = "baseline"

    # optional control knobs
    enable_weighted_confidence: bool = False
    min_source_score: int = 50
    require_independent_domains: bool = True
    min_overlap: int = 6
    max_sources_per_base_domain: int = 2

    max_claims: int = 6
    max_evidence_per_claim: int = 5
    max_debate_claims: int = 2
    enable_reflective_abstention: bool = True
    enable_faithful_correction: bool = True

    @validator("url")
    @classmethod
    def validate_url(cls, v: Optional[str]) -> Optional[str]:
        if v and len(v) > Config.MAX_INPUT_URL_LENGTH:
            raise ValueError(f"URL exceeds max length of {Config.MAX_INPUT_URL_LENGTH}")
        return v

    @validator("text")
    @classmethod
    def validate_text(cls, v: Optional[str]) -> Optional[str]:
        if v and len(v) > Config.MAX_INPUT_TEXT_LENGTH:
            raise ValueError(f"Text exceeds max length of {Config.MAX_INPUT_TEXT_LENGTH}")
        return v

    @validator("max_claims")
    @classmethod
    def validate_max_claims(cls, v: int) -> int:
        if v > Config.MAX_CLAIMS_HARD_LIMIT:
            raise ValueError(f"max_claims exceeds limit of {Config.MAX_CLAIMS_HARD_LIMIT}")
        if v < 1:
            raise ValueError("max_claims must be at least 1")
        return v

    @validator("max_evidence_per_claim")
    @classmethod
    def validate_max_evidence(cls, v: int) -> int:
        if v > Config.MAX_EVIDENCE_PER_CLAIM_LIMIT:
            raise ValueError(f"max_evidence_per_claim exceeds limit of {Config.MAX_EVIDENCE_PER_CLAIM_LIMIT
                }")
        if v < 1:
            raise ValueError("max_evidence_per_claim must be at least 1")
        return v

```

```
208
209 class EvidenceItem(BaseModel):
210     url: str
211     title: Optional[str] = None
212     snippet: str
213     passage: Optional[str] = None
214     content_hash: Optional[str] = None
215     retrieval_status: Optional[Literal["retrieved", "fetch_failed", "not_fetched"]] = None
216     retrieved_at_utc: Optional[str] = None
217     domain: str
218     domain_score: int
219     overlap: int = 0
220     semantic_score: Optional[float] = None
221     quality_score: Optional[float] = None
222     stance: Optional[Literal["support", "refute", "neutral"]] = None
223     source_type: Optional[str] = None
224     primary_source: Optional[bool] = None
225     primary_source_reason: Optional[str] = None
226     published_year: Optional[int] = None
227     recency_score: Optional[float] = None
228     directness_score: Optional[float] = None
229     quote_grounded: Optional[bool] = None
230     expertise_match: Optional[float] = None
231     numeric_match: Optional[bool] = None
232     entity_match: Optional[bool] = None
233     manipulation_flags: Optional[List[str]] = None
234     independence_cluster: Optional[str] = None
235     independence_reason: Optional[str] = None
236
237
238 class ClaimResult(BaseModel):
239     claim_text: str
240     verdict: Literal["SUPPORTED", "REFUTED", "NEI", "CONFLICTING"]
241     confidence: float
242     evidence: List[EvidenceItem]
243     debate_summary: Optional[str] = None
244     structured_verdict: Optional[str] = None
245     uncertainty_reasons: Optional[List[str]] = None
246     needs_human_review: Optional[bool] = None
247     human_review_reason: Optional[str] = None
248     claim_profile: Optional[Dict[str, Any]] = None
249
250     # optional Step 11 outputs
251     adjusted_verdict: Optional[Literal["SUPPORTED", "REFUTED", "NEI"]] = None
252     adjusted_confidence: Optional[float] = None
253     evidence_summary: Optional[Dict[str, Any]] = None
254     reflective: Optional[Dict[str, Any]] = None
255     faithful_correction: Optional[Dict[str, Any]] = None
256     evidence_graph: Optional[Dict[str, Any]] = None
257     retrieval_manifest: Optional[Dict[str, Any]] = None
258     graph_audit: Optional[Dict[str, Any]] = None
259
260
261 class AnalyzeResponse(BaseModel):
262     input_type: Literal["url", "text"]
263     domain: Optional[str] = None
264     extracted_text_chars: int
265     extracted_text_preview: str
266     domain_score: int
```

```
    domain_label: Literal["HIGH", "MEDIUM", "LOW"]
    final_misinformation_likelihood: float
    claims: List[ClaimResult]
    timestamp_utc: str
    metadata: Dict[str, Any]


# ----------------------------
# Helpers
# ----------------------------

def normalize_url(u: str) -> str:
    u = (u or "").strip()
    if not u:
        return ""
    if "://" not in u:
        u = "https://" + u
    return u


def extract_domain(url: str) -> Optional[str]:
    try:
        u = normalize_url(url)
        if not u:
            return None
        parsed = urlparse(u)
        host = (parsed.netloc or "").lower()
        if host.startswith("www."):
            host = host[4:]
        return host or None
    except Exception:
        return None


def domain_from_any_url(url: str) -> str:
    return extract_domain(url) or ""


def base_domain(domain: str) -> str:
    return credibility_base_domain(domain)


def is_blocked_domain(domain: str) -> bool:
    bd = base_domain((domain or "").lower())
    blocked = {
        "facebook.com", "x.com", "twitter.com", "tiktok.com",
        "instagram.com", "reddit.com", "pinterest.com",
        "worldarticledatabase.com", "wecanfigurethisout.org",
    }
    return bd in blocked


async def fetch_html(url: str) -> Tuple[str, str]:
    current_url = normalize_url(url)
    if not current_url:
        return "", ""

    try:
        headers = {
```

```
            "User-Agent": "FactValidatorBot/0.8.2 (thesis demo)",
            "Accept": "text/html,application/xhtml+xml",
        }
        timeout = httpx.Timeout(connect=20.0, read=30.0, write=20.0, pool=30.0)
        async with httpx.AsyncClient(
            headers=headers,
            follow_redirects=False,
            timeout=timeout,
            trust_env=False,
        ) as client:
            for _ in range(5):
                if not await is_public_http_url(current_url):
                    return "", ""
                r = await client.get(current_url)
                if r.is_redirect:
                    location = r.headers.get("location")
                    if not location:
                        return "", ""
                    current_url = urljoin(current_url, location)
                    continue
                if r.status_code >= 400:
                    return "", ""
                return str(r.url), r.text
    except Exception:
        return "", ""
    return "", ""


async def is_public_http_url(url: str) -> bool:
    parsed = urlparse(normalize_url(url))
    if parsed.scheme not in {"http", "https"} or not parsed.hostname:
        return False

    host = parsed.hostname
    if host.lower() == "localhost":
        return False
    try:
        addresses = await asyncio.get_running_loop().run_in_executor(
            None,
            socket.getaddrinfo,
            host,
            None,
        )
    except OSError:
        return False

    try:
        resolved = {ipaddress.ip_address(item[4][0]) for item in addresses}
    except ValueError:
        return False
    return bool(resolved) and all(
        not (
            address.is_private
            or address.is_loopback
            or address.is_link_local
            or address.is_multicast
            or address.is_reserved
            or address.is_unspecified
        )
```

```
        for address in resolved
    )


def extract_readable_text_from_html(html: str, url: str = "") -> str:
    try:
        extracted = trafilatura.extract(
            html,
            url=url or None,
            include_comments=False,
            include_tables=False,
            favor_recall=True,
        )
        return (extracted or "").strip()
    except Exception:
        return ""


def heuristic_claim_score(s: str) -> float:
    s = s.strip()
    if not s:
        return 0.0
    length = len(s)
    length_score = 1.0 - min(abs(length - 160) / 160.0, 1.0)
    has_number = any(ch.isdigit() for ch in s)
    number_bonus = 0.15 if has_number else 0.0
    has_relation = any(tok in s.lower() for tok in [" is ", " are ", " was ", " were ", " has ", " have ",
         " with "])
    relation_bonus = 0.10 if has_relation else 0.0
    score = 0.55 * length_score + number_bonus + relation_bonus
    return max(0.05, min(score, 0.95))


def clean_text_for_claims(text: str) -> str:
    t = (text or "").strip()
    t = re.sub(r"\bExplore Data\b", " ", t, flags=re.IGNORECASE)
    t = re.sub(r"\bResearch\s*&\s*Writing\b", " ", t, flags=re.IGNORECASE)
    t = re.sub(r"[ \t]+", " ", t)
    t = re.sub(r"\n{2,}", "\n", t)
    return t.strip()


def extract_claim_candidates(text: str, max_claims: int = 6) -> List[str]:
    text = clean_text_for_claims(text)
    if not text:
        return []

    blocks = [b.strip() for b in text.split("\n") if b.strip()]
    candidates: List[str] = []
    seen = set()

    boilerplate_markers = [
        "this topic page can be cited as",
        "published online at",
        "cite this work",
        "reuse this work",
        "license terms",
        "creative commons",
        "open access",
```

```
        "data produced by third parties",
        "the underlying data for this chart",
    ]

    for b in blocks[:250]:
        for s in sent_tokenize(b):
            s2 = " ".join(s.split()).strip()
            if len(s2) < 8 or len(s2) > 280:
                continue
            low = s2.lower()
            if any(m in low for m in boilerplate_markers):
                continue
            key = s2.lower()
            if key in seen:
                continue
            seen.add(key)
            candidates.append(s2)
            if len(candidates) >= 120:
                break
        if len(candidates) >= 120:
            break

    ranked = sorted(candidates, key=heuristic_claim_score, reverse=True)
    return ranked[:max_claims]


async def serpapi_search(query: str, num: int = 5) -> List[Dict[str, Any]]:
    if not SERPAPI_API_KEY:
        return []

    params = {
        "engine": "google",
        "q": query,
        "api_key": SERPAPI_API_KEY,
        "num": str(num),
        "hl": "en",
        "gl": "us",
        "no_cache": "true",
    }

    try:
        timeout = httpx.Timeout(connect=20.0, read=45.0, write=20.0, pool=45.0)
        async with httpx.AsyncClient(timeout=timeout, trust_env=False) as client:
            r = await client.get("https://serpapi.com/search", params=params)
            r.raise_for_status()
            data = r.json()
            organic = data.get("organic_results", []) or []
            out = []
            for item in organic[:num]:
                out.append(
                    {
                        "title": item.get("title"),
                        "link": item.get("link"),
                        "snippet": item.get("snippet") or "",
                    }
                )
            return out
    except Exception as exc:
        raise EvidenceSearchError("Search provider request failed.") from exc
```

```
502
503
504 def tokenize_for_overlap(s: str) -> List[str]:
505     s = (s or "").lower()
506     s = re.sub(r"[^a-z0-9\s]", " ", s)
507     toks = [t for t in s.split() if len(t) >= 4]
508     stop = {
509         "this", "that", "with", "from", "were", "have", "has", "been", "into", "also",
510         "their", "they", "them", "than", "more", "most", "such", "some", "many"
511     }
512     toks = [t for t in toks if t not in stop]
513     return toks[:80]
514
515
516 def compute_overlap(claim: str, snippet: str) -> int:
517     claim_toks = set(tokenize_for_overlap(claim))
518     if not claim_toks:
519         return 0
520     snip_toks = set(tokenize_for_overlap(snippet))
521     return len(claim_toks.intersection(snip_toks))
522
523
524 def select_evidence_passage(claim: str, text: str, max_chars: int = 1200) -> str:
525     sentences = [" ".join(sentence.split()).strip() for sentence in sent_tokenize(text or "")]
526     candidates = [sentence for sentence in sentences if len(sentence) >= 30]
527     if not candidates:
528         return " ".join((text or "").split())[:max_chars]
529     best_index = max(range(len(candidates)), key=lambda index: compute_overlap(claim, candidates[index]))
530     return " ".join(candidates[max(0, best_index - 1):min(len(candidates), best_index + 2)])[:max_chars]
531
532
533 async def fetch_evidence_passage(url: str, claim: str) -> Tuple[str, str, str]:
534     final_url, html = await fetch_html(url)
535     if not html:
536         return "", final_url or url, "fetch_failed"
537     passage = select_evidence_passage(claim, extract_readable_text_from_html(html, final_url or url))
538     if not passage:
539         return "", final_url or url, "fetch_failed"
540     return passage, final_url or url, "retrieved"
541
542
543 def baseline_verdict(
544     claim: str, evidence: List[EvidenceItem]
545 ) -> Tuple[Literal["SUPPORTED", "REFUTED", "NEI"], float, str]:
546     if not evidence:
547         return "NEI", 0.55, "No evidence retrieved."
548
549     claim_toks = set(tokenize_for_overlap(claim))
550     if not claim_toks:
551         return "NEI", 0.55, "Claim tokenization empty."
552
553     neg_cues = ["false", "hoax", "debunk", "misleading", "not true", "no evidence",
554                 "incorrect", "misinformation", "disinformation", "fabricated", "baseless"]
555     overlaps = []
556     for e in evidence:
557         ev_toks = set(tokenize_for_overlap(e.snippet))
558         overlap = len(claim_toks.intersection(ev_toks))
559         overlaps.append((overlap, e.domain_score, base_domain(e.domain), (e.snippet or "").lower()))
560
```

```
    # ---------------------------------------------------------------- #
    # REFUTATION: any credible source (score >= 65) contains a rebuttal cue
    # ---------------------------------------------------------------- #
    for ov, ds, bd, snip_low in overlaps:
        if ds >= 65 and any(cue in snip_low for cue in neg_cues):
            conf = min(0.90, 0.65 + ds / 1000) # scales slightly with credibility
            return "REFUTED", round(conf, 2), (
                f"Credible source '{bd}' (score {ds}) contains a refutation signal."
            )

    # ---------------------------------------------------------------- #
    # SUPPORT: 2+ independent credible domains corroborate
    # ---------------------------------------------------------------- #
    strong_support_domains = {bd for (ov, ds, bd, _) in overlaps if ds >= 65 and ov >= 5}
    if len(strong_support_domains) >= 2:
        return "SUPPORTED", 0.78, (
            "Two or more independent medium/high-credibility domains corroborate the claim."
        )

    # ---------------------------------------------------------------- #
    # SUPPORT: single HIGH-credibility source (score >= 80) with overlap >= 5
    # ---------------------------------------------------------------- #
    best_high = max(
        ((ov, ds, bd) for (ov, ds, bd, _) in overlaps if ds >= 80),
        key=lambda x: (x[0], x[1]),
        default=None,
    )
    if best_high and best_high[0] >= 5:
        ov, ds, bd = best_high
        conf = round(min(0.82, 0.60 + ds / 1000 + ov * 0.008), 2)
        return "SUPPORTED", conf, (
            f"High-credibility source '{bd}' (score {ds}) corroborates the claim."
        )

    # ---------------------------------------------------------------- #
    # SUPPORT: single MEDIUM-credibility source (score >= 65) with overlap >= 7
    # ---------------------------------------------------------------- #
    best_med = max(
        ((ov, ds, bd) for (ov, ds, bd, _) in overlaps if ds >= 65),
        key=lambda x: (x[0], x[1]),
        default=None,
    )
    if best_med and best_med[0] >= 7:
        ov, ds, bd = best_med
        conf = round(min(0.72, 0.52 + ds / 1000 + ov * 0.007), 2)
        return "SUPPORTED", conf, (
            f"Medium-credibility source '{bd}' (score {ds}) has strong keyword overlap with the claim."
        )

    # ---------------------------------------------------------------- #
    # NEI: confidence weighted by the best available source quality
    # ---------------------------------------------------------------- #
    best_any = max(overlaps, key=lambda x: (x[1], x[0]))
    nei_conf = round(min(0.58, 0.40 + best_any[1] / 1000), 2)
    return "NEI", nei_conf, "Evidence retrieved but insufficient strength for a definitive verdict."


def estimate_misinformation_likelihood(
    claims: List[ClaimResult],
```

```
    input_domain_score: int = 50,
) -> float:
    """
    Misinformation likelihood anchored to source credibility.

    Domain-score mapping (base likelihood before claim adjustments):
      score=100 →0.10 (very credible source)
      score=80 →0.26
      score=75 →0.30
      score=65 →0.38
      score=50 →0.50 (neutral / unknown)
      score=25 →0.70
      score=0 →0.90 (low-credibility source)
    """
    # Anchor: linear map of domain credibility →base misinformation prior
    base = round(0.90 - (min(max(input_domain_score, 0), 100) / 100.0) * 0.80, 4)

    if not claims:
        return float(max(0.05, min(base, 0.95)))

    adjustment = 0.0
    for c in claims:
        w = max(0.1, float(c.confidence)) # confidence-weighted adjustment
        if c.verdict == "REFUTED":
            adjustment += 0.12 * w
        elif c.verdict == "SUPPORTED":
            adjustment -= 0.09 * w
            # Extra bonus when supporting evidence comes from high-credibility sources
            if c.evidence:
                avg_ev_score = sum(
                    getattr(e, "domain_score", 0) for e in c.evidence
                ) / len(c.evidence)
                if avg_ev_score >= 75:
                    adjustment -= 0.04
        # NEI verdict: no adjustment -uncertainty already captured by the base

    return float(max(0.05, min(base + adjustment, 0.95)))


def derive_input_domain_signal(
    claims: List[ClaimResult],
    domain: Optional[str],
) -> Tuple[int, str, Dict[str, Any]]:
    """Derive a transparent source-quality signal without treating it as a verdict."""
    if domain:
        credibility = score_domain_rubric(domain)
        return int(credibility.score), str(credibility.label), {
            "source": "input_domain",
            "domain": domain,
            "reasons": credibility.reasons,
        }

    evidence_scores = [
        int(item.domain_score)
        for claim in claims
        for item in claim.evidence
        if item.domain_score is not None
    ]
    if evidence_scores:
```

```
        score = round(sum(evidence_scores) / len(evidence_scores))
        return score, _label(score), {
            "source": "evidence_aggregate",
            "evidence_items": len(evidence_scores),
        }
    return 50, "MEDIUM", {"source": "neutral_default", "reason": "no input domain or evidence sources"}


def _label(score: int) -> str:
    if score >= 80:
        return "HIGH"
    if score >= 50:
        return "MEDIUM"
    return "LOW"


def build_dashboard_summary(runs: List[Dict[str, Any]], limit: int) -> Dict[str, Any]:
    input_type_counts: Dict[str, int] = {}
    verifier_counts: Dict[str, int] = {}
    verdict_counts: Dict[str, int] = {
        "SUPPORTED": 0,
        "REFUTED": 0,
        "NEI": 0,
    }
    domain_counts: Dict[str, int] = {}
    likelihood_values: List[float] = []
    claims_analyzed = 0
    claims_requiring_human_review = 0

    for run in runs:
        input_type = str(run.get("input_type") or "unknown").lower()
        input_type_counts[input_type] = input_type_counts.get(input_type, 0) + 1

        verifier = str(run.get("verifier") or "unknown").lower()
        verifier_counts[verifier] = verifier_counts.get(verifier, 0) + 1

        domain = str(run.get("domain") or "").strip().lower()
        if domain:
            domain_counts[domain] = domain_counts.get(domain, 0) + 1

        response = run.get("response")
        if not isinstance(response, dict):
            continue

        like_raw = response.get("final_misinformation_likelihood")
        try:
            if like_raw is not None:
                likelihood_values.append(float(like_raw))
        except (TypeError, ValueError):
            pass

        claims = response.get("claims")
        if not isinstance(claims, list):
            continue

        for claim in claims:
            if not isinstance(claim, dict):
                continue
            claims_analyzed += 1
```

```
            if bool(claim.get("needs_human_review")):
                claims_requiring_human_review += 1

            verdict = str(claim.get("verdict") or "NEI").upper()
            verdict_counts[verdict] = verdict_counts.get(verdict, 0) + 1

    avg_likelihood = None
    if likelihood_values:
        avg_likelihood = round(sum(likelihood_values) / len(likelihood_values), 3)

    top_domains = [
        {"domain": domain, "count": count}
        for domain, count in sorted(domain_counts.items(), key=lambda kv: (-kv[1], kv[0]))[:8]
    ]

    return {
        "limit": limit,
        "total_runs": len(runs),
        "last_run_utc": runs[0].get("created_utc") if runs else None,
        "claims_analyzed": claims_analyzed,
        "claims_requiring_human_review": claims_requiring_human_review,
        "avg_misinformation_likelihood": avg_likelihood,
        "input_type_counts": input_type_counts,
        "verifier_counts": verifier_counts,
        "verdict_counts": verdict_counts,
        "top_domains": top_domains,
    }


# ----------------------------
# Routes
# ----------------------------

@app.get("/health")
def health():
    return {"status": "ok"}


@app.get("/health/deep")
async def health_deep():
    """Deep health check including Ollama availability."""
    ollama_available = await ollama_health.is_available()
    return {
        "status": "ok",
        "ollama_available": ollama_available,
        "debate_enabled": Config.FEATURE_DEBATE_MODE,
        "config": Config.get_all(),
    }


@app.get("/evaluation/benchmark")
def evaluation_benchmark():
    return _load_json_report(
        DOCS_DIR / "evaluation_benchmark.json",
        {"claims": []},
    )


@app.get("/evaluation/baselines")
```

```
def evaluation_baselines():
    return _load_json_report(
        BENCHMARK_RESULTS_DIR / "baseline_comparison_report.json",
        {"metadata": {}, "results": {}},
    )


@app.get("/evaluation/ablations")
def evaluation_ablations():
    return _load_json_report(
        BENCHMARK_RESULTS_DIR / "ablation_study_report.json",
        {"metadata": {}, "variants": {}},
    )


@app.get("/evaluation/comparative")
def evaluation_comparative():
    return _load_json_report(
        BENCHMARK_RESULTS_DIR / "comparative_analysis_report.json",
        {"metadata": {}, "ranking": [], "comparisons": [], "debate_lift": {}},
    )


@app.get("/evaluation/production-metrics")
def evaluation_production_metrics():
    return _load_json_report(
        BENCHMARK_RESULTS_DIR / "production_metrics_report.json",
        {
            "metadata": {},
            "latency": {},
            "throughput": {},
            "cost": {},
            "quality": {},
        },
    )


@app.get("/evaluation/explainability")
def evaluation_explainability():
    return _load_json_report(
        BENCHMARK_RESULTS_DIR / "explainability_demo_report.json",
        {"metadata": {}, "case_studies": []},
    )


@app.get("/evaluation/limitations")
def evaluation_limitations():
    return _load_json_report(
        BENCHMARK_RESULTS_DIR / "limitations_assessment_report.json",
        {"metadata": {}, "limitations": []},
    )


@app.get("/evaluation/reproducibility")
```

E.3 Exact Thesis Statistics Generator

This script reads the two authoritative prediction CSVs, checks claim alignment, constructs the full-proxy confusion matrix, computes class and dataset metrics, Wilson intervals, exact two-sided McNemar tests, paired bootstrap intervals with seed 42, Holm adjustments, paired risk differences, and matched-pair odds ratios.

```
"""Generate publication-safe statistics for the 5,000-claim proxy experiment.

The analysis treats correctness as paired binary data. It reports exact
two-sided McNemar tests, Holm-adjusted p-values, paired bootstrap confidence
intervals for accuracy differences, matched-pair odds ratios, confusion
matrices, and class/domain metrics.
"""

from __future__ import annotations

import argparse
import csv
import json
import math
from collections import defaultdict
from dataclasses import dataclass
from datetime import datetime, timezone
from pathlib import Path
from typing import Iterable

import numpy as np
from scipy.stats import binomtest

API_ROOT = Path(__file__).resolve().parents[1]
REPO_ROOT = API_ROOT.parents[1]
DEFAULT_RESULT_DIR = REPO_ROOT / "data" / "benchmarks" / "results_5000"
LABELS = ("SUPPORTED", "REFUTED", "NEI")


@dataclass(frozen=True)
class Prediction:
    system: str
    claim_id: str
    dataset: str
    gold: str
    predicted: str
    confidence: float

    @property
    def correct(self) -> bool:
        return self.gold == self.predicted


def _dataset_from_claim_id(claim_id: str) -> str:
    """Recover the frozen source dataset from its stable claim-ID prefix."""
    prefix = claim_id.split("-", 1)[0].strip().lower()
    names = {
        "fever": "FEVER",
        "liar": "LIAR",
        "scifact": "SciFact",
```

```
        "health": "PUBHEALTH_health_fact",
    }
    try:
        return names[prefix]
    except KeyError as exc:
        raise ValueError(f"Unrecognized dataset prefix in claim_id={claim_id!r}") from exc


def _parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Generate thesis statistics")
    parser.add_argument("--output-dir", default=str(DEFAULT_RESULT_DIR))
    parser.add_argument("--seed", type=int, default=42)
    parser.add_argument("--bootstrap-iterations", type=int, default=20000)
    return parser.parse_args()


def _load(path: Path, system_column: str) -> dict[str, dict[str, Prediction]]:
    systems: dict[str, dict[str, Prediction]] = defaultdict(dict)
    with path.open(newline="", encoding="utf-8") as stream:
        for row in csv.DictReader(stream):
            system = row[system_column]
            claim_id = row["claim_id"]
            if claim_id in systems[system]:
                raise ValueError(f"duplicate claim_id for {system}: {claim_id}")
            systems[system][claim_id] = Prediction(
                system=system,
                claim_id=claim_id,
                dataset=_dataset_from_claim_id(claim_id),
                gold=row["ground_truth_label"],
                predicted=row["predicted_label"],
                confidence=float(row.get("predicted_confidence", 0.0)),
            )
    return dict(systems)


def _accuracy(rows: Iterable[Prediction]) -> float:
    values = list(rows)
    return sum(row.correct for row in values) / len(values) if values else 0.0


def _wilson(correct: int, total: int, z: float = 1.96) -> tuple[float, float]:
    if total == 0:
        return (0.0, 0.0)
    p = correct / total
    denominator = 1.0 + z * z / total
    centre = p + z * z / (2.0 * total)
    margin = z * math.sqrt((p * (1.0 - p) + z * z / (4.0 * total)) / total)
    return ((centre - margin) / denominator, (centre + margin) / denominator)


def _class_metrics(rows: Iterable[Prediction], label: str) -> dict[str, float | int]:
    values = list(rows)
    tp = sum(r.gold == label and r.predicted == label for r in values)
    fp = sum(r.gold != label and r.predicted == label for r in values)
    fn = sum(r.gold == label and r.predicted != label for r in values)
    support = sum(r.gold == label for r in values)
    precision = tp / (tp + fp) if tp + fp else 0.0
    recall = tp / (tp + fn) if tp + fn else 0.0
    f1 = 2.0 * precision * recall / (precision + recall) if precision + recall else 0.0
```

```
    return {
        "label": label,
        "support": support,
        "true_positive": tp,
        "false_positive": fp,
        "false_negative": fn,
        "precision": precision,
        "recall": recall,
        "f1": f1,
    }


def _paired_bootstrap(
    full: list[Prediction],
    other: list[Prediction],
    iterations: int,
    seed: int,
) -> tuple[float, float]:
    rng = np.random.default_rng(seed)
    deltas = np.asarray([
        (1 if left.correct else 0) - (1 if right.correct else 0)
        for left, right in zip(full, other)
    ], dtype=np.int8)
    count = len(deltas)
    estimates = np.empty(iterations, dtype=np.float64)
    chunk_size = 500
    for start in range(0, iterations, chunk_size):
        stop = min(iterations, start + chunk_size)
        indices = rng.integers(0, count, size=(stop - start, count))
        estimates[start:stop] = deltas[indices].mean(axis=1)
    lower, upper = np.quantile(estimates, [0.025, 0.975], method="linear")
    return float(lower), float(upper)


def _holm_adjust(rows: list[dict]) -> None:
    ordered = sorted(enumerate(rows), key=lambda item: item[1]["p_value_unadjusted"])
    running_max = 0.0
    total = len(rows)
    for rank, (original_index, row) in enumerate(ordered):
        adjusted = min(1.0, (total - rank) * row["p_value_unadjusted"])
        running_max = max(running_max, adjusted)
        rows[original_index]["p_value_holm"] = running_max
        rows[original_index]["significant_holm_0_05"] = running_max < 0.05


def main() -> int:
    args = _parse_args()
    output_dir = Path(args.output_dir)
    output_dir.mkdir(parents=True, exist_ok=True)
    ablation_path = output_dir / "ablation_study_predictions.csv"
    baseline_path = output_dir / "baseline_comparison_predictions.csv"
    manifest_path = output_dir / "run_manifest.json"
    if manifest_path.is_file():
        generated_utc = json.loads(
            manifest_path.read_text(encoding="utf-8")
        ).get("generated_utc")
    else:
        generated_utc = None
    generated_utc = generated_utc or datetime.now(timezone.utc).isoformat()
```

```
    systems = _load(ablation_path, "variant")
    for name, rows in _load(baseline_path, "baseline").items():
        if name in systems:
            raise ValueError(f"duplicate system name across inputs: {name}")
        systems[name] = rows

    if "full_proxy" not in systems:
        raise ValueError("full_proxy is missing")
    anchor_ids = list(systems["full_proxy"])
    anchor_id_set = set(anchor_ids)
    validation: dict[str, dict[str, int | bool]] = {}
    for name, rows in systems.items():
        validation[name] = {
            "prediction_count": len(rows),
            "claim_ids_match_full_proxy": set(rows) == anchor_id_set,
        }
        if len(rows) != 5000 or set(rows) != anchor_id_set:
            raise ValueError(f"{name} does not align to the 5,000-claim anchor")

    full_rows = [systems["full_proxy"][claim_id] for claim_id in anchor_ids]
    confusion = [
        {
            "gold_label": gold,
            **{
                predicted: sum(r.gold == gold and r.predicted == predicted for r in full_rows)
                for predicted in LABELS
            },
        }
        for gold in LABELS
    ]
    with (output_dir / "confusion_matrix_full_proxy.csv").open(
        "w", newline="", encoding="utf-8"
    ) as stream:
        writer = csv.DictWriter(stream, fieldnames=["gold_label", *LABELS])
        writer.writeheader()
        writer.writerows(confusion)

    per_class = [_class_metrics(full_rows, label) for label in LABELS]
    with (output_dir / "per_class_metrics.csv").open(
        "w", newline="", encoding="utf-8"
    ) as stream:
        writer = csv.DictWriter(stream, fieldnames=list(per_class[0]))
        writer.writeheader()
        writer.writerows(per_class)

    system_metrics: list[dict] = []
    per_dataset: list[dict] = []
    for name, rows_by_id in systems.items():
        rows = [rows_by_id[claim_id] for claim_id in anchor_ids]
        correct = sum(row.correct for row in rows)
        lower, upper = _wilson(correct, len(rows))
        class_rows = [_class_metrics(rows, label) for label in LABELS]
        system_metrics.append(
            {
                "system": name,
                "n": len(rows),
                "accuracy": correct / len(rows),
                "accuracy_wilson_lower": lower,
```

```
                "accuracy_wilson_upper": upper,
                "macro_f1": sum(float(item["f1"]) for item in class_rows) / len(LABELS),
                "average_raw_confidence": sum(r.confidence for r in rows) / len(rows),
            }
        )
        datasets: dict[str, list[Prediction]] = defaultdict(list)
        for row in rows:
            datasets[row.dataset].append(row)
        for dataset, dataset_rows in sorted(datasets.items()):
            dataset_correct = sum(row.correct for row in dataset_rows)
            ds_lower, ds_upper = _wilson(dataset_correct, len(dataset_rows))
            per_dataset.append(
                {
                    "system": name,
                    "dataset": dataset,
                    "n": len(dataset_rows),
                    "accuracy": dataset_correct / len(dataset_rows),
                    "accuracy_wilson_lower": ds_lower,
                    "accuracy_wilson_upper": ds_upper,
                }
            )

    with (output_dir / "per_dataset_metrics.csv").open(
        "w", newline="", encoding="utf-8"
    ) as stream:
        writer = csv.DictWriter(stream, fieldnames=list(per_dataset[0]))
        writer.writeheader()
        writer.writerows(per_dataset)

    comparisons: list[dict] = []
    for index, name in enumerate(sorted(systems)):
        if name == "full_proxy":
            continue
        other_rows = [systems[name][claim_id] for claim_id in anchor_ids]
        full_wins = sum(left.correct and not right.correct for left, right in zip(full_rows, other_rows))
        full_losses = sum(not left.correct and right.correct for left, right in zip(full_rows, other_rows))
        discordant = full_wins + full_losses
        p_value = (
            float(binomtest(full_wins, discordant, 0.5, alternative="two-sided").pvalue)
            if discordant
            else 1.0
        )
        ci_lower, ci_upper = _paired_bootstrap(
            full_rows,
            other_rows,
            iterations=args.bootstrap_iterations,
            seed=args.seed + index,
        )
        comparisons.append(
            {
                "system": "full_proxy",
                "comparator": name,
                "n": len(anchor_ids),
                "full_accuracy": _accuracy(full_rows),
                "comparator_accuracy": _accuracy(other_rows),
                "paired_risk_difference": _accuracy(full_rows) - _accuracy(other_rows),
                "paired_bootstrap_lower": ci_lower,
                "paired_bootstrap_upper": ci_upper,
                "full_wins": full_wins,
```

```
                "full_losses": full_losses,
                "discordant_pairs": discordant,
                "matched_pair_odds_ratio": (full_wins + 0.5) / (full_losses + 0.5),
                "p_value_unadjusted": p_value,
            }
        )
    _holm_adjust(comparisons)

    with (output_dir / "paired_tests.csv").open(
        "w", newline="", encoding="utf-8"
    ) as stream:
        writer = csv.DictWriter(stream, fieldnames=list(comparisons[0]))
        writer.writeheader()
        writer.writerows(comparisons)

    report = {
        "metadata": {
            "generated_utc": generated_utc,
            "evaluation_type": "deterministic_proxy",
            "full_system_label": "full_proxy",
            "seed": args.seed,
            "bootstrap_iterations": args.bootstrap_iterations,
            "confidence_note": (
                "The proxy stores one heuristic raw confidence score. "
                "It does not store a multiclass probability vector, so a proper "
                "multiclass Brier score is not available."
            ),
        },
        "artifact_validation": validation,
        "system_metrics": system_metrics,
        "confusion_matrix_full_proxy": confusion,
        "per_class_metrics_full_proxy": per_class,
        "per_dataset_metrics": per_dataset,
        "paired_tests": comparisons,
    }
    (output_dir / "statistics_report.json").write_text(
        json.dumps(report, indent=2), encoding="utf-8"
    )

    selected = {
        row["comparator"]: row
        for row in comparisons
        if row["comparator"] in {"majority", "length", "ablate_debate"}
    }
    lines = [
        "# Thesis Statistics Summary",
        "",
        "The 5,000-claim experiment evaluates the deterministic FactValidator-Proxy,",
        "not the live SerpAPI/SentenceTransformer/Ollama application pipeline.",
        "",
        "## System metrics",
        "",
        "| System | Accuracy | Macro-F1 | Wilson 95% CI |",
        "|---|---:|---:|---:|",
    ]
    for row in sorted(system_metrics, key=lambda item: item["accuracy"], reverse=True):
        lines.append(
            f"| {row['system']} | {row['accuracy']:.4f} | {row['macro_f1']:.4f} | "
            f"[{row['accuracy_wilson_lower']:.4f}, {row['accuracy_wilson_upper']:.4f}] |"
```

```
        )
    lines.extend(
        [
            "",
            "## Selected exact paired tests",
            "",
            "| Comparison | Full wins | Full losses | Exact two-sided p | Holm p | Matched OR |",
            "|---|---:|---:|---:|---:|---:|",
        ]
    )
    for comparator in ("majority", "length", "ablate_debate"):
        row = selected[comparator]
        lines.append(
            f"| full_proxy vs {comparator} | {row['full_wins']} | {row['full_losses']} | "
            f"{row['p_value_unadjusted']:.5f} | {row['p_value_holm']:.5f} | "
            f"{row['matched_pair_odds_ratio']:.3f} |"
        )
    lines.extend(
        [
            "",
            "Unadjusted comparisons against majority and length are marginal and are not",
            "described as robust superiority after correction for multiple comparisons.",
            "The no-debate proxy is significantly better than the full proxy in the",
            "observed paired comparison; always-on proxy debate is therefore not supported.",
            "",
            "The confidence output is reported only as a raw-score calibration diagnostic.",
            "A proper multiclass Brier score requires prob_supported, prob_refuted, and",
            "prob_nei for every prediction.",
        ]
    )
    (output_dir / "statistics_summary.md").write_text(
        "\n".join(lines) + "\n", encoding="utf-8"
    )
    print(f"Wrote thesis statistics to {output_dir}")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

E.4 Frozen-Artifact Validator

The validator rejects missing manifest fields, split overlap, prediction-count errors, claim-ID mismatch, hash mismatch, and generated reports that differ from the frozen statistics.

```
"""Fail when frozen thesis artifacts are incomplete or internally inconsistent."""

from __future__ import annotations

import csv
import hashlib
import json
import re
from pathlib import Path

```

```
API_ROOT = Path(__file__).resolve().parents[1]
REPO_ROOT = API_ROOT.parents[1]
SPLIT_DIR = REPO_ROOT / "data" / "benchmarks" / "splits_5000"
RESULT_DIR = REPO_ROOT / "data" / "benchmarks" / "results_5000"
EXPECTED_SPLIT_COUNTS = {"train.json": 11986, "val.json": 2997, "test.json": 5000}
EXPECTED_DATASET_COUNTS = {
    "FEVER": 1829,
    "LIAR": 1490,
    "PUBHEALTH_health_fact": 1490,
    "SciFact": 191,
}
REQUIRED_MANIFEST_FIELDS = {
    "experiment_name",
    "evaluation_type",
    "git_commit",
    "generated_utc",
    "random_seed",
    "python_version",
    "platform",
    "train_claims",
    "validation_claims",
    "test_claims",
    "proxy_components",
    "live_components_not_executed",
    "input_files",
    "sha256",
}


def _sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as stream:
        for chunk in iter(lambda: stream.read(1024 * 1024), b""):
            digest.update(chunk)
    return digest.hexdigest()


def _normalize(text: str) -> str:
    return re.sub(r"\s+", " ", text.strip().lower())


def _load_split(path: Path) -> tuple[set[str], set[str]]:
    data = json.loads(path.read_text(encoding="utf-8"))
    claims = data.get("claims")
    if not isinstance(claims, list):
        raise ValueError(f"{path}: claims list missing")
    ids = {str(row.get("id") or row.get("source_id") or "") for row in claims}
    normalized = {_normalize(str(row.get("claim", ""))) for row in claims}
    if "" in ids or "" in normalized:
        raise ValueError(f"{path}: blank claim identifier or text")
    if len(ids) != len(claims):
        raise ValueError(f"{path}: duplicate claim identifiers")
    return ids, normalized


def _prediction_systems(path: Path, column: str) -> dict[str, set[str]]:
    systems: dict[str, set[str]] = {}
    counts: dict[str, int] = {}
    with path.open(newline="", encoding="utf-8") as stream:
```

```
        for row in csv.DictReader(stream):
            name = row[column]
            systems.setdefault(name, set()).add(row["claim_id"])
            counts[name] = counts.get(name, 0) + 1
    for name, ids in systems.items():
        if counts[name] != 5000 or len(ids) != 5000:
            raise ValueError(
                f"{path.name}:{name} has {counts[name]} rows and {len(ids)} unique IDs"
            )
    return systems


def _validate_per_dataset_metrics(path: Path, systems: set[str]) -> None:
    observed: dict[str, dict[str, int]] = {}
    with path.open(newline="", encoding="utf-8") as stream:
        for row in csv.DictReader(stream):
            system = row["system"]
            dataset = row["dataset"]
            if dataset in observed.setdefault(system, {}):
                raise ValueError(f"{path.name}: duplicate {system}/{dataset} row")
            observed[system][dataset] = int(row["n"])
    if set(observed) != systems:
        raise ValueError(f"{path.name}: system set differs from prediction artifacts")
    for system, counts in observed.items():
        if counts != EXPECTED_DATASET_COUNTS:
            raise ValueError(
                f"{path.name}:{system} dataset counts differ: {counts}"
            )


def main() -> int:
    split_ids: dict[str, set[str]] = {}
    split_text: dict[str, set[str]] = {}
    for filename, expected in EXPECTED_SPLIT_COUNTS.items():
        ids, normalized = _load_split(SPLIT_DIR / filename)
        if len(ids) != expected:
            raise ValueError(f"{filename}: expected {expected}, found {len(ids)}")
        split_ids[filename] = ids
        split_text[filename] = normalized

    filenames = list(EXPECTED_SPLIT_COUNTS)
    for index, left in enumerate(filenames):
        for right in filenames[index + 1 :]:
            id_overlap = split_ids[left] & split_ids[right]
            text_overlap = split_text[left] & split_text[right]
            if id_overlap or text_overlap:
                raise ValueError(
                    f"split overlap {left}/{right}: "
                    f"{len(id_overlap)} IDs, {len(text_overlap)} normalized claims"
                )

    systems = {}
    systems.update(
        _prediction_systems(
            RESULT_DIR / "ablation_study_predictions.csv", "variant"
        )
    )
    systems.update(
        _prediction_systems(
```

```
            RESULT_DIR / "baseline_comparison_predictions.csv", "baseline"
        )
    )
    anchor = systems.get("full_proxy")
    if not anchor:
        raise ValueError("full_proxy predictions missing")
    for name, ids in systems.items():
        if ids != anchor:
            raise ValueError(f"claim IDs differ for {name}")
    if anchor != split_ids["test.json"]:
        raise ValueError("prediction claim IDs differ from test split")

    manifest_path = RESULT_DIR / "run_manifest.json"
    manifest = json.loads(manifest_path.read_text(encoding="utf-8"))
    missing = REQUIRED_MANIFEST_FIELDS - set(manifest)
    if missing:
        raise ValueError(f"run_manifest.json missing fields: {sorted(missing)}")
    for relative, expected_hash in manifest["sha256"].items():
        path = REPO_ROOT / relative
        actual_hash = _sha256(path)
        if actual_hash != expected_hash:
            raise ValueError(f"hash mismatch: {relative}")

    required_outputs = [
        "statistics_report.json",
        "confusion_matrix_full_proxy.csv",
        "per_class_metrics.csv",
        "per_dataset_metrics.csv",
        "paired_tests.csv",
        "statistics_summary.md",
    ]
    missing_outputs = [name for name in required_outputs if not (RESULT_DIR / name).is_file()]
    if missing_outputs:
        raise ValueError(f"missing statistical outputs: {missing_outputs}")
    _validate_per_dataset_metrics(
        RESULT_DIR / "per_dataset_metrics.csv", set(systems)
    )

    print(
        f"Validated {len(systems)} systems, 5,000 aligned predictions each, "
        "disjoint splits, manifest hashes, and statistical outputs."
    )
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

E.5 Run-Manifest Builder

The manifest builder records the proxy/live boundary, environment, random seed, split counts, paths, and SHA-256 hashes of every authoritative split and prediction file.

```
"""Build the immutable-input manifest for the 5,000-claim proxy experiment."""

```

```python
from __future__ import annotations

import hashlib
import json
import platform
import subprocess
from datetime import datetime, timezone
from pathlib import Path

API_ROOT = Path(__file__).resolve().parents[1]
REPO_ROOT = API_ROOT.parents[1]
SPLIT_DIR = REPO_ROOT / "data" / "benchmarks" / "splits_5000"
RESULT_DIR = REPO_ROOT / "data" / "benchmarks" / "results_5000"


def _sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as stream:
        for chunk in iter(lambda: stream.read(1024 * 1024), b""):
            digest.update(chunk)
    return digest.hexdigest()


def _git(*args: str) -> str:
    try:
        return subprocess.check_output(
            ["git", *args], cwd=REPO_ROOT, text=True, stderr=subprocess.DEVNULL
        ).strip()
    except (OSError, subprocess.CalledProcessError):
        return "unknown"


def main() -> int:
    inputs = {
        "train": SPLIT_DIR / "train.json",
        "validation": SPLIT_DIR / "val.json",
        "test": SPLIT_DIR / "test.json",
    }
    predictions = {
        "ablation_predictions": RESULT_DIR / "ablation_study_predictions.csv",
        "baseline_predictions": RESULT_DIR / "baseline_comparison_predictions.csv",
    }
    for path in [*inputs.values(), *predictions.values()]:
        if not path.is_file():
            raise FileNotFoundError(path)

    manifest = {
        "experiment_name": "factvalidator_proxy_5000",
        "evaluation_type": "deterministic_proxy",
        "git_commit": _git("rev-parse", "HEAD"),
        "git_branch": _git("branch", "--show-current"),
        "working_tree_dirty": bool(_git("status", "--porcelain")),
        "generated_utc": datetime.now(timezone.utc).isoformat(),
        "random_seed": 42,
        "python_version": platform.python_version(),
        "platform": platform.platform(),
        "train_claims": 11986,
        "validation_claims": 2997,
        "test_claims": 5000,
```

```
        "proxy_components": {
            "lexical_model": True,
            "category_priors": True,
            "heuristic_semantic_signals": True,
            "deterministic_debate_rule": True,
            "quality_filter": True,
        },
        "live_components_not_executed": [
            "SerpAPI retrieval",
            "live domain credibility scoring",
            "SentenceTransformer reranking",
            "Ollama debate",
        ],
        "input_files": {
            key: str(path.relative_to(REPO_ROOT)).replace("\\", "/")
            for key, path in inputs.items()
        },
        "prediction_files": {
            key: str(path.relative_to(REPO_ROOT)).replace("\\", "/")
            for key, path in predictions.items()
        },
        "sha256": {
            str(path.relative_to(REPO_ROOT)).replace("\\", "/"): _sha256(path)
            for path in [*inputs.values(), *predictions.values()]
        },
    }
    output = RESULT_DIR / "run_manifest.json"
    output.write_text(json.dumps(manifest, indent=2), encoding="utf-8")
    print(f"Wrote {output}")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

E.6 Cache Validation and Quarantine

Runtime web cache is not benchmark evidence. This validator decodes each record as UTF-8, checks its JSON structure, reports malformed entries, and moves invalid records into quarantine instead of silently accepting them.

```
"""Validate runtime evidence-cache records and optionally quarantine invalid files.

Runtime cache records are not authoritative thesis evidence. This utility makes
cache corruption visible and prevents malformed records from being silently
accepted by the live application.

Usage:
  python services/api/Scripts/validate_cache.py
  python services/api/Scripts/validate_cache.py --quarantine
"""

from __future__ import annotations

import argparse
```

```
import json
import shutil
from datetime import datetime, timezone
from pathlib import Path
from typing import Any

API_ROOT = Path(__file__).resolve().parents[1]
REPO_ROOT = API_ROOT.parents[1]


def _parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Validate evidence cache JSON")
    parser.add_argument(
        "--cache-dir",
        default=str(REPO_ROOT / "data" / "cache"),
        help="Directory containing runtime cache JSON files",
    )
    parser.add_argument(
        "--quarantine",
        action="store_true",
        help="Move invalid files into cache-dir/quarantine",
    )
    parser.add_argument(
        "--report",
        default=str(REPO_ROOT / "data" / "cache_validation_report.json"),
        help="JSON report path",
    )
    return parser.parse_args()


def _validate_record(value: Any) -> list[str]:
    errors: list[str] = []
    if not isinstance(value, dict):
        return ["root must be a JSON object"]
    if not isinstance(value.get("ts"), (int, float)):
        errors.append("ts must be numeric")
    if not isinstance(value.get("results"), list):
        errors.append("results must be a list")
    return errors


def _quarantine(path: Path, quarantine_dir: Path) -> Path:
    quarantine_dir.mkdir(parents=True, exist_ok=True)
    destination = quarantine_dir / path.name
    counter = 1
    while destination.exists():
        destination = quarantine_dir / f"{path.stem}.{counter}{path.suffix}"
        counter += 1
    shutil.move(str(path), str(destination))
    return destination


def main() -> int:
    args = _parse_args()
    cache_dir = Path(args.cache_dir).resolve()
    quarantine_dir = cache_dir / "quarantine"
    report_path = Path(args.report).resolve()

    records: list[dict[str, Any]] = []
```

```
    for path in sorted(cache_dir.glob("*.json")):
        if path.name == "example_cache_record.json":
            continue
        status = "valid"
        errors: list[str] = []
        destination: str | None = None
        try:
            text = path.read_text(encoding="utf-8", errors="strict")
            value = json.loads(text)
            errors.extend(_validate_record(value))
        except UnicodeDecodeError as exc:
            errors.append(f"invalid UTF-8: {exc}")
        except json.JSONDecodeError as exc:
            errors.append(f"invalid JSON: {exc}")
        except OSError as exc:
            errors.append(f"read error: {exc}")

        if errors:
            status = "invalid"
            if args.quarantine and path.exists():
                destination = str(_quarantine(path, quarantine_dir))
                status = "quarantined"

        records.append(
            {
                "file": str(path),
                "status": status,
                "errors": errors,
                "quarantine_destination": destination,
            }
        )

    summary = {
        "total": len(records),
        "valid": sum(r["status"] == "valid" for r in records),
        "invalid": sum(r["status"] == "invalid" for r in records),
        "quarantined": sum(r["status"] == "quarantined" for r in records),
    }
    report = {
        "generated_utc": datetime.now(timezone.utc).isoformat(),
        "cache_dir": str(cache_dir),
        "quarantine_enabled": bool(args.quarantine),
        "summary": summary,
        "records": records,
    }
    report_path.parent.mkdir(parents=True, exist_ok=True)
    report_path.write_text(json.dumps(report, indent=2), encoding="utf-8")

    print(
        "Cache validation: "
        f"{summary['valid']} valid, {summary['invalid']} invalid, "
        f"{summary['quarantined']} quarantined"
    )
    print(f"Report: {report_path}")
    return 1 if summary["invalid"] else 0


if __name__ == "__main__":
    raise SystemExit(main())
```

E.7 Operational Scenario Generator

The operational report is generated from declared scenario assumptions. The listing makes explicit why the resulting throughput and cost numbers are projections rather than controlled load-test measurements.

```
"""Generate explicitly labelled operational scenario projections."""

from __future__ import annotations

import json
from datetime import datetime, timezone
from pathlib import Path

API_ROOT = Path(__file__).resolve().parents[1]
REPO_ROOT = API_ROOT.parents[1]
OUTPUT = (
    REPO_ROOT
    / "data"
    / "benchmarks"
    / "results_5000"
    / "operational_projection_report.json"
)


def main() -> int:
    baseline_latency = 8.2
    debate_latency = 72.0
    no_cache = 77.0
    with_cache = 22.0
    report = {
        "metadata": {
            "generated_utc": datetime.now(timezone.utc).isoformat(),
            "artifact_type": "operational_projection",
            "measurement_status": (
                "Scenario projection from stated assumptions; not a controlled "
                "concurrent load-test measurement."
            ),
        },
        "assumptions": {
            "baseline_seconds_per_claim": baseline_latency,
            "debate_seconds_per_claim": debate_latency,
            "claims_per_month": 1000,
            "monthly_usd_without_cache": no_cache,
            "monthly_usd_with_cache": with_cache,
        },
        "projections": {
            "baseline_claims_per_hour": 3600.0 / baseline_latency,
            "debate_claims_per_hour": 3600.0 / debate_latency,
            "debate_latency_ratio": debate_latency / baseline_latency,
            "monthly_savings_usd": no_cache - with_cache,
            "monthly_savings_fraction": (no_cache - with_cache) / no_cache,
        },
        "required_for_measured_claim": [
            "repeated warm-cache and cold-cache runs",
            "median, p90, and p95 latency",
            "declared concurrency",
            "failure and timeout counts",
```

```
            "hardware and provider request counts",
        ],
    }
    OUTPUT.parent.mkdir(parents=True, exist_ok=True)
    OUTPUT.write_text(json.dumps(report, indent=2), encoding="utf-8")
    print(f"Wrote {OUTPUT}")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

E.8 Reproducibility Audit

The committed audit records the branch and source commit, environment, dependency lock hash, artifact hashes, executed commands, complete test result, and artifact-validation result.

```
"""Generate the final thesis reproducibility audit from the correction branch."""

from __future__ import annotations

import argparse
import hashlib
import json
import os
import platform
import subprocess
from datetime import datetime, timezone
from pathlib import Path
from typing import Any

API_ROOT = Path(__file__).resolve().parents[1]
REPO_ROOT = API_ROOT.parents[1]
RESULT_DIR = REPO_ROOT / "data" / "benchmarks" / "results_5000"


def _parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Run thesis reproducibility audit")
    parser.add_argument(
        "--skip-tests",
        action="store_true",
        help="Record tests as not executed (intended only for quick local inspection)",
    )
    return parser.parse_args()


def _run(command: list[str], timeout: int = 300) -> dict[str, Any]:
    completed = subprocess.run(
        command,
        cwd=REPO_ROOT,
        text=True,
        stdout=subprocess.PIPE,
        stderr=subprocess.STDOUT,
        timeout=timeout,
```

```
        check=False,
    )
    return {
        "command": " ".join(command),
        "exit_code": completed.returncode,
        "output": completed.stdout[-20000:],
    }


def _git(*args: str) -> str:
    result = _run(["git", *args], timeout=30)
    return result["output"].strip() if result["exit_code"] == 0 else "unknown"


def _sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as stream:
        for chunk in iter(lambda: stream.read(1024 * 1024), b""):
            digest.update(chunk)
    return digest.hexdigest()


def _ram_bytes() -> int | None:
    try:
        import psutil

        return int(psutil.virtual_memory().total)
    except (ImportError, OSError):
        return None


def _gpu() -> str:
    try:
        result = subprocess.run(
            ["nvidia-smi", "--query-gpu=name", "--format=csv,noheader"],
            text=True,
            stdout=subprocess.PIPE,
            stderr=subprocess.DEVNULL,
            timeout=10,
            check=False,
        )
        value = result.stdout.strip()
        return value or "not detected"
    except (OSError, subprocess.TimeoutExpired):
        return "not detected"


def main() -> int:
    args = _parse_args()
    commands: list[dict[str, Any]] = []
    if args.skip_tests:
        test_result = {
            "command": "pytest services/api/tests -q",
            "exit_code": None,
            "output": "not executed (--skip-tests)",
        }
    else:
        test_result = _run(["pytest", "services/api/tests", "-q"], timeout=600)
    commands.append(test_result)
```

```
    validation = _run(
        ["python", "services/api/Scripts/validate_thesis_artifacts.py"],
        timeout=300,
    )
    commands.append(validation)

    tracked_artifacts = [
        REPO_ROOT / "services" / "api" / "requirements.lock",
        REPO_ROOT / "data" / "benchmarks" / "splits_5000" / "train.json",
        REPO_ROOT / "data" / "benchmarks" / "splits_5000" / "val.json",
        REPO_ROOT / "data" / "benchmarks" / "splits_5000" / "test.json",
        RESULT_DIR / "ablation_study_predictions.csv",
        RESULT_DIR / "baseline_comparison_predictions.csv",
        RESULT_DIR / "statistics_report.json",
    ]
    hashes = {
        str(path.relative_to(REPO_ROOT)).replace("\\", "/"): _sha256(path)
        for path in tracked_artifacts
        if path.is_file()
    }
    report = {
        "metadata": {
            "generated_utc": datetime.now(timezone.utc).isoformat(),
            "git_commit": _git("rev-parse", "HEAD"),
            "git_branch": _git("branch", "--show-current"),
            "git_tag_exact": _git("describe", "--tags", "--exact-match"),
            # Untracked local build outputs do not alter the committed source
            # snapshot being audited. Track only modifications to versioned files.
            "working_tree_dirty": bool(
                _git("status", "--porcelain", "--untracked-files=no")
            ),
        },
        "environment": {
            "python_version": platform.python_version(),
            "os": platform.platform(),
            "cpu": platform.processor() or os.environ.get("PROCESSOR_IDENTIFIER", "unknown"),
            "gpu": _gpu(),
            "ram_bytes": _ram_bytes(),
        },
        "dependency_lock": {
            "path": "services/api/requirements.lock",
            "sha256": hashes.get("services/api/requirements.lock"),
        },
        "artifact_hashes": hashes,
        "commands": commands,
        "test_result": {
            "exit_code": test_result["exit_code"],
            "passed": test_result["exit_code"] == 0,
            "summary_output": test_result["output"][-4000:],
        },
        "artifact_validation": {
            "exit_code": validation["exit_code"],
            "passed": validation["exit_code"] == 0,
            "summary_output": validation["output"][-4000:],
        },
    }
    RESULT_DIR.mkdir(parents=True, exist_ok=True)
    (RESULT_DIR / "reproducibility_audit_report.json").write_text(
        json.dumps(report, indent=2), encoding="utf-8"
```

```
    )
    summary = [
        "# Reproducibility Audit Summary",
        "",
        f"- Generated UTC: {report['metadata']['generated_utc']}",
        f"- Branch: `{report['metadata']['git_branch']}`",
        f"- Commit: `{report['metadata']['git_commit']}`",
        f"- Python: `{report['environment']['python_version']}`",
        f"- OS: `{report['environment']['os']}`",
        f"- CPU: `{report['environment']['cpu']}`",
        f"- GPU: `{report['environment']['gpu']}`",
        f"- RAM bytes: `{report['environment']['ram_bytes']}`",
        f"- Tests passed: **{report['test_result']['passed']}**",
        f"- Artifact validation passed: **{report['artifact_validation']['passed']}**",
        "",
        "## Executed commands",
        "",
    ]
    for item in commands:
        summary.append(f"- `{item['command']}` -> exit {item['exit_code']}")
    summary.extend(
        [
            "",
            "## Test output",
            "",
            "```text",
            test_result["output"][-4000:].strip(),
            "```",
            "",
            "The audit records artifact integrity and executed software checks. "
            "It does not convert proxy benchmark results into live-application results.",
        ]
    )
    (RESULT_DIR / "reproducibility_audit_summary.md").write_text(
        "\n".join(summary) + "\n", encoding="utf-8"
    )
    print(f"Wrote reproducibility audit to {RESULT_DIR}")
    return 0 if report["test_result"]["passed"] and report["artifact_validation"]["passed"] else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

E.9 Continuous-Integration Gate

When executed, the repository workflow installs the hash-pinned backend environment, executes the backend suite, regenerates thesis statistics, checks that committed reports are deterministic, validates the frozen artifacts, and builds the frontend. Its executable definition is preserved at `.github/workflows/ci.yml`; the path is recorded here instead of duplicating runner-specific comment encoding in the PDF.

E.10 Direct Backend Dependency Specification

The human-maintained direct dependency specification is reproduced below. The complete transitive environment, including hashes, is frozen separately in `services/api/requirements.lock`.

```
# Direct backend dependencies. Compile with pip-tools to requirements.lock.
fastapi==0.95.2
uvicorn[standard]==0.34.0
sqlmodel==0.0.30
httpx[http2]==0.28.1
trafilatura==2.0.0
lxml[html_clean]==6.0.2
lxml_html_clean==0.4.4
python-dotenv==1.0.0
tldextract==5.3.1
nltk==3.9.3
watchfiles==1.1.1
# huggingface-hub exposes this dependency only on supported x86_64 platforms.
# Pin it explicitly so a lock compiled on Windows is complete for Linux CI.
hf-xet==1.5.2
sentence-transformers==3.4.1
transformers==4.49.0
slowapi==0.1.9
python-json-logger==3.3.0
pydantic[email]==1.10.21
pytest==9.0.2
pytest-asyncio==1.3.0
pytest-mock==3.15.1
scipy==1.15.3
numpy==1.26.4
```

Appendix F

Glossary of Terms

Table F.1: Glossary of terms used throughout the thesis.

Term	Definition
Atomic claim	A single, independently verifiable factual statement produced by decomposing a longer input document.
Credibility prior	The explicit, rule-based domain score defined in Section 3.8, used before any learned scoring is applied.
Debate overhead	The latency multiplier introduced by enabling the optional Prover–Skeptic–Judge workflow (Section 3.7).
Evidence graph	The typed node/edge structure used by the Chapter 9 prototype to represent claims, passages, and their argumentative relations.
Frozen benchmark	A fixed, versioned test split whose claims, labels, prediction records, and hashes do not change between evaluation runs.
Independence cluster	A set of evidence items linked by shared domain or high textual similarity (Section 3.7), treated as one corroborating source rather than several.
NEI	"Not Enough Information" – the verdict returned when available evidence does not support a decisive SUPPORTED or REFUTED outcome.
Provenance	The recorded origin of a piece of evidence: URL, content hash, retrieval timestamp, and retrieval status.
Selective risk	The error rate computed only over the subset of claims a system chooses to decide on, as opposed to abstaining from (Section 3.6).

Term	Definition
Undercut	An evidentiary relation that weakens a claim's assumption, scope, or source without directly asserting its negation.

Appendix G

Summary of Mathematical Notation

Table G.1: Notation used across Chapters 3, 6, and 9.

Symbol	Meaning
$y_i, \hat{y}_i$	Gold and predicted label for claim i.
n, K	Test-set size; number of classes ($K = 3$).
$\hat{p}$	Observed accuracy, used in the confidence-interval formula (Section 3.2).
$\overline{\text{score}}$	Average heuristic proxy score on the 0–100 scale (Section 3.3).
B_m, M	The m-th confidence bin and total number of bins, used in the ECE definition (Section 3.3).
n_{01}, n_{10}	Discordant-pair counts used in McNemar's test (Section 3.4).
S_θ	The subset of claims on which a selective system chooses to decide, at threshold θ (Section 3.6).
$R(\theta)$	Selective risk at threshold θ (Section 3.6).
d_i, T_i	Base domain and token set of evidence item i, used in the independence-clustering rule (Section 3.7).
τ	Jaccard-similarity threshold for independence clustering ($\tau = 0.78$).
$\pi(p)$	Provenance tuple (u, h, t, s) for passage p (Section 9.3).
$G = (V, E)$	The typed evidence graph for one claim (Section 9.4).
$A(G)$	The audit function mapping a graph to $\{\text{PROCEED}, \text{NEI}, \text{CONFLICTING}, \text{HUMAN\_REVIEW}\}$ (Section 9.4).

Bibliography

[1] J. Thorne, A. Vlachos, C. Christodoulopoulos, and A. Mittal, "FEVER: A Large-Scale Dataset for Fact Extraction and Verification," in *Proc. NAACL-HLT*, 2018, pp. 809–819.

[2] W. Y. Wang, "Liar, Liar Pants on Fire: A New Benchmark Dataset for Fake News Detection," in *Proc. ACL*, 2017, pp. 422–426.

[3] D. Wadden, S. Lin, K. Lo, L. L. Wang, M. van Zuylen, A. Cohan, and H. Hajishirzi, "Fact or Fiction: Verifying Scientific Claims," in *Proc. EMNLP*, 2020.

[4] I.-C. Chern, S. Chern, S. Chen, W. Yuan, K. Feng, C. Zhou, J. He, G. Neubig, and P. Liu, "FacTool: Factuality Detection in Generative AI – A Tool-Augmented Framework for Multi-Task and Multi-Domain Scenarios," *arXiv:2307.13528*, 2023.

[5] K.-H. Huang, H. P. Chan, and H. Ji, "Zero-shot Faithful Factual Error Correction," *arXiv:2305.07982*, 2023.

[6] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems*, 2020.

[7] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On Calibration of Modern Neural Networks," in *Proc. ICML*, 2017.

[8] B. Wen et al., "Know Your Limits: A Survey of Abstention in Large Language Models," *arXiv:2407.18418*, 2024.

[9] Y. Du et al., "Improving Factuality and Reasoning in Language Models through Multi-Agent Debate," *arXiv:2305.14325*, 2023.

Ringraziamenti

Desidero ringraziare il mio relatore, il Prof. Roberto Pietrantuono, per la guida metodologica durante lo sviluppo di questa tesi, e il Dipartimento di Ingegneria Elettrica e delle Tecnologie dell'Informazione dell'Università degli Studi di Napoli Federico II per il contesto di ricerca in cui questo lavoro è stato svolto.